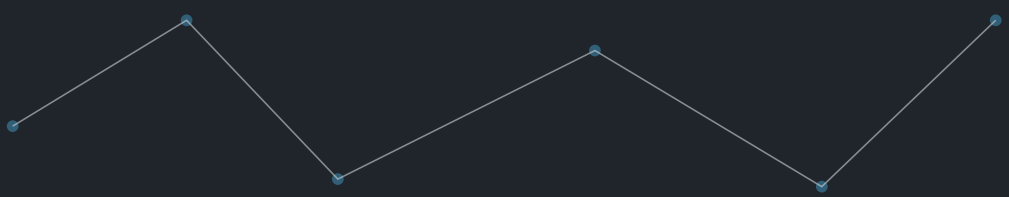
Build HTTP from Scratch

A 12-Week Journey from Python Sockets to a REST API



Mentoring Edition | Version 1.0

Build HTTP from Scratch

A 12-Week Journey from Python Sockets to a REST API

Mentoring Edition
November 2026

About this edition

This mentoring edition is a guided textbook and laboratory manual. It teaches the ideas beneath HTTP before asking the reader to build a server. The software is intentionally small enough to inspect completely. It is not offered as a production web server.

Build HTTP from Scratch
A 12-Week Journey from Python Sockets to a REST API
Mentoring Edition, Version 1.0
November 2026

The examples use Python's standard library. Protocol descriptions are based on the standards listed in the bibliography. Code may be used and modified for educational purposes.

How to use this book

This book is both a roadmap and a construction manual. The first seven chapters form a theory gate: do not rush through them simply because they do not yet create a server. A parser is much easier to design when you can explain what a byte stream is, where message boundaries come from, and which HTTP rules exist to prevent two recipients from disagreeing about a request.

Beginning with Part II, every major step has a frozen source snapshot under `code/stages`. Read a chapter, predict the behavior, run the snapshot, make a small change, and then compare your result with the next stage. Part III turns the reusable framework into a complete SQLite-backed Task API.

Learning objectives

By the end, you should be able to:

- narrate a request from a URL through DNS, TCP, HTTP, routing, and storage;
- explain why TCP reads do not correspond to HTTP messages;
- parse a strict, length-framed HTTP/1.1 subset safely;
- design request, response, routing, middleware, and server abstractions;
- implement and test a resource-oriented JSON API;
- state exactly why the finished server is educational rather than production-ready.

The weekly rhythm

Plan for six to eight focused hours each week: two or three hours of reading, two or three hours of code-along work, and one or two hours for exercises and reflection. Keep a lab notebook. For every surprising result, write the bytes you expected, the bytes you observed, and the rule that explains the difference.

Weeks	Focus	Deliverable
1–4	Network and HTTP theory	Annotated request journey and theory checkpoint
5–9	Socket server and framework	Tested `tinyhttp` package
10–12	SQLite REST API and hardening	Complete Task API and final assessment

Conventions

Wire messages use a monospaced block. The symbol `CRLF` means the two bytes carriage return (13) and line feed (10), written in Python as `b"\r\n"`. Commands use `python`; on Windows, `py` may be the correct launcher. Paths use forward slashes, even though Windows also accepts backslashes in many shell contexts.

> **Security boundary**
>
> Run the examples on `127.0.0.1`. Binding to `0.0.0.0` exposes the learning server to other machines that can reach your computer. Do not expose it to the public Internet.

Contents

How to use this book **iv**
The weekly rhythm . . . iv
Conventions . . . v

I Foundations: from electricity to HTTP semantics **1**

1 One request, end to end **2**
1.1 Begin with a URL . . . 2
1.2 The phases . . . 3
1.3 Client, server, and resource . . . 3
1.4 Where the server begins and ends . . . 4
1.5 Failure belongs to a phase . . . 4
1.6 Latency is accumulated waiting . . . 4
Chapter summary . . . 4
Review questions . . . 5

2 Bytes, encodings, and protocol layers **6**
2.1 A character is not a byte . . . 6
2.2 Framing creates messages . . . 7
2.3 Layers manage complexity . . . 7
2.4 Encapsulation and changing envelopes . . . 8
2.5 Packet boundaries are not message boundaries . . . 8
2.6 Numbers and byte order . . . 8
Chapter summary . . . 9
Review questions . . . 9

3 Addresses, ports, routing, and DNS **10**
3.1 IP identifies an interface location . . . 10
3.1.1 Address scopes that matter in the lab . . . 10
3.2 Ports identify transport endpoints . . . 11
3.3 Routing chooses the next hop . . . 11
3.4 NAT rewrites addressing context . . . 11
3.5 DNS maps names to data . . . 11
3.5.1 Why DNS answers vary . . . 12
3.6 Names return at the HTTP layer . . . 12
3.7 A disciplined diagnostic order . . . 12
Chapter summary . . . 13
Review questions . . . 13

4 TCP streams and the socket interface **14**
4.1 A reliable byte stream . . . 14
4.2 Establishing a connection . . . 14
4.3 Sequence, acknowledgment, and retransmission . . . 15
4.4 Two kinds of control . . . 15
4.5 Closing is a protocol too . . . 15
4.6 Sockets: the application boundary . . . 16
4.6.1 Blocking and timeouts . . . 16
4.7 Partial operations are ordinary . . . 16
4.8 Why not UDP? . . . 17
Chapter summary . . . 17
Review questions . . . 17

5 The HTTP semantic model **18**
5.1 Messages carry intentions and results . . . 18
5.2 URI, origin, resource, representation . . . 18
5.3 Methods are semantics, not route names . . . 19
5.4 Status codes describe the result . . . 19
5.5 Fields modify or describe messages . . . 20
5.6 Stateless does not mean no state . . . 20
5.7 Content versus representation . . . 20
Chapter summary . . . 20
Review questions . . . 21

6 HTTP/1.1 on the wire **22**
6.1 Message shape . . . 22
6.2 The request line . . . 22
6.3 Field parsing . . . 23
6.4 Where does the body end? . . . 23
6.5 Chunked transfer coding . . . 23
6.6 Response framing and special cases . . . 24
6.7 Persistent connections . . . 24
6.8 Limits are protocol behavior . . . 24
Chapter summary . . . 24
Review questions . . . 25

7 Web architecture, REST, and security boundaries 26
7.1 Intermediaries are normal . . . 26
7.2 Caching changes the path . . . 26
7.3 What TLS provides . . . 27
7.4 REST as an architectural style . . . 27
7.4.1 Model nouns before endpoints . . . 27
7.5 Input is hostile until proven otherwise . . . 27
7.6 Why newer HTTP versions exist . . . 28
7.7 The implementation contract . . . 29
Chapter summary . . . 29
Review questions . . . 29

II Building an HTTP server from raw sockets 31

8 The first server: one connection, one response 32
8.1 Prepare the laboratory . . . 32
8.2 Read the complete program . . . 32
8.2.1 Create and configure . . . 33
8.2.2 Bind and listen . . . 34
8.2.3 Accept and communicate . . . 34
8.3 Constructing the response . . . 34
8.4 Inspect rather than guess . . . 34
8.5 What this program does not know . . . 35
Chapter summary . . . 35
Review questions . . . 35

9 Receiving bytes without inventing boundaries 36
9.1 The tempting incorrect version . . . 36
9.2 The buffered snapshot . . . 36
9.3 End-of-stream is data about the stream . . . 38
9.4 Delimiter search and strict CRLF . . . 38
9.5 Timeouts belong beside byte limits . . . 38
9.6 Design the next buffer . . . 39
Chapter summary . . . 39
Review questions . . . 39

10 Building an incremental request parser 40
10.1 From a teaching parser to a strict parser . . . 40
10.2 The final parsing contract . . . 42
10.3 Read the complete final parser . . . 43
10.3.1 Tokens and line syntax . . . 47
10.3.2 Framing before content . . . 48
10.3.3 Targets and decoding . . . 48
10.4 Internal trust is earned in layers . . . 48
10.5 Parser tests as executable grammar examples . . . 48
Chapter summary . . . 49
Review questions . . . 49

11 Responses as values and bytes **50**
11.1 The response object . . . 50
11.2 Headers must be safe to serialize . . . 52
11.3 One serialization path . . . 52
11.4 HEAD and no-content statuses differ . . . 52
11.5 JSON and problem factories . . . 53
11.6 Test meaning and wire form separately . . . 53
Chapter summary . . . 53
Review questions . . . 53

12 Routing methods and resource paths **55**
12.1 A small first router . . . 55
12.2 Static routes are more specific . . . 57
12.3 Path match and method match are separate facts . . . 59
12.4 Path parameters are strings first . . . 60
12.5 Queries refine a resource request . . . 60
12.6 Routing is not authorization . . . 60
Chapter summary . . . 60
Review questions . . . 60

13 JSON, error boundaries, and middleware **62**
13.1 JSON is one layer, not the wire protocol . . . 62
13.2 A transitional application layer . . . 62
13.3 Middleware is nested control flow . . . 64
13.4 The final application boundary . . . 65
13.5 Stable errors are part of the API . . . 67
13.6 Observability without coupling . . . 68
Chapter summary . . . 68
Review questions . . . 68

14 Bounded concurrency and connection lifecycle **69**
14.1 From serial to concurrent . . . 69
14.2 Bounded is a correctness property . . . 71
14.3 One parser per connection . . . 71
14.4 Persistent but finite . . . 71
14.5 Read the final server . . . 72
14.5.1 Readiness and ephemeral ports . . . 77
14.5.2 Timeout and protocol errors . . . 77
14.5.3 Shutdown . . . 77
14.6 Threads and SQLite . . . 77
14.7 Alternatives . . . 78
Chapter summary . . . 78
Review questions . . . 78

15 The complete tinyhttp framework **79**
15.1 Package boundaries . . . 79
15.2 A minimal application . . . 79
15.3 Trace one request . . . 80

15.4 Configuration is policy . . . 80
15.5 Run the verification suite . . . 81
15.6 How to extend honestly . . . 81
Chapter summary . . . 81
Review questions . . . 81

III From server to backend REST API 83

16 Designing the Task REST API 84
16.1 Resource model . . . 84
16.2 Collection operations . . . 85
16.3 Individual operations . . . 85
16.4 Validation sequence . . . 85
16.5 Error contract . . . 86
16.6 What the API deliberately omits . . . 86
Chapter summary . . . 86
Review questions . . . 86

17 Persistence with SQLite 88
17.1 Why SQLite fits the capstone . . . 88
17.2 Schema as durable validation . . . 88
17.3 Parameterized SQL . . . 89
17.4 Connections and transactions . . . 89
17.5 The database and repository code . . . 90
17.6 Filtering, searching, and pagination . . . 93
17.7 Repository tests use temporary files . . . 93
Chapter summary . . . 94
Review questions . . . 94

18 Implementing the Task API 95
18.1 Application construction . . . 95
18.2 Create flow . . . 101
18.3 Read and list flow . . . 101
18.4 PUT and PATCH . . . 101
18.5 Date and timestamp choices . . . 101
18.6 Run the API . . . 101
18.7 Trace a PATCH . . . 102
Chapter summary . . . 103
Review questions . . . 103

19 Testing from functions to hostile streams 104
19.1 A testing pyramid for this project . . . 104
19.2 Parser tests are protocol examples . . . 104
19.3 Application and API tests . . . 107
19.4 Real socket integration . . . 110
19.5 Failure injection checklist . . . 112
19.6 Hardening review . . . 112
19.7 One-command verification . . . 113

Chapter summary . . . 113
Review questions . . . 113

20 From learning server to production ecosystem 114
20.1 What frameworks no longer hide . . . 114
20.2 WSGI and ASGI . . . 114
20.3 A typical production path . . . 115
20.4 Authentication and authorization . . . 115
20.5 Observability and operations . . . 115
20.6 Safe next projects . . . 116
20.7 Final assessment . . . 116
20.8 The central lesson . . . 116
Chapter summary . . . 116
Review questions . . . 117

IV Appendices 118

A Setup and troubleshooting 119
A.1 Required tools . . . 119
A.2 Optional virtual environment . . . 119
A.3 Repository map . . . 119
A.4 Run tests . . . 120
A.5 Build the book . . . 120
A.6 Run the API . . . 120
A.7 Common failures . . . 121
A.7.1 Address already in use . . . 121
A.7.2 Connection refused . . . 121
A.7.3 Client waits forever . . . 121
A.7.4 400 invalid Host . . . 121
A.7.5 411 on POST, PUT, or PATCH . . . 121
A.7.6 415 Unsupported Media Type . . . 121
A.7.7 SQLite database is locked . . . 121
A.7.8 Font or LaTeX error . . . 121
A.8 A debugging worksheet . . . 122

B HTTP and framework quick reference 123
B.1 Request and response shapes . . . 123
B.2 Methods . . . 123
B.3 Statuses used . . . 124
B.4 Selected fields . . . 124
B.5 Default server limits . . . 125
B.6 Task endpoints . . . 125
B.7 Problem document . . . 125
B.8 Framework public API . . . 125
B.9 Parser decision sequence . . . 126
B.10 Code review checklist . . . 126

C Exercise solutions and mentor notes 127

C.1 Foundations 127
C.2 Building the server 129
C.3 Backend and production bridge 131

D Glossary **134**

References and further reading **138**

Bibliography **139**

Part I

Foundations: from electricity to HTTP semantics

CHAPTER 1

One request, end to end

Before building a server, build a precise story of what a server participates in. A URL typed into a browser crosses many boundaries; HTTP is only one layer of that journey.

Learning objectives

- Identify the client, origin server, resolver, routers, and intermediaries in a web request.
- Separate name resolution, connection establishment, security, HTTP exchange, and application work.
- Distinguish a protocol from an implementation and a resource from its representation.
- Locate failures by phase instead of calling every problem “the network.”

1.1 Begin with a URL

Imagine entering `https://api.example.test/tasks/17?view=compact` into a client. The string is an instruction, not a connection. It identifies a scheme (`https`), an authority (`api.example.test`), a path (`/tasks/17`), and a query (`view=compact`). The client must turn that description into communication with some process.

The scheme determines important defaults and rules. HTTPS normally implies a secure connection and a default port of 443. The hostname is meaningful to humans and to the Domain Name System, but the Internet Protocol forwards packets using numeric addresses. The path and query are carried inside the HTTP request; routers in the network do not use them to choose a physical route.

Mental model

A URL answers *what origin and resource do I want?*; DNS helps answer *which address can reach that origin?*; TCP creates an ordered byte stream to a port; TLS protects that stream; HTTP gives the bytes message semantics; application code decides what the resource means.

1.2 The phases

A typical first request proceeds through the following phases. Real clients reuse cached DNS answers, existing connections, and cached responses, so some phases can disappear on later requests.

1. The client parses and validates the URL.
2. It consults local configuration and caches, then asks a DNS resolver for one or more IP addresses.
3. The operating system selects a route and a source address.
4. The client asks the operating system to open a TCP connection to the selected address and port.
5. For HTTPS, client and server perform a TLS handshake. The client checks that the certificate is valid for the requested hostname.
6. The client serializes an HTTP request into bytes and writes them to the connection.
7. The server operating system accepts the connection and makes received bytes available to a process through a socket.
8. Server code reads, frames, parses, validates, routes, and handles the request. It might query a database or another service.
9. The server serializes an HTTP response and writes it to the connection.
10. The client parses the response, interprets its status and fields, and uses the representation. A browser might then request more resources.

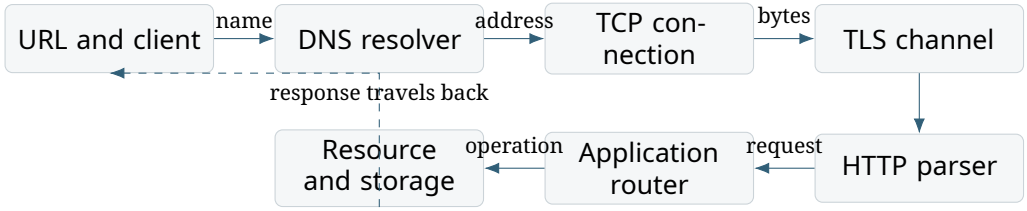


Figure 1.1: The conceptual phases of a first HTTPS request. Each arrow hides a separate interface and possible failure.

1.3 Client, server, and resource

Client and **server** describe roles in one interaction. A process can be a server while receiving an API request and then become a client when it calls a database proxy or another service. The machine itself is not inherently the server; the listening process is performing the server role.

An HTTP **resource** is an abstraction identified by a URI. A task is not the JSON text sent across the wire. JSON is one **representation** of that task at a particular time. The same resource might have an HTML representation, a compact JSON representation, or no transferable representation after a successful deletion. This distinction later explains media types, content negotiation, validators, and status codes.

1.4 Where the server begins and ends

Our Python code will not implement Ethernet, IP, or TCP. The operating system does that work and exposes a socket interface. “From scratch” therefore means from the lowest portable application interface: raw TCP sockets. We will manually define the HTTP boundary, parse requests, serialize responses, route resources, compose middleware, control concurrency, and connect the application to SQLite.

This boundary is important. Calling `recv` does not retrieve “one HTTP request.” It retrieves some bytes currently available from a TCP stream. The HTTP implementation must decide whether those bytes contain a complete header, how long the body is, and whether bytes after that body belong to a later request.

1.5 Failure belongs to a phase

Precise phase language makes debugging faster.

Phase	Typical symptom	First question
DNS	Name cannot be resolved	Did the resolver return an address?
Routing	Destination appears unreachable	Is there a route to that address?
TCP	Connection refused or times out	Is a process listening on the chosen address and port?
TLS	Certificate or handshake failure	Does identity verification match the hostname?
HTTP framing	Client waits, closes, or reports a malformed response	Are lines, lengths, and boundaries correct?
Application	A valid 4xx or 5xx response	Which resource rule or dependency failed?

> **Pitfall**
>
> An HTTP 404 Not Found proves that an HTTP response came back. DNS, routing, TCP, and enough HTTP parsing already worked. Treating it as a generic connectivity failure sends debugging in the wrong direction.

1.6 Latency is accumulated waiting

The total time observed by a user includes name resolution, connection and TLS round trips, queueing, application work, database time, response transfer, and client processing. A server handler that takes 20 milliseconds does not imply a 20-millisecond user experience. Conversely, connection reuse can remove several round trips even when application code is unchanged.

This is the first systems lesson of the book: behavior emerges from a chain of components. Measurements must state where they begin and end.

Chapter summary

A URL initiates several distinct protocols and interfaces. DNS supplies an address; IP forwards packets; TCP exposes an ordered byte stream; TLS can protect it; HTTP defines

messages; application code gives resources meaning. Our server will begin at the socket boundary and implement the layers above it.

Review questions

1. Why does a router not need the path `/tasks/17` to forward an IP packet?
2. Which phases have already succeeded when a client receives a valid 404 response?
3. Why is a JSON document a representation rather than the resource itself?
4. In what situation can one process be both an HTTP server and an HTTP client?

Exercise 1.1: Draw the journey

Draw the request journey for a local URL such as `http://127.0.0.1:8080/health`. Cross out the phases that do not occur and explain why. Then redraw it for an HTTPS API reached through a reverse proxy.

Exercise 1.2: Classify failures

Classify each symptom by its earliest likely phase: “name not known,” “connection refused,” “certificate hostname mismatch,” “400 Bad Request,” and “database busy.” For each, name one observation that would confirm the classification.

CHAPTER 2

Bytes, encodings, and protocol layers

Networks move finite sequences of bits. Every line of text, JSON value, header field, and database result must eventually cross a boundary as bytes.

Learning objectives

- Distinguish bits, bytes, text, characters, and encodings.
- Explain framing and why a receiver needs an unambiguous boundary rule.
- Use layering and encapsulation without confusing the models with physical reality.
- Distinguish packets, TCP segments, byte streams, and application messages.

2.1 A character is not a byte

A **bit** has two possible values. Eight bits form a **byte**, an integer from 0 through 255. A **character** is an abstract element of text. An **encoding** maps characters to bytes and bytes back to characters.

ASCII assigns one byte to a small set of characters. UTF-8 represents every Unicode code point using one to four bytes and deliberately matches ASCII for the first 128 values. The word “café” has four characters but five UTF-8 bytes because é uses two. Lengths in an HTTP message describe bytes, not Python characters.

Mental model

Text does not travel through a socket. Bytes travel. Encoding is the contract used before sending; decoding is the matching operation after receiving. A protocol must state or imply that contract wherever text has meaning.

In Python, `str` holds Unicode text and `bytes` holds byte values. Combining them implicitly would hide an encoding decision, so Python requires an explicit conversion. This apparent inconvenience prevents a large class of wire-format mistakes.

2.2 Framing creates messages

Suppose a sender writes the bytes for `HELLO` followed immediately by the bytes for `WORLD`. The receiver sees `HELLOWORLD`. Nothing inherent in that stream identifies two words. A protocol needs a **framing** rule.

Common framing strategies include:

- a delimiter, such as a newline ending a command;
- a length prefix, such as a four-byte integer followed by that many bytes;
- a fixed size, when every record occupies the same number of bytes;
- connection close, where end-of-stream terminates the message;
- self-describing chunks, each carrying its own length and a final marker.

HTTP/1.1 combines strategies. The header section uses a blank line delimiter. The body length can come from method/status semantics, `Content-Length`, transfer coding, or closing a connection in limited circumstances. Correct framing is security-critical: if two recipients choose different boundaries, an attacker may make one request look like two.

On the wire

The notation below names bytes, not the four printable letters “CRLF.”

```
GET /health HTTP/1.1 CRLF
Host: localhost CRLF
CRLF
```

On the wire, each `CRLF` is the pair `0D 0A` in hexadecimal. The second consecutive pair ends the header section.

2.3 Layers manage complexity

A layered model gives each protocol a focused job and a stable interface to the layer above it. The practical Internet stack is often described with four or five layers:

Layer	Examples	Responsibility
Application	HTTP, DNS, TLS	Meaning and rules used by applications
Transport	TCP, UDP, QUIC	Communication between processes; reliability or datagrams
Internet	IPv4, IPv6	Addressing and forwarding across networks
Link	Ethernet, Wi-Fi	Delivery on one local link
Physical	Copper, fiber, radio	Signals that represent bits

The seven-layer OSI model remains useful vocabulary, but Internet implementations do not always fit its divisions exactly. TLS, for example, sits between an application protocol and a transport in common usage. Models help reasoning; they are not laws of nature.

2.4 Encapsulation and changing envelopes

When an application sends an HTTP request, each lower layer adds control information. TCP groups some stream bytes into segments with sequence-related metadata. IP carries a transport segment in a packet with source and destination addresses. A link carries the packet in a frame suitable for the next hop.

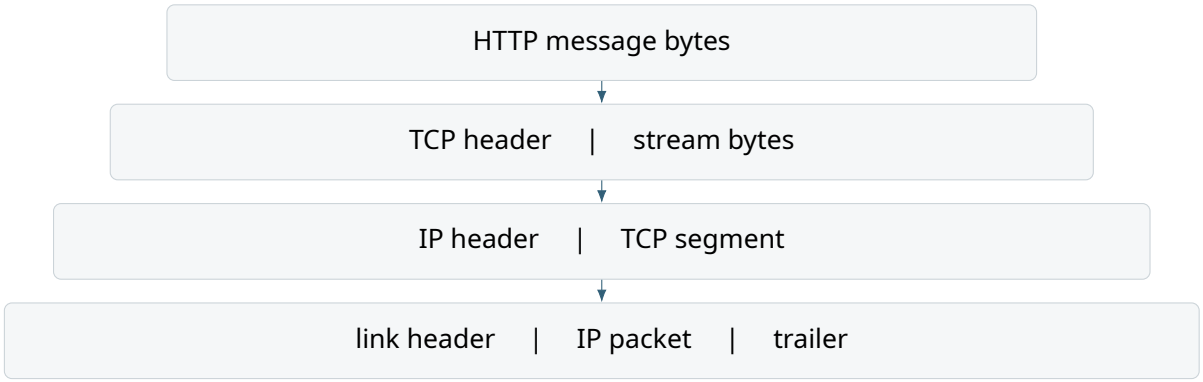


Figure 2.1: Conceptual encapsulation. Sizes are illustrative, not proportional.

At a router, the link envelope normally changes because the next link has new local addressing. The IP packet continues toward its destination, though fields such as the hop limit change. At the destination, layers remove and interpret their own metadata before delivering stream bytes to the application.

2.5 Packet boundaries are not message boundaries

The word “packet” is often used loosely. Be precise when precision matters: a link transmits frames, IP forwards packets, TCP transmits segments, and HTTP defines messages. These boundaries need not align. One HTTP header can span several TCP segments. Several short HTTP writes can be combined. One call to receive bytes can return half a line, exactly one request, or a request plus the beginning of the next request.

TCP preserves order and detects missing data; it does not preserve the sender’s write-call boundaries. Therefore a correct parser accumulates bytes and applies HTTP’s framing rules. It never assumes one receive operation equals one message.

> **Pitfall**
>
> Testing only on localhost often hides fragmentation bugs. Local transfers are fast and may make a small request appear in one read. That outcome is permitted, not guaranteed. Later tests deliberately feed one byte at a time.

2.6 Numbers and byte order

Binary protocols must also agree how multi-byte numbers are ordered. Network protocols traditionally use big-endian, often called network byte order: the most significant byte comes first. HTTP/1.1’s visible integers are decimal text, so `Content-Length: 12` is the ASCII sequence for the characters 1 and 2, not a binary integer. HTTP/2 and HTTP/3 use binary framing where byte order again becomes visible to implementers.

Chapter summary

Bytes are the common currency of network interfaces. Encodings give bytes text meaning; framing gives a stream message boundaries; layering gives complicated systems manageable interfaces. TCP is an ordered stream, not a message queue, so HTTP parsing must be incremental and boundary-aware.

Review questions

1. Why can four characters require more than four bytes?
2. Name three framing strategies and one trade-off for each.
3. Why may a link-layer frame change at every router?
4. What outcomes are legal for a receive call after one HTTP request was sent?

Exercise 2.1: Design a frame

Design a tiny protocol for sending a sequence of UTF-8 notes. Choose a framing strategy. Explain how the receiver detects an incomplete note, an empty note, and the end of the sequence. Then identify one malformed input it must reject.

Exercise 2.2: Count the bytes

For the texts `hello`, `café`, and a word in a language you use, record character count and UTF-8 byte count. Explain which count belongs in a length field that frames network bytes.

CHAPTER 3

Addresses, ports, routing, and DNS

A server is reachable only when several names and numbers agree: the hostname, IP address, transport port, listening socket, and route.

Learning objectives

- Explain the roles of IP addresses, transport ports, and socket endpoints.
- Distinguish loopback, private, public, unspecified, and wildcard addresses.
- Describe recursive DNS resolution and caching at a practical level.
- Explain routing and NAT without attributing application semantics to them.

3.1 IP identifies an interface location

An IP address identifies a network interface in a routing context. IPv4 writes 32 bits as four decimal octets, for example `192.0.2.10`. IPv6 uses 128 bits and hexadecimal groups, allowing compressed forms such as `2001:db8::10`. The documentation ranges used here are reserved for examples and should not be treated as real destinations.

An address is not a permanent identity for a person, application, or physical machine. Interfaces can have several addresses; addresses can change; load balancers can advertise one address for many servers; a mobile device can move between networks. HTTP uses host and origin concepts above this layer precisely because an IP address alone does not identify an application resource namespace.

3.1.1 Address scopes that matter in the lab

- `127.0.0.1` is IPv4 loopback. Traffic stays inside the local host.
- `::1` is IPv6 loopback.
- Private IPv4 ranges are used inside networks and are not globally routed.
- Public addresses can be globally reachable subject to routing and policy.
- `0.0.0.0` used while binding means all local IPv4 interfaces; it is not a remote destination to which a client should normally connect.

Binding our server to loopback is a safety decision. A process listening on `127.0.0.1:8080` is reachable by local clients but not directly through the machine's

other interfaces. Binding the same process to `0.0.0.0:8080` may expose it to a local network, a container bridge, or more, depending on firewall and routing rules.

3.2 Ports identify transport endpoints

An IP address gets traffic to a host interface. A 16-bit TCP or UDP port helps the operating system deliver it to a transport endpoint. Port numbers range from 0 through 65535. Ports below 1024 are commonly treated as privileged on Unix-like systems. HTTP conventionally uses 80 and HTTPS 443, while development servers often choose unprivileged ports such as 8000 or 8080.

A TCP connection is identified by protocol plus a four-tuple: local address, local port, remote address, and remote port. This explains how a listening server can support many clients on the same local port. Each accepted connection has a different remote endpoint, and clients normally use temporary source ports.

Mental model

The listening socket is a reception desk at one address and port. Accepting a connection creates a separate conversation socket. The desk remains available for new arrivals while workers handle existing conversations.

3.3 Routing chooses the next hop

Hosts and routers maintain routing tables. A route associates an address prefix with an outgoing interface and possibly a next-hop router. The most specific matching prefix wins. If no more specific route exists, a default route usually points toward a gateway.

Routers forward IP packets hop by hop. They do not establish one end-to-end physical wire and do not need HTTP paths. Different packets can in principle take different routes. IP makes a best effort; TCP above it recovers from loss and reordering for applications that require a reliable stream.

3.4 NAT rewrites addressing context

Network Address Translation is common at boundaries between private and public networks. A gateway can map many internal address-and-port pairs to one public address with distinct ports. It records the mapping so reply traffic can be directed back to the correct internal endpoint.

NAT is not a substitute for an application security policy, and it is not part of HTTP. It changes the network view of endpoints. This can complicate inbound connections, logging, and the interpretation of client addresses. Reverse proxies add another layer: an application may see the proxy’s TCP address while the original address is described in a trusted HTTP field.

3.5 DNS maps names to data

The Domain Name System is a distributed, hierarchical database. A hostname can have IPv4 address records (A), IPv6 address records (AAAA), aliases (CNAME), and many other record types. DNS does not test whether an HTTP server is healthy; it returns configured data and associated cache lifetimes.

A typical client asks a **recursive resolver**. The resolver may already have a cached answer. Otherwise it follows delegations: root servers direct it toward a top-level domain, those servers direct it toward authoritative servers, and an authoritative server supplies the requested record. The resolver caches answers according to their time-to-live values and returns them to the client.

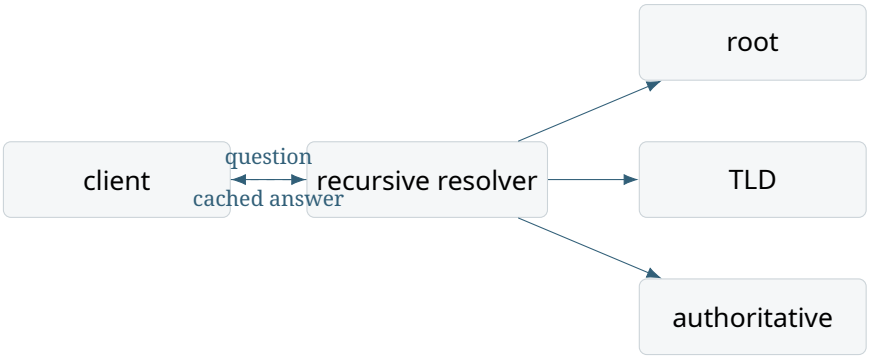


Figure 3.1: A simplified recursive lookup. Referrals and replies are condensed.

3.5.1 Why DNS answers vary

DNS can return several addresses for redundancy or geographic distribution. Different resolvers may have differently aged caches. A deployment can change records, but old answers remain valid until their TTLs expire. A failed new server therefore does not always affect every client at once, and a fixed DNS record does not immediately repair clients holding an older answer.

3.6 Names return at the HTTP layer

After DNS, the hostname still matters. One IP address can host many HTTP origins. In HTTP/1.1 the client sends a `Host` field so the server or reverse proxy can select the intended origin. For HTTPS, the hostname is also used during TLS identity verification and commonly during server-name negotiation. Replacing a hostname with its address can therefore change both TLS and HTTP behavior.

Pitfall

“It pings” does not prove an HTTP service works. Ping commonly uses ICMP, not TCP port 8080, TLS, or HTTP. It can be blocked when HTTP works, or succeed when no application is listening.

3.7 A disciplined diagnostic order

When a server cannot be reached, ask questions from lower ambiguity to higher:

1. What exact hostname, address, and port did the client choose?
2. Did name resolution return the expected address family and address?
3. Does the host have a route toward that destination?
4. Is the server process listening on the intended local address and port?
5. Do firewall or namespace boundaries permit the connection?

6. Did TCP connect? If so, continue to TLS and HTTP evidence.

Chapter summary

Addresses locate interfaces for IP forwarding; ports locate transport endpoints; DNS maps names to configured data; routing chooses next hops; NAT translates endpoint views. A server becomes reachable only when these mechanisms agree with its bind address and the client's chosen destination.

Review questions

1. Why can many TCP clients share a server's port 8080?
2. What is the practical difference between binding to loopback and to a wildcard address?
3. Why might two clients temporarily receive different DNS answers for one name?
4. Why can replacing a hostname with its IP address break HTTPS?

Exercise 3.1: Endpoint inventory

For a local development server, write the listening endpoint, one possible client endpoint, and the resulting four-tuple. Repeat for two simultaneous clients and underline what changes.

Exercise 3.2: Resolution narrative

Narrate an uncached DNS lookup from a laptop to an authoritative answer. Then repeat the story when every relevant answer is cached. Identify which network round trips disappear.

CHAPTER 4

TCP streams and the socket interface

TCP turns unreliable packet delivery into an ordered stream between two endpoints. Sockets are the operating-system interface through which our Python process joins that stream.

Learning objectives

- Explain connection establishment, ordered delivery, acknowledgments, and closing.
- Distinguish reliability, flow control, congestion control, and application timeouts.
- Describe listening and connected sockets and the bind–listen–accept lifecycle.
- Predict partial reads, partial writes, blocking, and slow-client behavior.

4.1 A reliable byte stream

TCP provides two applications with an ordered, bidirectional sequence of bytes. It numbers bytes conceptually, detects loss, retransmits data, removes duplicates, and delivers data to the receiving application in order. If a later segment arrives before an earlier one, the implementation normally waits rather than showing the application a gap.

Reliability does not mean success is guaranteed. A cable can remain disconnected, a process can crash, or a peer can stop reading forever. TCP either continues trying within its policies or reports failure. Applications still need deadlines, cancellation, and error handling.

4.2 Establishing a connection

The familiar three-way handshake synchronizes connection state and initial sequence numbers:

1. The client sends a segment with the SYN flag.
2. The server replies with SYN and ACK.
3. The client acknowledges the server's SYN.

The handshake also negotiates capabilities. It costs a network round trip before ordinary application data can flow, which is one reason connection reuse matters. A server's listening socket and backlog participate before application code accepts the established connection.

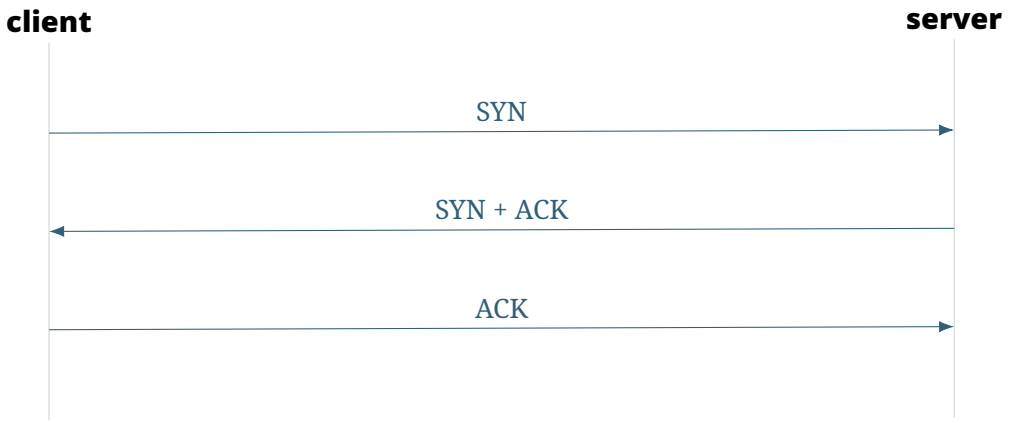


Figure 4.1: A simplified TCP three-way handshake.

4.3 Sequence, acknowledgment, and retransmission

A TCP acknowledgment says, roughly, "I have received all bytes before this next expected sequence number." Senders keep unacknowledged data so it can be retransmitted. They infer loss through timers and duplicate acknowledgment patterns. Checksums detect corruption within their scope.

The application does not receive these segments directly. It writes bytes to a socket buffer and reads bytes from another buffer. TCP can split one write into many segments or combine data from several writes. Network offload mechanisms make the relationship even less visible. The application must treat the result as a stream.

4.4 Two kinds of control

Flow control protects the receiver. The receiver advertises how much buffer capacity is available; a sender that ignores this would overwhelm the destination. **Congestion control** protects the network path. The sender adjusts how much data is in flight based on observed conditions such as loss and delay.

These mechanisms can make a write block or make progress slowly. A successful write means bytes were accepted by the local socket machinery, not that a remote application has already processed them. Likewise, a successful TCP handshake does not mean an HTTP request will be accepted.

4.5 Closing is a protocol too

Each direction of a TCP connection closes independently with FIN and acknowledgment exchange. A peer can finish sending while still receiving. An abrupt failure can produce a reset (RST), which discards the graceful-close story. Servers must expect clients to disappear between any two operations.

The endpoint that actively closes can remain in a `TIME_WAIT` state. This helps prevent delayed segments from an old connection being mistaken for a new one with the same endpoint tuple. It is normal protocol housekeeping, not automatically evidence of a leak.

4.6 Sockets: the application boundary

A socket is an operating-system object representing a communication endpoint. The typical server lifecycle is:

1. **create** a TCP socket;
2. **bind** it to a local address and port;
3. mark it as **listening** with a backlog;
4. **accept** a connection, producing a new connected socket;
5. receive and send bytes on the connected socket;
6. close the connected socket and eventually the listener.

The listener does not carry HTTP request bytes for an accepted client. The connected socket does. This separation lets one accept loop hand connections to workers while continuing to receive new arrivals.

4.6.1 Blocking and timeouts

A blocking call waits until it can make progress or report an event. Accept can wait for a client. Receive can wait for bytes. Send can wait for buffer capacity. Simple blocking code is ideal for learning because control flow is visible, but unbounded waiting gives a slow or silent peer control over a worker.

A timeout changes an infinite wait into an explicit failure path. The server we build will set a read/idle timeout, limit header and body sizes, bound the number of workers and queued connections, and limit requests per persistent connection. These constraints are part of correctness, not optional tuning.

> **Security boundary**
>
> An attacker does not need high bandwidth to exhaust a naive server. Many connections that send headers one byte at a time can occupy all blocking workers. Deadlines, byte limits, bounded queues, and graceful rejection reduce this risk.

4.7 Partial operations are ordinary

A receive call returns up to the requested number of bytes. It may return fewer because that is what is currently available. An empty byte string indicates orderly end-of-stream. A low-level send can accept fewer bytes than supplied; Python's `sendall` repeatedly sends until all bytes are accepted or an error occurs.

The correct question is never "Did one read return my request?" It is "Does my accumulated buffer contain enough bytes, according to HTTP framing, to parse one complete request?"

4.8 Why not UDP?

UDP sends independent datagrams and does not provide TCP's ordered reliable stream. Applications that choose it must accept loss/reordering or implement the needed behavior. HTTP/3 uses QUIC over UDP, but QUIC itself supplies secure, reliable streams and congestion control. "Uses UDP" does not mean "ignores reliability." We implement HTTP/1.1 over TCP because its text framing exposes the fundamentals clearly.

Chapter summary

TCP establishes state, presents ordered bytes, recovers from common packet loss, and controls sender behavior, but applications still need framing, timeouts, limits, and failure handling. A listening socket accepts connected sockets; each connected socket carries an unstructured byte stream from HTTP's perspective.

Review questions

1. Why can one application write become several TCP segments?
2. What is the difference between flow control and congestion control?
3. What does an empty result from a receive operation mean?
4. Why does the server need both a listener and connected sockets?

Exercise 4.1: Stream outcomes

A client writes a 600-byte header and a 200-byte body. List at least four legal ways a server's sequence of receive calls could divide those 800 bytes. State which HTTP rule tells the server when the body is complete.

Exercise 4.2: Slow client budget

Suppose a server has 16 blocking workers and no read timeout. Explain how 16 clients can prevent a seventeenth valid client from being handled. Propose a timeout, queue, and rejection policy and describe the trade-off of each.

CHAPTER 5

The HTTP semantic model

HTTP is a family of stateless request/response protocols with shared semantics. First learn what a message means; the next chapter studies exactly how HTTP/1.1 writes it as bytes.

Learning objectives

- Distinguish URI, origin, resource, representation, content, and field metadata.
- Explain method safety, idempotency, and the major method semantics.
- Interpret status-code classes without treating every 2xx or 4xx alike.
- Explain stateless interaction and where application state still lives.

5.1 Messages carry intentions and results

An HTTP client sends a request that identifies a target and expresses an intention through a method, fields, and optional content. A server interprets that request with respect to a resource and returns one or more responses. A final response has a status code, fields, and optional content.

HTTP separates a uniform interface from implementation details. A client does not need to know whether a task is stored in SQLite, computed from another service, or generated temporarily. The server does not need to know why the client wants it. This separation allows both sides to evolve behind the protocol boundary.

5.2 URI, origin, resource, representation

A URI identifies a resource. For HTTP and HTTPS, an **origin** is the tuple of scheme, host, and port after applying defaults. Thus `http://example.test/tasks` and `https://example.test/tasks` are different origins, even though their host and path text match.

The resource is conceptual. Its current representation is a sequence of content bytes plus metadata describing how to interpret them. A JSON media type tells a recipient to process bytes as JSON; a character encoding connects text bytes to characters. Representation metadata can also describe language, compression, freshness, and validators.

5.3 Methods are semantics, not route names

The method token is case-sensitive and states the request's primary semantics. Standard method names are uppercase.

Method	Safe	Idempotent	Intended meaning
GET	yes	yes	Transfer a current representation
HEAD	yes	yes	Same as GET without response content
POST	no	no	Perform resource-specific processing
PUT	no	yes	Replace the target's current representation
PATCH	no	not guaranteed	Apply a described partial modification
DELETE	no	yes	Remove the association with the target resource
OPTIONS	yes	yes	Describe communication options

Safe means the requested semantics are read-only; logging and accounting side effects can still occur. **Idempotent** means repeating the same request has the same intended effect as performing it once. It does not require byte-for- byte identical responses. A repeated DELETE can first return success and later return not found while the resource remains deleted.

These properties guide retries. A client can often retry an idempotent operation after losing a connection without fearing duplicate intended effects. Retrying a POST that creates a charge or task may duplicate work unless the API adds an idempotency mechanism.

> **Pitfall**
>
> Using GET for a state-changing action because it is convenient violates method semantics. Crawlers, caches, link previewers, and retrying clients are entitled to treat GET as safe.

5.4 Status codes describe the result

The first digit groups status codes:

- **1xx Informational**: an interim response before a final response;
- **2xx Successful**: the request was received, understood, and accepted;
- **3xx Redirection**: another step or a cached representation is involved;
- **4xx Client Error**: the request cannot be fulfilled as sent;
- **5xx Server Error**: the server failed to fulfill an apparently valid request.

Choose the specific code that communicates the outcome. 200 OK commonly carries a representation. 201 Created identifies a newly created resource, usually with a `Location` field. 204 No Content reports success without content. 400 Bad Request means malformed or invalid request syntax at a broad level; 404 Not Found hides or reports absence; 405 Method Not Allowed means the resource exists for other methods and should include `Allow`. 415 Unsupported Media Type rejects the content format. 422 Unprocess-

able Content can report semantically invalid fields. 500 Internal Server Error is not a license to reveal a traceback.

5.5 Fields modify or describe messages

HTTP fields are named metadata. Names are case-insensitive; values follow field-specific syntax. Some fields can appear more than once or contain lists, so blindly storing every name in a one-value dictionary loses information.

Examples include:

- `Host`, which identifies the target authority in HTTP/1.1;
- `Content-Type`, which describes content media type;
- `Content-Length`, which participates in HTTP/1.1 framing;
- `Accept`, which expresses response representation preferences;
- `Authorization`, which carries credentials under a chosen scheme;
- `Cache-Control`, which gives cache directives;
- `Location`, which identifies a created or redirect target.

Fields are not all interchangeable strings. Their individual semantics determine whether whitespace, combination, repetition, and ordering matter.

5.6 Stateless does not mean no state

HTTP is stateless in the sense that each request can be understood without an implicit transport conversation state defining its meaning. Application state still exists in databases, caches, and clients. A client can send a cookie or authorization credential on each relevant request, making context explicit in the request.

Persistent TCP connections are a performance mechanism, not an application session. Two requests on one connection should not secretly mean different things merely because of their connection order, unless a protocol extension explicitly defines such state.

5.7 Content versus representation

Modern HTTP specifications use **content** for the bytes after transfer codings have been removed. In HTTP/1.1, chunk boundaries are message framing; they are not part of the content delivered to an application. Likewise, TLS records protect transport bytes but are not part of the HTTP representation.

This distinction keeps semantics consistent across versions. HTTP/2 uses binary frames and HTTP/3 uses QUIC streams, but a GET, status 404, media type, and cache directive retain shared meaning.

Chapter summary

HTTP gives resources a uniform request/response interface. Methods express intentions and properties such as safety and idempotency. Status codes communicate results. Fields carry typed metadata. Representations transfer resource state, while HTTP's stateless model keeps each request independently meaningful.

Review questions

1. Why are a resource and its JSON representation different things?
2. How can an idempotent request return different responses when repeated?
3. What distinguishes 404 from 405?
4. Why is a persistent TCP connection not an application session?

Exercise 5.1: Method audit

Choose methods for creating, reading, replacing, partially updating, and deleting a task. For each, state whether an automatic retry is generally safe and which application-specific caveat remains.

Exercise 5.2: Choose the result

Choose and justify status codes for: successful creation, successful deletion with no body, malformed JSON, an unknown task, a known path with the wrong method, an unsupported media type, invalid task fields, and an unexpected database failure.

CHAPTER 6

HTTP/1.1 on the wire

> HTTP semantics are abstract; HTTP/1.1 gives them a textual start line, fields, framing rules, and connection behavior. Small parsing shortcuts here can become security vulnerabilities.

Learning objectives

- Read request lines, status lines, header sections, and message bodies as bytes.
- Determine message-body length for the subset implemented in this book.
- Explain persistent connections and why exact consumption matters.
- Recognize ambiguous framing and request-smuggling risk.

6.1 Message shape

An HTTP/1.1 message contains a start line terminated by CRLF, zero or more field lines each terminated by CRLF, an empty CRLF line, and an optional body. A request uses a request line; a response uses a status line.

```
POST /tasks?notify=false HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json
Content-Length: 21

{"title":"Read RFC"}
```

The displayed blank line is mandatory. In actual bytes, every line ends with
r
n. The body is 21 UTF-8 bytes in this example. A parser must not search for a newline to end JSON; JSON can contain escaped newlines and the next request may follow immediately.

6.2 The request line

The request line contains method, one space, request target, one space, and HTTP version. For a direct request to an origin server, the common target form is the path and optional

query, called **origin-form**. Proxies can receive an absolute URI. CONNECT and OPTIONS define other forms.

Our server accepts HTTP/1.1 origin-form targets and the asterisk target for server-wide OPTIONS. It rejects other forms explicitly. It also preserves method case so a lowercase `get` is not silently turned into the standard GET.

6.3 Field parsing

Field names use a restricted token grammar and are case-insensitive. A colon separates name from value. Obsolete line folding is rejected. Control characters in values are rejected. Exactly one non-empty `Host` field is required for the implemented HTTP/1.1 subset.

Repeated fields need field-specific handling. The final `Headers` type stores an ordered list and exposes both `get` and `get_all`. For framing, the implementation deliberately rejects repeated `Content-Length` even when values match. This is stricter than some permitted normalization and reduces ambiguity in an educational parser.

6.4 Where does the body end?

Message framing is not the same as media parsing. The server must first isolate the exact body bytes. Only then should JSON code interpret them.

For requests in our subset:

1. A `Transfer-Encoding` field is rejected because transfer codings, including chunked coding, are not implemented.
2. A simultaneous `Transfer-Encoding` and `Content-Length` is rejected as ambiguous.
3. More than one `Content-Length` is rejected.
4. One valid non-negative `Content-Length` frames exactly that many bytes.
5. Without either field, the request has a zero-length body. Endpoints that require content can answer `411 Length Required`.

The parser keeps bytes after the framed body. On a persistent connection those bytes can begin the next request. Discarding them would corrupt the stream; including them in the first body would do the same.

Security boundary

Request smuggling occurs when two HTTP recipients disagree about where one request ends and another begins. Conflicting length fields, transfer-coding differences, and permissive whitespace parsing are common ingredients. A small server behind a proxy must not assume ambiguity is harmless merely because its own parser chooses one interpretation.

6.5 Chunked transfer coding

Chunked coding represents a body as hexadecimal-size chunks followed by a zero-size chunk and optional trailer fields. It lets a sender begin before the final content length is known. A compliant general-purpose HTTP/1.1 parser needs careful support for its grammar, extensions, decoded-size limits, trailers, and interactions with other fields.

The book explains chunking but the framework returns 501 Not Implemented and closes the connection whenever `Transfer-Encoding` is present. This is an explicit product boundary, not an accidental omission hidden behind incorrect parsing.

6.6 Response framing and special cases

A response status line contains version, three-digit status, and an optional reason phrase. Reason phrases are for humans and do not define status semantics. Our serializer adds `Date`, `Server`, `Connection`, and usually `Content-Length`.

Responses to HEAD carry the fields that a corresponding GET would produce but do not carry the content bytes. The `Content-Length` can therefore describe the hypothetical GET body while no body follows. Informational responses, 204 No Content, and 304 Not Modified have rules that prohibit message content; our serializer suppresses it.

6.7 Persistent connections

HTTP/1.1 uses persistent connections by default. After a framed response, the connection can carry another request. Either side can signal `Connection: close`. Persistence avoids repeated TCP and TLS handshakes, but it also means a worker can be occupied by an idle peer.

The final server supports sequential requests with a five-second idle timeout and at most 100 requests per connection. It always sends a length-delimited response when content is allowed. Early stages send `Connection: close` so only one message needs to be considered while fundamentals are introduced.

HTTP/1.1 pipelining permits a client to send later requests without waiting for earlier responses, with responses kept in order. It proved difficult to use robustly in browsers and is not a focus here. Our incremental buffer can retain an already-arrived next request, but each connection processes requests sequentially.

6.8 Limits are protocol behavior

Without explicit limits, a peer can send an enormous target, unending fields, or a declared body too large to store. The final configuration limits the request line to 8 KiB, the header section to 32 KiB and 100 fields, and the body to 1 MiB. It answers with 414 URI Too Long, 431 Request Header Fields Too Large, or 413 Content Too Large as appropriate.

> **Pitfall**
>
> Calling a single receive operation with a 32 KiB buffer does not impose a valid 32 KiB header limit. The call can return fewer bytes, and a peer can continue. The implementation must accumulate and count until it finds the delimiter or crosses the limit.

Chapter summary

HTTP/1.1 uses CRLF-delimited start and field lines, a blank line, and explicit body-framing rules. Persistent connections make exact byte consumption mandatory. A strict subset can be correct when it clearly rejects unsupported framing and ambiguous inputs; a permissive half-implementation is dangerous.

Review questions

1. Why does JSON parsing begin only after HTTP body framing succeeds?
2. What happens to bytes after the declared request body?
3. Why does a HEAD response sometimes contain a nonzero Content-Length?
4. Why does this server reject transfer coding instead of ignoring it?

Exercise 6.1: Frame two requests

Write two GET requests consecutively with no extra separator beyond the first request's final CRLF pair. Mark the exact boundary. Repeat with a POST whose body ends immediately before the second request.

Exercise 6.2: Ambiguity review

Analyze a request containing both `Transfer-Encoding: chunked` and `Content-Length: 4`. Explain how disagreement between a proxy and origin could cause the following bytes to be interpreted differently.

CHAPTER 7

Web architecture, REST, and security boundaries

A useful HTTP server lives in an ecosystem of TLS terminators, proxies, caches, clients, and standards. This chapter completes the theory gate and defines exactly what we will build.

Learning objectives

- Explain forward proxies, reverse proxies, gateways, and caches.
- Describe TLS goals and the trust decisions behind HTTPS.
- Apply REST constraints to a resource-oriented API without treating REST as a synonym for JSON.
- Compare HTTP/1.1, HTTP/2, and HTTP/3 at a conceptual level.
- State the supported, rejected, and explanation-only features of the project.

7.1 Intermediaries are normal

HTTP was designed to pass through intermediaries. A **forward proxy** is selected on behalf of a client. A **reverse proxy** accepts traffic on behalf of origin applications. A **gateway** translates or fronts another system. A **cache** reuses stored responses when protocol rules allow.

A reverse proxy may terminate TLS, enforce request limits, authenticate users, balance traffic, compress responses, and forward a simpler request to an application server. This is a common production boundary. It does not excuse the application from validating inputs, and it creates a trust question: fields such as a forwarded client address are meaningful only when set or sanitized by a trusted intermediary.

7.2 Caching changes the path

A cache can answer a request without contacting the origin. Cache keys, freshness lifetimes, validators, request directives, response directives, and method/status rules determine whether reuse is valid. An `ETag` validator can support a conditional GET; if the

stored representation is still current, the origin can return 304 Not Modified without the representation content.

Caching is not implemented in `tinyhttp`, but it is important to API design. Unsafe methods should invalidate relevant stored responses. Personalized data must not accidentally become shared. A backend developer must understand that a response can affect later requests even after its connection closes.

7.3 What TLS provides

TLS wraps a transport connection to provide confidentiality, integrity, and authenticated server identity when certificates are validated correctly. During the handshake, peers negotiate cryptographic parameters, the server presents a certificate chain, and the client checks trust, hostname, validity, and policy. The handshake establishes keys used to protect application data.

TLS does not make a vulnerable application safe. It protects data in transit between TLS peers. An authorized client can still send malicious input; an application can still expose secrets in a response; decrypted data still exists at endpoints. Certificate verification is essential—encryption to an unverified attacker is not the intended security property of HTTPS.

Our learning server does not implement cryptography and does not terminate TLS. Production deployments normally place an established TLS implementation and production server or proxy in front of application code.

7.4 REST as an architectural style

REST is not “HTTP plus JSON” and not a route naming convention. Its constraints include client–server separation, stateless requests, cacheability, a uniform interface, a layered system, and optional code-on-demand. The uniform interface includes identification of resources, manipulation through representations, self-descriptive messages, and hypermedia as the engine of application state.

Many practical APIs use several REST ideas without implementing mature hypermedia. This book will be precise: the Task API is resource-oriented and uses HTTP semantics carefully, but it does not claim every possible REST maturity property.

7.4.1 Model nouns before endpoints

The core collection resource is `/tasks`; an individual task is `/tasks/{id}`. Methods carry operations, so paths do not become `/createTask` or `/deleteTask`. Representations contain task fields. Filters and pagination refine the collection request rather than inventing a new verb-shaped endpoint.

Statelessness means every request carries what the server needs to interpret it. Database state persists tasks, but the meaning of PATCH does not depend on a secret sequence of prior requests on the same TCP connection.

7.5 Input is hostile until proven otherwise

Security is an end-to-end property. The server will enforce:

- strict message syntax and unambiguous framing;

- finite line, field, body, time, worker, and queue limits;
- header-value validation to prevent response splitting;
- media-type and UTF-8 checks before JSON parsing;
- allow-lists, lengths, types, and formats for task fields;
- parameterized SQL rather than string interpolation;
- generic 500 responses while logging internal exceptions;
- loopback binding by default.

Authentication is intentionally absent. A toy password or static token can teach unsafe habits if mistaken for a deployable identity system. The security chapter explains where authentication and authorization belong and why they need established password hashing, token/session policy, secret management, revocation, TLS, and audit design.

7.6 Why newer HTTP versions exist

HTTP/1.1 can reuse connections but concurrent requests often require several connections, and ordered responses can delay later work. HTTP/2 retains HTTP semantics while using binary frames and multiplexed streams on one connection. It compresses fields using HPACK. Packet loss still affects the underlying TCP connection, even if application streams are independent at the HTTP layer.

HTTP/3 maps HTTP semantics to QUIC, which runs over UDP and provides encrypted, multiplexed streams with loss recovery. Loss affecting one QUIC stream need not block delivery on unrelated streams in the same way TCP ordering can. QPACK handles field compression under these constraints.

We implement HTTP/1.1 because a human can inspect its wire messages and because it makes framing visible. We do not pretend the same parser can be adapted to HTTP/2 by adding a condition; the binary framing and connection machinery are different implementations of shared semantics.

7.7 The implementation contract

Category	Included	Boundary
Protocol	HTTP/1.1 origin-form, Host, Content-Length, core methods, HEAD/OPTIONS, persistent sequential requests	Other versions rejected
Framework	request/response types, multi-value fields, router, middleware, problem responses	No templates, static files, cookies, sessions, or dependency injection
Transport	IPv4 loopback TCP, bounded blocking workers, timeouts, clean shutdown	No TLS, IPv6 listener, event loop, or production tuning
Content	UTF-8 JSON with 1 MiB request limit	Chunking, streaming, compression, ranges, and negotiation explained but not implemented
Backend	SQLite Task CRUD, validation, filtering, pagination, transactions	No accounts, authentication, or distributed database

An explicit boundary makes the project honest. Unsupported features receive a clear error rather than accidental behavior. The final production bridge will show how WSGI, ASGI, mature servers, reverse proxies, and frameworks address broader needs.

Checkpoint

Before Part II, you should be able to explain, without code, how a URL becomes an HTTP request; why TCP has no request boundary; how HTTP/1.1 finds the end of a length-framed body; why methods and status codes have semantics; and which features our strict subset rejects. If one explanation is vague, revisit the relevant chapter before implementing its parser.

Chapter summary

HTTP systems include trusted and untrusted intermediaries, caches, TLS, and multiple protocol versions. REST supplies architectural constraints, not a JSON branding label. The server we build has a deliberately strict contract: enough HTTP/1.1 for a meaningful backend, explicit limits, and clear production boundaries.

Review questions

1. When may a reverse proxy field describing client identity be trusted?
2. Why does TLS not replace application input validation?
3. Which REST constraint is violated by hidden per-connection application state?
4. Why is HTTP/2 not simply HTTP/1.1 with more headers?

Exercise 7.1: Theory gate

On one page, diagram a POST from a client to the future Task API through DNS, TCP, an optional TLS proxy, HTTP parsing, routing, validation, SQLite, and the response. Label every byte/text conversion and every trust boundary.

Exercise 7.2: Feature classification

Classify each as implemented, explicitly rejected, or explanation-only: `Content-Length`, chunked request bodies, GET, HTTP/2, TLS, repeated ordinary fields, duplicate Content-Length, JSON, authentication, and keep-alive. Explain why each boundary is visible rather than silent.

Part II

Building an HTTP server from raw sockets

CHAPTER 8

The first server: one connection, one response

> The theory gate is complete. We now cross the operating-system boundary with the smallest server that can answer a real HTTP client. Its limitations are intentional and visible.

Learning objectives

- Create, configure, bind, listen, accept, receive, send, and close a TCP socket.
- Construct a byte-correct fixed HTTP/1.1 response.
- Explain every resource lifetime in the first snapshot.
- Observe why successful toy behavior is not yet a parser or framework.

8.1 Prepare the laboratory

Use Python 3.12 or newer. The examples use only the standard library. Work in the repository root, then enter the first snapshot directory. On Linux and macOS, the launcher is commonly `python3`; on Windows it may be `py`.

```
cd code/stages/01_fixed_response
python server.py
```

The process prints a loopback URL and waits. Open the URL in a browser, use an HTTP client, or create a connection in a second terminal. This snapshot exits after serving one accepted connection, so restart it for another attempt.

Security boundary

Keep `HOST = "127.0.0.1"`. The first server has no timeouts, input limits, robust parser, concurrency policy, TLS, or authentication. Its purpose is to make the socket lifecycle concrete on one machine.

8.2 Read the complete program

```python
"""Stage 1: send one fixed HTTP response, then stop."""

from __future__ import annotations

import socket


HOST = "127.0.0.1"
PORT = 8080
BODY = b"Hello from a raw Python socket!\n"


def response_bytes() -> bytes:
    return (
        b"HTTP/1.1 200 OK\r\n"
        b"Content-Type: text/plain; charset=utf-8\r\n"
        + f"Content-Length: {len(BODY)}\r\n".encode("ascii")
        + b"Connection: close\r\n\r\n"
        + BODY
    )


def main() -> None:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind((HOST, PORT))
        listener.listen(1)
        print(f"Open http://{HOST}:{PORT}")
        connection, address = listener.accept()
        with connection:
            print("client:", address)
            connection.recv(4096)
            connection.sendall(response_bytes())


if __name__ == "__main__":
    main()
```

The program is short enough to explain without abstraction.

8.2.1 Create and configure

`socket.socket(AF_INET, SOCK_STREAM)` is implicit in the snapshot's default constructor on common Python builds: IPv4 plus TCP. Later code states these arguments explicitly. The socket object owns an operating-system resource, so a `with` statement guarantees close when control leaves the block.

`SO_REUSEADDR` makes repeated local development starts less likely to fail because of recently used address state. It does not permit two ordinary listeners to share the same endpoint arbitrarily, and it is not a security feature.

8.2.2 Bind and listen

`bind((HOST, PORT))` selects the local endpoint. Another process already listening there normally causes an address-in-use error. Binding can also fail when the address is not local or policy forbids the port.

`listen(1)` marks the socket as a listener. The numeric argument relates to queued connection attempts; it is not a maximum lifetime client count. Our later server uses a larger listen backlog plus a separate bounded application queue.

8.2.3 Accept and communicate

`accept()` blocks until a connection is available and returns a new socket plus the peer address. The listener and connection now have distinct jobs. The program receives some request bytes but does not interpret them. It sends a fixed response with `sendall`, closes the connection, then closes the listener as the process exits.

8.3 Constructing the response

The response has a status line, three fields, the blank line, and content:

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 32
Connection: close

Hello from a raw Python socket!
```

The source calculates the length from `BODY`. This matters even in a fixed example: editing content without editing a handwritten length would cause the client to wait for missing bytes or treat later bytes as a new message. The entire response is a `bytes` value before it reaches the socket.

Try it

Change the body to include a non-ASCII word. Keep it as a UTF-8 byte literal by encoding a Python string before constructing the response. Compare character count with `len(body)`, which counts bytes. Do not hardcode the new length.

8.4 Inspect rather than guess

An optional client such as `curl -v http://127.0.0.1:8080/` displays the request and response fields. If curl is unavailable, use the standard-library client later included in the project. Browser developer tools also show HTTP metadata, though a browser can add requests such as `/favicon.ico`.

Record three views in the lab notebook:

1. the source bytes we intend to send;
2. the server's printed peer endpoint;
3. the client's interpreted status, fields, and body.

If they disagree, locate the boundary: Python construction, socket transfer, or HTTP interpretation.

8.5 What this program does not know

The server ignores method, target, version, Host, fields, and content. It assumes one receive is enough before replying. It has no error response, timeout, second client, or persistent connection. It would answer a random string with 200 OK.

That is acceptable for one milestone because the limitation is named. The next stages replace assumptions one at a time. This incremental method is useful in systems work: keep a known-good end-to-end path while improving one boundary.

Chapter summary

The first server demonstrates the complete socket lifecycle and a valid fixed HTTP response. Resource ownership is explicit, output is bytes, body length is computed, and loopback limits exposure. It is a wire demonstration, not yet an HTTP implementation.

Review questions

1. Why does accept return a new socket instead of using the listener for data?
2. What can cause bind to fail?
3. Why does the response calculate Content-Length from bytes?
4. Which invalid inputs would still receive 200 from this snapshot?

Exercise 8.1: Predict before running

Before each experiment, predict the result: change the path, send POST, send an empty line instead of an HTTP request, connect and send nothing, and open two clients. Run only after writing predictions, then explain every mismatch.

Exercise 8.2: Resource lifetime

Draw the lifetime of listener and connection objects. Mark when the operating system owns queued connections, when Python owns a descriptor, and which `with` block guarantees each close.

CHAPTER 9

Receiving bytes without inventing boundaries

A correct server must tolerate a header arriving in any number of pieces. We replace the single-read assumption with an explicit buffer, delimiter search, and limit.

Learning objectives

- Accumulate partial reads until an HTTP header delimiter is complete.
- Treat connection close before completion as a protocol error.
- Enforce a cumulative byte limit rather than relying on receive-buffer size.
- Identify the bytes that belong after the header section.

9.1 The tempting incorrect version

A common first implementation calls `recv(4096)` and parses the result as one request. The number 4096 is a maximum returned by that call, not a promise that the peer's complete message fits or arrives together. A valid request can be split after any byte.

Another incorrect version loops until a receive returns fewer than 4096 bytes. A short read does not mark a message boundary. It only describes what was available at that moment. The client can send more later.

HTTP supplies the header boundary: `b"
r
n
r
n"`. Therefore the algorithm accumulates until it finds that exact sequence.

9.2 The buffered snapshot

```
"""Stage 2: buffer a complete header section across arbitrary recv calls."""

from __future__ import annotations

```

```python
import socket


HOST = "127.0.0.1"
PORT = 8080
MAX_HEADER_BYTES = 32 * 1024


def receive_header(connection: socket.socket) -> bytes:
    buffer = bytearray()
    while b"\r\n\r\n" not in buffer:
        chunk = connection.recv(4096)
        if not chunk:
            raise ValueError("client closed before the header was complete")
        buffer.extend(chunk)
        if len(buffer) > MAX_HEADER_BYTES:
            raise ValueError("header section is too large")
    marker = buffer.index(b"\r\n\r\n") + 4
    return bytes(buffer[:marker])


def make_response(status: int, reason: str, body: bytes) -> bytes:
    return (
        f"HTTP/1.1 {status} {reason}\r\n".encode("ascii")
        + b"Content-Type: text/plain; charset=utf-8\r\n"
        + f"Content-Length: {len(body)}\r\n".encode("ascii")
        + b"Connection: close\r\n\r\n"
        + body
    )


def main() -> None:
    with socket.socket() as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind((HOST, PORT))
        listener.listen()
        print(f"Listening on http://{HOST}:{PORT}")
        while True:
            connection, address = listener.accept()
            with connection:
                try:
                    header = receive_header(connection)
                    print(address, repr(header))
                    response = make_response(200, "OK", b"Header received\n")
                except ValueError as error:
                    response = make_response(400, "Bad Request", str(error).
    ↪ encode())
                connection.sendall(response)


if __name__ == "__main__":
    main()
```

The `bytearray` is mutable, so extending it does not create a new bytes object on every receive. Each loop iteration has four possible outcomes:

1. the delimiter is already present and the loop stops;
2. more bytes arrive and are appended;
3. an empty result reports end-of-stream before completion;
4. the cumulative size exceeds the configured limit.

After finding the delimiter, the function returns through its end. Notice a design flaw that is acceptable only at this stage: bytes after the delimiter are discarded. Those bytes might be a request body or the next request. Chapter 10 introduces a persistent incremental parser whose buffer survives and consumes exactly one framed message.

> **Pitfall**
>
> The receive chunk size and protocol limit solve different problems. A 4 KiB chunk is an I/O choice. A 32 KiB header limit is a policy applied to the total accumulated header. Increasing one does not correctly implement the other.

9.3 End-of-stream is data about the stream

When `recv` returns `b""`, the peer has performed an orderly close in that direction. If the parser has no buffered bytes, the connection ended between requests. If it has the beginning of a request, the message is incomplete and should not be dispatched.

A reset or other socket error follows a different exception path. In both cases the application must stop treating the stream as a source of a valid request. There is no invented “best effort” completion.

9.4 Delimiter search and strict CRLF

Searching for the correct delimiter finds a candidate header end. A strict parser must still reject bare line feeds or stray carriage returns inside the start and field lines. Merely finding one correct final delimiter does not make every earlier line correct.

Why be strict? Tolerant recipients often tolerate different things. If a reverse proxy treats a bare newline as a boundary while the origin does not, their stream interpretations diverge. Strictness reduces the space of ambiguous inputs.

9.5 Timeouts belong beside byte limits

A size limit stops a peer that sends too much. It does not stop a peer that sends one permitted byte every minute. A read timeout limits how long a worker waits for progress or for the next request. Later, each connected socket receives a five-second timeout and incomplete timed-out requests receive 408 when practical.

Timeout values express a service policy. Too short rejects legitimate slow clients; too long preserves resources for abusive or failed clients. Production systems coordinate timeouts across proxies, application servers, and upstream dependencies.

Try it

Use a tiny client or interactive TCP tool to type a request slowly. Observe that stage 2 waits because it has no timeout. Then mentally apply the final policy: after five seconds without completion, close the connection and release the worker.

9.6 Design the next buffer

The final parser needs an operation with different ownership:

- `feed(data)` appends newly received bytes and applies early limits;
- `pop_request()` returns one complete request or `None`;
- parsed bytes are removed, while later bytes remain;
- parser state belongs to one connection, not one receive call.

This API separates I/O from syntax. Unit tests can feed fragments without opening a real socket. The server loop can ask for another chunk only when the parser reports that the current buffer is incomplete.

Chapter summary

Receive operations expose stream availability, not HTTP boundaries. A correct reader accumulates bytes, searches for protocol delimiters, handles early close, and enforces cumulative limits and timeouts. It must preserve bytes beyond one message for later parsing.

Review questions

1. Why is a short receive not an end-of-message signal?
2. What is wrong with discarding bytes after the header delimiter?
3. Which attacks are addressed by byte limits and which by timeouts?
4. Why should parser state belong to a connection?

Exercise 9.1: Fragment table

Take a small GET request and divide it at every possible byte boundary. For each division, show the buffer before and after the second fragment. The parsing result must be identical for every division.

Exercise 9.2: Limit placement

Explain where to enforce request-line, total-header, field-count, and body limits. For each, state the earliest moment the server has enough information to reject the request without waiting for more bytes.

CHAPTER 10

Building an incremental request parser

> Parsing turns hostile bytes into a trusted internal structure. The parser's most important output is not convenience; it is one unambiguous interpretation of exactly one message.

Learning objectives

- Separate incremental buffering, syntactic parsing, framing, and semantic validation.
- Build structured requests without losing repeated fields or trailing bytes.
- Reject malformed lines, unsupported versions, conflicting framing, and invalid targets.
- Test the parser independently of sockets with fragmented inputs.

10.1 From a teaching parser to a strict parser

Stage 3 introduces a readable request dataclass and a direct parsing function. It is intentionally smaller than the final implementation.

```
"""Stage 3: parse method, target, version, fields, and a length-framed body."""

from __future__ import annotations

import socket
from dataclasses import dataclass


HOST = "127.0.0.1"
PORT = 8080


@dataclass
class Request:
```

```
    method: str
    target: str
    version: str
    headers: dict[str, str]
    body: bytes


def parse_head(head: bytes) -> tuple[str, str, str, dict[str, str]]:
    lines = head.split(b"\r\n")
    try:
        method_raw, target_raw, version_raw = lines[0].split(b" ")
    except ValueError as error:
        raise ValueError("malformed request line") from error
    method = method_raw.decode("ascii")
    target = target_raw.decode("ascii")
    version = version_raw.decode("ascii")
    if version != "HTTP/1.1":
        raise ValueError("only HTTP/1.1 is supported")
    headers: dict[str, str] = {}
    for line in lines[1:]:
        name, separator, value = line.partition(b":")
        if not separator:
            raise ValueError("malformed header")
        key = name.decode("ascii").casefold()
        if key in headers:
            raise ValueError("duplicate header")
        headers[key] = value.decode("iso-8859-1").strip()
    if not headers.get("host"):
        raise ValueError("Host is required")
    return method, target, version, headers


def receive_request(connection: socket.socket) -> Request:
    buffer = bytearray()
    while b"\r\n\r\n" not in buffer:
        chunk = connection.recv(4096)
        if not chunk:
            raise ValueError("incomplete header")
        buffer.extend(chunk)
        if len(buffer) > 32 * 1024:
            raise ValueError("header too large")
    marker = buffer.index(b"\r\n\r\n")
    method, target, version, headers = parse_head(bytes(buffer[:marker]))
    raw_length = headers.get("content-length", "0")
    if not raw_length.isdecimal():
        raise ValueError("invalid Content-Length")
    length = int(raw_length)
    if length > 1024 * 1024:
        raise ValueError("body too large")
    body_start = marker + 4
    while len(buffer) - body_start < length:
        chunk = connection.recv(4096)
        if not chunk:
            raise ValueError("incomplete body")
```

```
        buffer.extend(chunk)
    return Request(method, target, version, headers, bytes(buffer[body_start:
    ↪ body_start + length]))


def response(body: bytes, status: str = "200 OK") -> bytes:
    return (
        f"HTTP/1.1 {status}\r\nContent-Type: text/plain; charset=utf-8\r\n"
        f"Content-Length: {len(body)}\r\nConnection: close\r\n\r\n".encode()
        + body
    )


def main() -> None:
    with socket.socket() as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind((HOST, PORT))
        listener.listen()
        print(f"Listening on http://{HOST}:{PORT}")
        while True:
            connection, _ = listener.accept()
            with connection:
                try:
                    request = receive_request(connection)
                    body = f"{request.method} {request.target}\n".encode()
                    connection.sendall(response(body))
                except ValueError as error:
                    connection.sendall(response(str(error).encode(), "400 Bad
    ↪  Request"))


if __name__ == "__main__":
    main()
```

The stage establishes the right order: find the header boundary, parse the start line and fields, determine content length, receive the declared body, then create a request. It still uses a one-value dictionary for fields, does not preserve leftover bytes, and handles errors with broad 400 responses. Those limitations motivate the final types rather than remaining hidden.

10.2 The final parsing contract

`RequestParser` owns one byte buffer for one TCP connection. Calling `feed` never promises a complete request. Calling `pop_request` returns either one structured request or `None`. A protocol violation raises `HTTPError` with a safe status and detail.

The algorithm is:

1. find the first CRLF–CRLF header boundary;
2. if absent, apply early line/header limits and request more bytes;
3. validate strict line endings;
4. parse the request line and require HTTP/1.1;

5. parse each field into a multi-value collection and require one Host;
6. determine body length or reject unsupported/ambiguous framing;
7. if the declared body is incomplete, return `None`;
8. copy exactly the body, delete exactly the consumed prefix, parse the target, and construct `Request`.

This order avoids interpreting a partial body and preserves a following request.

10.3 Read the complete final parser

```
"""Incremental, strict parsing for the supported HTTP/1.1 request subset."""

from __future__ import annotations

import re
from dataclasses import dataclass
from urllib.parse import parse_qs, unquote, urlsplit

from .models import Headers, HTTPError, Request


TOKEN = re.compile(rb"^[!#$%&'*+\-.^_`|~0-9A-Za-z]+$")
INVALID_VALUE = re.compile(r"[\x00-\x08\x0a-\x1f\x7f]")


@dataclass(frozen=True, slots=True)
class ParserLimits:
    request_line: int = 8 * 1024
    header_bytes: int = 32 * 1024
    header_count: int = 100
    body_bytes: int = 1024 * 1024


class RequestParser:
    """Accept bytes over time and return complete requests one at a time."""

    def __init__(self, limits: ParserLimits | None = None) -> None:
        self.limits = limits or ParserLimits()
        self._buffer = bytearray()

    @property
    def has_buffered_data(self) -> bool:
        return bool(self._buffer)

    def feed(self, data: bytes) -> None:
        self._buffer.extend(data)
        if b"\r\n" not in self._buffer and len(self._buffer) > self.limits.
    ↪ request_line:
            raise HTTPError(414, "The request target is too long.", code="
    ↪ uri_too_long")
```

```
        if b"\r\n\r\n" not in self._buffer and len(self._buffer) > self.
    ↪ limits.header_bytes:
            raise HTTPError(
                431,
                "The request header section is too large.",
                code="headers_too_large",
            )

    def pop_request(
        self,
        client: tuple[str, int] | None = None,
    ) -> Request | None:
        marker = self._buffer.find(b"\r\n\r\n")
        if marker < 0:
            return None
        head_end = marker + 4
        if head_end > self.limits.header_bytes:
            raise HTTPError(
                431,
                "The request header section is too large.",
                code="headers_too_large",
            )
        head = bytes(self._buffer[:marker])
        if self._contains_bare_line_break(head):
            raise HTTPError(400, "HTTP lines must end with CRLF.", code="
    ↪ invalid_line_end")
        lines = head.split(b"\r\n")
        if not lines or len(lines[0]) > self.limits.request_line:
            raise HTTPError(414, "The request line is too long.", code="
    ↪ uri_too_long")
        method, target, version = self._parse_request_line(lines[0])
        headers = self._parse_headers(lines[1:])
        content_length = self._content_length(headers)
        total = head_end + content_length
        if len(self._buffer) < total:
            return None
        body = bytes(self._buffer[head_end:total])
        del self._buffer[:total]
        path, query = self._parse_target(target)
        return Request(
            method=method,
            target=target,
            version=version,
            headers=headers,
            body=body,
            path=path,
            query=query,
            client=client,
        )

    @staticmethod
    def _contains_bare_line_break(value: bytes) -> bool:
        return re.search(rb"(?<!\r)\n|\r(?!\n)", value) is not None

```

```
    @staticmethod
    def _parse_request_line(line: bytes) -> tuple[str, str, str]:
        parts = line.split(b" ")
        if len(parts) != 3 or not all(parts):
            raise HTTPError(400, "Malformed request line.", code="
    ↪ invalid_request_line")
        method_raw, target_raw, version_raw = parts
        if not TOKEN.fullmatch(method_raw):
            raise HTTPError(400, "Invalid request method.", code="
    ↪ invalid_method")
        try:
            method = method_raw.decode("ascii")
            target = target_raw.decode("ascii")
            version = version_raw.decode("ascii")
        except UnicodeDecodeError as error:
            raise HTTPError(
                400,
                "The request line must contain ASCII bytes.",
                code="invalid_request_line",
            ) from error
        if version != "HTTP/1.1":
            raise HTTPError(
                505,
                "This educational server supports HTTP/1.1 only.",
                code="http_version_not_supported",
            )
        return method, target, version

    def _parse_headers(self, lines: list[bytes]) -> Headers:
        if len(lines) > self.limits.header_count:
            raise HTTPError(431, "Too many header fields.", code="
    ↪ too_many_headers")
        headers = Headers()
        for line in lines:
            if line.startswith((b" ", b"\t")):
                raise HTTPError(
                    400,
                    "Obsolete folded header lines are not accepted.",
                    code="folded_header",
                )
            name_raw, separator, value_raw = line.partition(b":")
            if not separator or not TOKEN.fullmatch(name_raw):
                raise HTTPError(400, "Malformed header field.", code="
    ↪ invalid_header")
            name = name_raw.decode("ascii")
            value = value_raw.decode("iso-8859-1").strip(" \t")
            if INVALID_VALUE.search(value):
                raise HTTPError(
                    400,
                    "A header contains a forbidden control character.",
                    code="invalid_header_value",
                )
            headers.add(name, value)
        hosts = headers.get_all("Host")
```

```
        if len(hosts) != 1 or not hosts[0]:
            raise HTTPError(
                400,
                "HTTP/1.1 requires exactly one non-empty Host header.",
                code="invalid_host",
            )
        return headers

    def _content_length(self, headers: Headers) -> int:
        transfer_encodings = headers.get_all("Transfer-Encoding")
        lengths = headers.get_all("Content-Length")
        if transfer_encodings and lengths:
            raise HTTPError(
                400,
                "Transfer-Encoding and Content-Length cannot be combined.",
                code="ambiguous_framing",
            )
        if transfer_encodings:
            raise HTTPError(
                501,
                "Transfer-Encoding is explained in the book but not
    ↪ implemented.",
                code="transfer_encoding_not_implemented",
            )
        if len(lengths) > 1:
            raise HTTPError(
                400,
                "Duplicate Content-Length fields are rejected.",
                code="ambiguous_content_length",
            )
        if not lengths:
            return 0
        try:
            length = int(lengths[0], 10)
        except ValueError as error:
            raise HTTPError(
                400,
                "Content-Length must be a non-negative decimal integer.",
                code="invalid_content_length",
            ) from error
        if length < 0:
            raise HTTPError(
                400,
                "Content-Length cannot be negative.",
                code="invalid_content_length",
            )
        if length > self.limits.body_bytes:
            raise HTTPError(
                413,
                "The request body exceeds the configured limit.",
                code="body_too_large",
            )
        return length

```

```
    @staticmethod
    def _parse_target(target: str) -> tuple[str, dict[str, list[str]]]:
        if target == "*":
            return "*", {}
        if not target.startswith("/"):
            raise HTTPError(
                400,
                "Only origin-form request targets are supported.",
                code="unsupported_request_target",
            )
        parsed = urlsplit(target)
        if parsed.scheme or parsed.netloc or parsed.fragment:
            raise HTTPError(400, "Invalid origin-form target.", code="
    ↪ invalid_target")
        if re.search(r"%(?![0-9A-Fa-f]{2})", target):
            raise HTTPError(
                400,
                "The request target contains an invalid percent escape.",
                code="invalid_percent_encoding",
            )
        try:
            segments = [unquote(segment, errors="strict") for segment in
    ↪ parsed.path.split("/")]
        except UnicodeDecodeError as error:
            raise HTTPError(
                400,
                "The request path contains invalid UTF-8 percent encoding.",
                code="invalid_path_encoding",
            ) from error
        path = "/".join(segments) or "/"
        try:
            query = parse_qs(
                parsed.query,
                keep_blank_values=True,
                strict_parsing=False,
                encoding="utf-8",
                errors="strict",
            )
        except UnicodeDecodeError as error:
            raise HTTPError(
                400,
                "The query string contains invalid UTF-8 percent encoding.",
                code="invalid_query_encoding",
            ) from error
        return path, query
```

10.3.1 Tokens and line syntax

The method and field name use a token allow-list. The request line must contain exactly three non-empty space-separated components. This intentionally rejects tabs and extra spaces instead of trying to guess intent. Version mismatches receive 505 rather than falling through into a parser designed for different syntax.

Header values are decoded with ISO-8859-1 as a byte-preserving mapping for the HTTP/1.x field octets accepted by this implementation. Individual application fields still need their own semantics. Control characters and obsolete folding are rejected.

10.3.2 Framing before content

The parser inspects every occurrence of `Transfer-Encoding` and `Content-Length`. It never accepts both. It rejects repeated lengths and does not wait for a body whose declared size already exceeds the limit. Only a single valid decimal length is used to slice the stream.

This decision belongs in the HTTP parser, not in a JSON handler. A handler should receive only a body whose byte boundary is already established.

10.3.3 Targets and decoding

The parser accepts a path beginning with slash or the asterisk target. It rejects an absolute URI because this server is an origin, not a forward proxy. It rejects fragments, which are client-side URI components and should not be sent as HTTP request targets.

Percent escapes are validated before decoding. Path segments are decoded separately so an encoded slash inside a segment does not secretly create a new routing segment. Query parsing preserves repeated values in lists. Invalid UTF-8 becomes a 400 problem instead of leaking a Unicode exception.

10.4 Internal trust is earned in layers

After parser success, code may trust that the request has one message boundary, a legal supported version, valid field syntax, and a parsed target. It may not yet trust that JSON is well formed, a task status is allowed, a referenced task exists, or a user is authorized. Each layer narrows uncertainty according to its expertise.

Mental model

Validation is not one function at the edge. Transport validates stream progress; HTTP validates message syntax and framing; content code validates media syntax; the application validates domain rules; storage enforces durable constraints.

10.5 Parser tests as executable grammar examples

The test suite feeds requests one byte at a time, puts two requests in one feed, tests repeated fields, rejects missing Host, rejects transfer coding, and applies limits before waiting for an oversized body. These cases are more valuable than testing only a browser's ordinary GET.

For every accepted example, add fragmentation variants. For every rejected example, assert both status and reason category. A parser test should not depend on timing or the operating system's packet choices.

Try it

From `code/final`, run:

```
python -m unittest tests.test_models_parser -v
```

Then add a test for a malformed percent escape such as `/tasks/%ZZ`. Make the expectation explicit before running it.

Chapter summary

The incremental parser owns a connection buffer and consumes exactly one framed message. It validates strict start/field syntax, preserves repeated fields, rejects ambiguous framing, enforces limits early, and produces decoded path and query structures. Later layers receive a safer request, not magically trusted application data.

Review questions

1. Why can `pop_request` return `None` without indicating an error?
2. Which bytes remain after one request is removed from the buffer?
3. Why are percent escapes checked before decoding?
4. What validation still remains after HTTP parsing succeeds?

Exercise 10.1: Adversarial table

Create a table of at least twelve requests: four valid, four incomplete, and four invalid. Include duplicate lengths, invalid Host, a large declared body, an encoded path, and two buffered requests. Record expected parser outcome and status where applicable.

Exercise 10.2: Invariant proof

State an invariant for the parser buffer before and after `pop_request`. Explain why deleting exactly `header_end + content_length` bytes preserves that invariant for persistent connections.

CHAPTER 11

Responses as values and bytes

> Handlers should describe a result; one serializer should enforce wire rules. Centralizing response construction prevents every route from reimplementing HTTP.

Learning objectives

- Model status, fields, and body independently of socket I/O.
- Serialize legal status lines and fields without response splitting.
- Implement Content-Length, HEAD, no-content statuses, and connection fields correctly.
- Keep internal error detail out of public 500 responses.

11.1 The response object

A handler should return a `Response`, not write to the socket. The response contains an integer status, a multi-value `Headers` collection, and a bytes body. This separation gives middleware a chance to inspect or replace the result, lets tests call handlers without networking, and gives one serializer authority over HTTP framing.

Stage 4 isolates the concept in a small program:

```
"""Stage 4: represent and serialize responses rather than joining ad hoc
    ↪ bytes."""

from __future__ import annotations

import socket
from dataclasses import dataclass, field
from http import HTTPStatus


HOST = "127.0.0.1"
PORT = 8080


@dataclass
```

```
class Response:
    status: int = 200
    body: bytes = b""
    headers: dict[str, str] = field(default_factory=dict)


def serialize(response: Response, *, head_request: bool = False) -> bytes:
    reason = HTTPStatus(response.status).phrase
    headers = dict(response.headers)
    headers.setdefault("Content-Type", "text/plain; charset=utf-8")
    headers["Content-Length"] = str(len(response.body))
    headers["Connection"] = "close"
    start = f"HTTP/1.1 {response.status} {reason}\r\n".encode("ascii")
    fields = b"".join(
        f"{name}: {value}\r\n".encode("iso-8859-1")
        for name, value in headers.items()
    )
    body = b"" if head_request else response.body
    return start + fields + b"\r\n" + body


def parse_request_line(data: bytes) -> tuple[str, str]:
    line = data.split(b"\r\n", 1)[0]
    method, target, version = line.decode("ascii").split(" ")
    if version != "HTTP/1.1":
        raise ValueError("unsupported HTTP version")
    return method, target


def main() -> None:
    with socket.socket() as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind((HOST, PORT))
        listener.listen()
        print(f"Listening on http://{HOST}:{PORT}")
        while True:
            connection, _ = listener.accept()
            with connection:
                request_data = connection.recv(32 * 1024)
                try:
                    method, target = parse_request_line(request_data)
                    answer = Response(body=f"You requested {target}\n".encode
    ↪ ())
                except (UnicodeError, ValueError):
                    method = "GET"
                    answer = Response(status=400, body=b"Bad request\n")
                connection.sendall(serialize(answer, head_request=method == "
    ↪ HEAD"))


if __name__ == "__main__":
    main()
```

The stage already computes content length, uses the standard status phrase, and sends no body for HEAD. The final serializer adds date/server fields, persistent connection policy, special no-content statuses, and header validation.

11.2 Headers must be safe to serialize

Response splitting becomes possible when untrusted text inserts carriage return or line feed into a field value. A value such as `"safe
r
nX-Injected: yes"` would create a second field if serialized blindly. The final `Headers.add` requires a legal token name and rejects control characters in values.

This validation occurs when the collection is built, not after bytes have been joined. As a result, every response producer shares the same invariant: header items can be represented as one field line each.

11.3 One serialization path

The server's response method performs these steps:

1. resolve the standard reason phrase for the numeric status;
2. copy fields so server-added metadata does not mutate the handler's object;
3. set Date, Server, and Connection;
4. determine whether the status permits content;
5. set Content-Length to the representation-body length when permitted;
6. suppress transmitted bytes for HEAD while preserving that length;
7. encode the start line as ASCII and fields as ISO-8859-1;
8. join one blank CRLF line and the permitted body, then call `sendall`.

The serializer operates on bytes at its final boundary. It does not call `json.dumps`; JSON is a response factory concern.

11.4 HEAD and no-content statuses differ

For HEAD, the server computes the response that GET would have produced and suppresses content bytes. A recipient can learn the media type and expected length without transferring the representation.

For 204 and 304, the status semantics prohibit message content. The serializer suppresses the body and removes `Content-Length` in this implementation. A handler that mistakenly attaches bytes cannot violate the wire rule.

> **Pitfall**
>
> Do not change a HEAD response's Content-Length to zero merely because zero body bytes are transmitted. That field describes the selected representation a GET would have carried, enabling useful metadata checks.

11.5 JSON and problem factories

`Response.json(value)` uses compact separators, preserves Unicode, encodes UTF-8, and sets `application/json; charset=utf-8`. A central factory means all handlers use the same encoding and media type.

`Response.problem` creates an RFC 9457-style problem document with `type`, `title`, `status`, `detail`, optional `instance`, and optional field errors. It changes the media type to `application/problem+json`. Clients can handle errors structurally instead of scraping prose.

An expected `HTTPError` carries public-safe detail. An unexpected exception is logged internally and converted to a generic 500 problem. Stack traces, database paths, SQL statements, and secrets must not cross the response boundary.

On the wire

An invalid task might receive:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json; charset=utf-8
Content-Length: ...

{"type":"urn:task-api:problem:validation-error",
 "title":"Validation failed","status":422,
 "detail":"One or more values failed validation.",
 "errors":{"title":"Must be a non-empty string."}}
```

The line wrapping above is for print; the actual JSON is one compact byte sequence.

11.6 Test meaning and wire form separately

Handler tests should assert response status, fields, and decoded JSON. Serializer tests should assert exact CRLF framing, length, HEAD suppression, 204 behavior, and connection policy. Keeping these tests separate reveals which abstraction failed.

Chapter summary

Response values decouple application decisions from network serialization. Centralized header validation and serialization enforce length, HEAD, no-content, and connection rules consistently. JSON and problem factories produce predictable media types while unexpected failures remain private.

Review questions

1. Why should a handler not write directly to a connection?
2. How can a newline in a field value become a vulnerability?
3. How do HEAD and 204 differ in their treatment of Content-Length?
4. Which error information belongs in logs but not a 500 response?

Exercise 11.1: Serializer matrix

Create expected start line, Content-Length presence, and transmitted body length for GET/200, HEAD/200, GET/204, GET/304, and GET/500. Turn the matrix into unit tests before changing serializer code.

Exercise 11.2: Injection attempt

Construct a response field whose value contains CRLF. Verify that `Headers` rejects it before serialization. Explain why percent-encoding that input would not be a general replacement for validation.

CHAPTER 12

Routing methods and resource paths

The HTTP parser identifies a target; the router maps method and path shape to application code. Correct routing distinguishes absence from a method mismatch and makes precedence deterministic.

Learning objectives

- Register handlers by method and segment pattern.
- Match static and parameter segments with deterministic precedence.
- Distinguish 404, 405, automatic HEAD, and OPTIONS behavior.
- Keep query filtering outside path-shape matching.

12.1 A small first router

Stage 5 introduces a decorator, a route table, and `{task_id}` capture:

```
"""Stage 5: route structured requests by method and segment pattern."""

from __future__ import annotations

import socket
from dataclasses import dataclass
from http import HTTPStatus
from typing import Callable
from urllib.parse import urlsplit


HOST = "127.0.0.1"
PORT = 8080


@dataclass
class Request:
    method: str
```

```
    path: str
    params: dict[str, str]


@dataclass
class Response:
    status: int
    body: bytes


Handler = Callable[[Request], Response]
ROUTES: list[tuple[str, tuple[str, ...], Handler]] = []


def route(method: str, pattern: str):
    def register(handler: Handler) -> Handler:
        ROUTES.append((method, tuple(segment for segment in pattern.split("/")
    ↪  if segment), handler))
        return handler
    return register


def resolve(method: str, path: str) -> tuple[Handler | None, dict[str, str]]:
    actual = tuple(segment for segment in path.split("/") if segment)
    for expected_method, pattern, handler in ROUTES:
        if method != expected_method or len(actual) != len(pattern):
            continue
        params: dict[str, str] = {}
        for expected, value in zip(pattern, actual, strict=True):
            if expected.startswith("{"):
                params[expected[1:-1]] = value
            elif expected != value:
                break
        else:
            return handler, params
    return None, {}


@route("GET", "/hello")
def hello(request: Request) -> Response:
    return Response(200, b"Hello, router!\n")


@route("GET", "/tasks/{task_id}")
def task(request: Request) -> Response:
    return Response(200, f"Task {request.params['task_id']}\n".encode())


def handle(data: bytes) -> Response:
    try:
        request_line = data.split(b"\r\n", 1)[0].decode("ascii")
        method, target, version = request_line.split(" ")
        if version != "HTTP/1.1":
            return Response(505, b"HTTP version not supported\n")
```

```
        path = urlsplit(target).path
        handler, params = resolve(method, path)
        if handler is None:
            return Response(404, b"Not found\n")
        return handler(Request(method, path, params))
    except (UnicodeError, ValueError):
        return Response(400, b"Bad request\n")


def serialize(response: Response) -> bytes:
    reason = HTTPStatus(response.status).phrase
    return (
        f"HTTP/1.1 {response.status} {reason}\r\n"
        "Content-Type: text/plain; charset=utf-8\r\n"
        f"Content-Length: {len(response.body)}\r\n"
        "Connection: close\r\n\r\n".encode()
        + response.body
    )


def main() -> None:
    with socket.socket() as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind((HOST, PORT))
        listener.listen()
        print(f"Listening on http://{HOST}:{PORT}")
        while True:
            connection, _ = listener.accept()
            with connection:
                connection.sendall(serialize(handle(connection.recv(32 * 1024)
    ↪ )))


if __name__ == "__main__":
    main()
```

The decorator records configuration at import time and returns the original handler, so the function remains directly testable. Resolution splits paths into segments and matches each against either literal text or a parameter placeholder.

The stage is intentionally naive: registration order can let a parameter route shadow a static route, it reports every miss as 404, and it knows nothing about HEAD or OPTIONS. The final router addresses each point.

12.2 Static routes are more specific

Consider `/users/me` and `/users/{user_id}`. Both structurally match the path `/users/me`, but the literal route expresses a more specific resource. The final router sorts candidates by parameter count and then stable registration order. Static routes therefore win without making behavior depend on an accidental decorator order.

Only whole segments can be parameters. Patterns such as `/files/{name}.json` are outside the small router's grammar. Parameter names must be valid, unique Python identifiers. These rules keep matching easy to teach and reason about.

```
"""A small segment router with static-route precedence."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable

from .models import Request, Response


Handler = Callable[[Request], Response]


@dataclass(slots=True)
class Route:
    method: str
    pattern: str
    segments: tuple[str, ...]
    handler: Handler
    parameter_count: int
    order: int

    def match(self, path: str) -> dict[str, str] | None:
        path_segments = tuple(segment for segment in path.split("/") if segment)
        if len(path_segments) != len(self.segments):
            return None
        parameters: dict[str, str] = {}
        for expected, actual in zip(self.segments, path_segments, strict=True):
            if expected.startswith("{") and expected.endswith("}"):
                parameters[expected[1:-1]] = actual
            elif expected != actual:
                return None
        return parameters


class Router:
    def __init__(self) -> None:
        self._routes: list[Route] = []

    def add(self, method: str, pattern: str, handler: Handler) -> None:
        if not pattern.startswith("/"):
            raise ValueError("Route patterns must start with /")
        segments = tuple(segment for segment in pattern.split("/") if segment)
        names = [
            segment[1:-1]
            for segment in segments
```

```
            if segment.startswith("{") and segment.endswith("}")
        ]
        if any(not name.isidentifier() for name in names) or len(names) != len(set(names)):
            raise ValueError(f"Invalid route parameter in {pattern!r}")
        normalized = method.upper()
        if any(route.method == normalized and route.pattern == pattern for route in self._routes):
            raise ValueError(f"Duplicate route: {normalized} {pattern}")
        self._routes.append(
            Route(
                method=normalized,
                pattern=pattern,
                segments=segments,
                handler=handler,
                parameter_count=len(names),
                order=len(self._routes),
            )
        )
        self._routes.sort(key=lambda route: (route.parameter_count, route.order))

    def resolve(
        self,
        method: str,
        path: str,
    ) -> tuple[Handler | None, dict[str, str], set[str]]:
        allowed: set[str] = set()
        candidate: tuple[Handler, dict[str, str]] | None = None
        for route in self._routes:
            parameters = route.match(path)
            if parameters is None:
                continue
            allowed.add(route.method)
            if route.method == method and candidate is None:
                candidate = (route.handler, parameters)
        if "GET" in allowed:
            allowed.add("HEAD")
        if allowed:
            allowed.add("OPTIONS")
        if candidate is None and method == "HEAD":
            return self.resolve("GET", path)
        if candidate is None:
            return None, {}, allowed
        return candidate[0], candidate[1], allowed
```

12.3 Path match and method match are separate facts

Resolution gathers methods allowed by every route whose path pattern matches. If no pattern matches, the result is 404. If a pattern matches but not for the request method, the result is 405 with an `Allow` field. This distinction helps clients discover and correct method use.

When GET is available, HEAD is also allowed. A HEAD request falls back to the GET handler, after which the serializer suppresses content bytes. OPTIONS receives an automatically generated 204 response with allowed methods. The application does not require every resource handler to repeat protocol mechanics.

12.4 Path parameters are strings first

The router captures `42` from `/tasks/42` as a string. The Task layer then decides that identifiers must be positive base-10 integers. This ownership matters: a generic router should not assume every parameter is an integer, UUID, slug, or date.

After a match, parameters are placed on the `Request`. A handler receives one object rather than a changing positional signature. More advanced frameworks offer typed converters and dependency injection; the explicit request structure keeps this project's data flow visible.

12.5 Queries refine a resource request

The path chooses the collection `/tasks`. Query parameters such as `status=todo`, `limit=20`, and `offset=0` refine the requested representation. They do not participate in route selection. The parser provides a mapping from each query name to a list of values so the application can reject unexpected repetition instead of silently choosing one.

> **Pitfall**
>
> Do not route on a raw target string. A target can contain a query, percent encoding, and repeated values. Parse the target once, match the decoded path by well-defined segment rules, and validate query semantics at the resource layer.

12.6 Routing is not authorization

Finding a handler means only that code exists for a method/path shape. A future authorization layer would still decide whether the authenticated principal may act on that particular resource. Returning 404 instead of 403 to hide existence is an application security policy, not a router inevitability.

Chapter summary

The router maps method and decoded path segments to handlers, with static routes ahead of parameter routes. It reports path absence separately from method mismatch, supplies HEAD/OPTIONS behavior, and leaves parameter/query meaning to the application.

Review questions

1. Why should `/users/me` win over `/users/{user_id}`?
2. What evidence distinguishes 404 from 405?
3. Why does the router leave task ID conversion to the handler layer?
4. Why should query parameters not be part of the route pattern here?

Exercise 12.1: Route conflict table

Register static and parameter routes for tasks, task statistics, and one task. For ten paths and four methods, predict handler, captured parameters, allowed methods, and final status. Then encode the table as tests.

Exercise 12.2: Add a nested resource

Design patterns for `/projects/{project_id}/tasks` and one nested task. Identify which parameters the current router captures and which validation remains in application code. Do not implement a database relation yet.

CHAPTER 13

JSON, error boundaries, and middleware

A backend framework becomes useful when handlers can work with structured content, expected failures become consistent responses, and cross-cutting behavior can wrap every route.

Learning objectives

- Validate media type and UTF-8 before decoding JSON.
- Separate malformed content, invalid domain values, and unexpected failures.
- Compose middleware around a terminal router handler.
- Preserve a clear request-to-response data flow.

13.1 JSON is one layer, not the wire protocol

JSON parsing begins only after HTTP framing produces exact body bytes. The request helper verifies that `Content-Type` names `application/json`, decodes UTF-8, then calls the standard-library JSON parser. These failures have different causes:

- wrong or missing media type: 415;
- invalid UTF-8: 400;
- malformed JSON grammar: 400;
- valid JSON with wrong task fields: 422;

A JSON number, list, string, or null is syntactically valid. A Task endpoint requires an object, so domain validation must still check the top-level type.

13.2 A transitional application layer

Stage 6 demonstrates JSON factories, a terminal dispatcher, and timing middleware in one file:

```
"""Stage 6: JSON responses, errors, handlers, and composable middleware."""

```

```
from __future__ import annotations

import json
import socket
import time
from dataclasses import dataclass
from http import HTTPStatus
from typing import Any, Callable
from urllib.parse import urlsplit


@dataclass
class Request:
    method: str
    path: str


@dataclass
class Response:
    status: int
    body: bytes
    content_type: str = "application/json; charset=utf-8"

    @classmethod
    def json(cls, value: Any, status: int = 200) -> "Response":
        return cls(status, json.dumps(value, separators=(",", ":")).encode())


Handler = Callable[[Request], Response]
Middleware = Callable[[Request, Handler], Response]


def dispatch(request: Request) -> Response:
    if request.method == "GET" and request.path == "/health":
        return Response.json({"status": "ok"})
    return Response.json(
        {"type": "about:blank", "title": "Not Found", "status": 404},
        status=404,
    )


def timing(request: Request, next_handler: Handler) -> Response:
    started = time.monotonic()
    response = next_handler(request)
    elapsed = (time.monotonic() - started) * 1000
    print(request.method, request.path, response.status, f"{elapsed:.1f}ms")
    return response


def compose(handler: Handler, middleware: list[Middleware]) -> Handler:
    for current in reversed(middleware):
        next_handler = handler
        handler = lambda request, current=current, next_handler=next_handler:
    ↪ current(request, next_handler)
```

```
    return handler


def handle(data: bytes) -> Response:
    try:
        line = data.split(b"\r\n", 1)[0].decode("ascii")
        method, target, version = line.split(" ")
        if version != "HTTP/1.1":
            return Response.json({"title": "Version Not Supported", "status": 505}, 505)
        request = Request(method, urlsplit(target).path)
        return compose(dispatch, [timing])(request)
    except (UnicodeError, ValueError):
        return Response.json({"title": "Bad Request", "status": 400}, 400)


def serialize(response: Response) -> bytes:
    return (
        f"HTTP/1.1 {response.status} {HTTPStatus(response.status).phrase}\r\n"
        f"Content-Type: {response.content_type}\r\n"
        f"Content-Length: {len(response.body)}\r\n"
        "Connection: close\r\n\r\n".encode()
        + response.body
    )


def main() -> None:
    with socket.socket() as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind(("127.0.0.1", 8080))
        listener.listen()
        print("Listening on http://127.0.0.1:8080")
        while True:
            connection, _ = listener.accept()
            with connection:
                connection.sendall(serialize(handle(connection.recv(32 * 1024))))


if __name__ == "__main__":
    main()
```

The lambda defaults in `compose` are deliberate. Python closures capture variables by reference; binding each current middleware and next handler as a default prevents every wrapper from later referring to the final loop values. The final `Application` uses a named wrapper with the same idea for clarity.

13.3 Middleware is nested control flow

Given middleware A and B, request flow is:

```
A before
  B before
    route handler
  B after
A after
```

Each middleware receives a request and the next handler. It may modify the request within documented bounds, return early, call the next handler once, inspect or replace the response, or translate an expected exception. Logging, request IDs, authentication, CORS, compression, and metrics are common examples, though not all belong in this small project.

Order matters. An exception-recovery layer must surround code whose exceptions it catches. A timing layer must begin before and finish after its measured work. An authentication layer placed after a handler is not authentication.

13.4 The final application boundary

```
"""Routing, middleware composition, and safe application error handling."""

from __future__ import annotations

import logging
from collections.abc import Callable

from .models import HTTPError, Request, Response
from .router import Handler, Router


Middleware = Callable[[Request, Handler], Response]
SUPPORTED_METHODS = {"GET", "HEAD", "POST", "PUT", "PATCH", "DELETE", "
    ↪ OPTIONS"}


class Application:
    def __init__(self) -> None:
        self.router = Router()
        self._middleware: list[Middleware] = []
        self.logger = logging.getLogger("tinyhttp.application")

    def route(self, method: str, pattern: str) -> Callable[[Handler], Handler
    ↪ ]:
        def register(handler: Handler) -> Handler:
            self.router.add(method, pattern, handler)
            return handler

        return register

    def add_middleware(self, middleware: Middleware) -> None:
        self._middleware.append(middleware)

    def handle(self, request: Request) -> Response:
```

```
        try:
            if request.method not in SUPPORTED_METHODS:
                raise HTTPError(
                    501,
                    f"Method {request.method} is not implemented.",
                    code="method_not_implemented",
                )
            handler = self._dispatch
            for middleware in reversed(self._middleware):
                next_handler = handler

                def wrapped(
                    current: Request,
                    middleware: Middleware = middleware,
                    next_handler: Handler = next_handler,
                ) -> Response:
                    return middleware(current, next_handler)

                handler = wrapped
            response = handler(request)
            if not isinstance(response, Response):
                raise TypeError("Handlers must return a Response")
            return response
        except HTTPError as error:
            response = Response.problem(
                error.status,
                error.detail,
                instance=request.path,
                code=error.code,
                title=error.title,
                errors=error.errors,
                type_uri=error.type_uri,
            )
            if error.status == 405:
                _, _, allowed = self.router.resolve(request.method, request.
    ↪ path)
                response.headers.set("Allow", ", ".join(sorted(allowed)))
            return response
        except Exception:
            self.logger.exception("Unhandled application exception")
            return Response.problem(
                500,
                "The server encountered an unexpected error.",
                instance=request.path,
                code="internal_server_error",
            )

    def _dispatch(self, request: Request) -> Response:
        if request.path == "*" and request.method == "OPTIONS":
            response = Response(status=204)
            response.headers.set("Allow", ", ".join(sorted(SUPPORTED_METHODS))
    ↪ )
            return response
```

```
        handler, parameters, allowed = self.router.resolve(request.method,
    ↪ request.path)
        if request.method == "OPTIONS" and allowed:
            response = Response(status=204)
            response.headers.set("Allow", ", ".join(sorted(allowed)))
            return response
        if handler is None and allowed:
            raise HTTPError(
                405,
                f"Method {request.method} is not allowed for this resource.",
                code="method_not_allowed",
            )
        if handler is None:
            raise HTTPError(404, "No route matches this path.", code="
    ↪ not_found")
        request.path_params = parameters
        response = handler(request)
        if allowed and response.status == 405:
            response.headers.set("Allow", ", ".join(sorted(allowed)))
        return response


__all__ = ["Application", "Handler", "Middleware"]
```

`Application.handle` is the safe public boundary. It rejects unsupported methods, composes middleware, dispatches the route, verifies that handlers return a Response, converts `HTTPError`, and catches unexpected exceptions.

Expected errors carry status, public detail, a stable machine code, optional field errors, and a problem type URI. Unexpected exceptions are logged through the standard `logging` package and become a generic 500 document. The catch is broad only at this outer boundary; inner code should catch only errors it can handle meaningfully.

> **Security boundary**
>
> Never return `str(exception)` for an unexpected error. SQLite exceptions, filesystem errors, and programmer mistakes can expose paths, schema details, or secrets. Give clients a stable generic problem and give operators the traceback through controlled logging.

13.5 Stable errors are part of the API

Clients should branch on status and stable problem type/code, not English prose. The Task API uses a type such as `urn:task-api:problem:validation-error` and includes an `errors` object for individual fields. The prose remains useful to humans and may improve over time without breaking client logic.

The `instance` identifies the request path associated with the occurrence. In a larger system it could be a traceable occurrence URI, but it must not leak private identifiers or internal topology.

13.6 Observability without coupling

The server access log records client address, method, target, status, response bytes, and elapsed time. Application logs record unexpected failures. These are different perspectives. A request ID middleware could connect them, but this book avoids inventing a distributed tracing system.

Do not log complete request bodies by default. Bodies may contain credentials, personal data, or large values. Log structure and identifiers deliberately, with retention and access policy in mind.

Chapter summary

JSON decoding validates media type, encoding, and syntax before domain rules. Expected failures become structured problem responses; unexpected failures stay private. Middleware wraps the terminal dispatcher in a visible order and is the framework's tool for reusable cross-cutting behavior.

Review questions

1. Why is a valid JSON list still invalid for task creation?
2. What changes when two middleware functions exchange order?
3. Why should clients not branch on an English error sentence?
4. Which request data is dangerous to log by default?

Exercise 13.1: Error taxonomy

For twelve failures spanning HTTP syntax, JSON, task fields, missing tasks, SQL, and programmer errors, choose the layer that owns detection, public status, stable problem code, and whether a traceback is logged.

Exercise 13.2: Request ID middleware

Design middleware that accepts a syntactically safe incoming request ID or generates one, places it on the response, and makes it available to logs. Define length and character limits and explain why arbitrary input must not be copied into logs or fields.

CHAPTER 14

Bounded concurrency and connection lifecycle

> Correct parsing is not enough if one silent client can consume every worker. The final server makes capacity, waiting, persistence, overload, and shutdown explicit.

Learning objectives

- Explain the accept loop, finite connection queue, and fixed workers.
- Handle per-connection parsing, timeouts, keep-alive, and maximum requests.
- Define overload behavior instead of permitting unbounded memory growth.
- Stop accepting and release resources during graceful shutdown.

14.1 From serial to concurrent

The early snapshots handle one accepted connection at a time. If a client pauses, new clients wait even when CPU and database capacity are available. Stage 7 adds a fixed set of worker threads and a finite queue:

```
"""Stage 7: add a fixed worker pool and finite connection queue."""

from __future__ import annotations

import json
import queue
import socket
import threading
from http import HTTPStatus


HOST = "127.0.0.1"
PORT = 8080
WORKERS = 4
QUEUE_SIZE = 16
```

```
connections: queue.Queue[tuple[socket.socket, tuple[str, int]] | None] = queue.Queue(QUEUE_SIZE)


def response(status: int, document: dict) -> bytes:
    body = json.dumps(document, separators=(",", ":")).encode()
    return (
        f"HTTP/1.1 {status} {HTTPStatus(status).phrase}\r\n"
        "Content-Type: application/json; charset=utf-8\r\n"
        f"Content-Length: {len(body)}\r\n"
        "Connection: close\r\n\r\n".encode()
        + body
    )


def worker() -> None:
    while True:
        item = connections.get()
        if item is None:
            connections.task_done()
            return
        connection, address = item
        try:
            with connection:
                connection.settimeout(5)
                data = connection.recv(32 * 1024)
                if data.startswith(b"GET /health HTTP/1.1\r\n"):
                    result = response(200, {"status": "ok", "worker": threading.current_thread().name})
                else:
                    result = response(404, {"title": "Not Found", "status": 404})
                connection.sendall(result)
                print(address, len(data), "bytes")
        finally:
            connections.task_done()


def main() -> None:
    for number in range(WORKERS):
        threading.Thread(target=worker, name=f"worker-{number + 1}", daemon=True).start()
    with socket.socket() as listener:
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind((HOST, PORT))
        listener.listen(128)
        print(f"Listening on http://{HOST}:{PORT} with {WORKERS} workers")
        while True:
            connection, address = listener.accept()
            try:
                connections.put_nowait((connection, address))
            except queue.Full:
                with connection:
```

```
65                      connection.sendall(response(503, {"title": "Server Busy",
    ↪ "status": 503}))
66
67
68 if __name__ == "__main__":
69     main()
```

The accept loop does little work: accept, enqueue, or reject. Each worker takes a connection, applies a timeout, processes it, closes it, and marks the queue item complete. Four workers permit four blocking conversations; a queue of sixteen allows a short burst without allocating unlimited threads.

14.2 Bounded is a correctness property

An unbounded queue converts overload into growing memory use and increasing latency. An unbounded thread-per-connection design can exhaust memory, scheduler capacity, descriptors, and database connections. A bounded service reaches a known saturation point and rejects excess work predictably.

The final defaults are sixteen workers and a 64-connection application queue. When the queue is full, the accept loop sends a small 503 problem and closes the connection. The response is best effort: a peer may disappear before reading it.

Capacity values are not universal recommendations. Production sizing requires measurement of workload, latency, memory, downstream capacity, and failure behavior. Here they are visible teaching defaults.

14.3 One parser per connection

Each worker constructs a `RequestParser` when it begins a connection. The parser buffer survives across receive calls and, for keep-alive, across requests. It is never shared between connections. This avoids locks inside the parser and prevents bytes from different clients mixing.

The worker loop asks the parser for a request before receiving more data. This detail matters when the previous receive already contained two requests. It processes buffered work without waiting for network input that may never come.

14.4 Persistent but finite

HTTP/1.1 persistence is enabled by default. The server closes when:

- the client sends `Connection: close`;
- keep-alive is disabled in configuration;
- 100 requests have used the connection;
- parsing fails, input times out, the peer closes, or socket I/O fails;
- server shutdown begins.

Each response states `Connection: keep-alive` or `close`. The five- second socket timeout bounds both a partial request and idle time before the next request. A production server would often use separate header, body, idle, write, and total deadlines.

14.5 Read the final server

```
"""Bounded threaded HTTP server and response serialization."""

from __future__ import annotations

import logging
import queue
import socket
import threading
import time
from dataclasses import dataclass
from email.utils import formatdate
from http import HTTPStatus

from .application import Application
from .models import HTTPError, Request, Response
from .parser import ParserLimits, RequestParser


@dataclass(slots=True)
class ServerConfig:
    host: str = "127.0.0.1"
    port: int = 8080
    workers: int = 16
    queue_size: int = 64
    backlog: int = 128
    read_timeout: float = 5.0
    keep_alive: bool = True
    max_requests_per_connection: int = 100
    request_line_limit: int = 8 * 1024
    header_bytes_limit: int = 32 * 1024
    header_count_limit: int = 100
    body_bytes_limit: int = 1024 * 1024

    def parser_limits(self) -> ParserLimits:
        return ParserLimits(
            request_line=self.request_line_limit,
            header_bytes=self.header_bytes_limit,
            header_count=self.header_count_limit,
            body_bytes=self.body_bytes_limit,
        )


class Server:
    def __init__(self, app: Application, config: ServerConfig | None = None)
    ↪ -> None:
        self.app = app
        self.config = config or ServerConfig()
        self.logger = logging.getLogger("tinyhttp.server")
        self._connections: queue.Queue[
            tuple[socket.socket, tuple[str, int]] | None
        ] = queue.Queue(maxsize=self.config.queue_size)
```

```
        self._stop = threading.Event()
        self._ready = threading.Event()
        self._listener: socket.socket | None = None
        self._workers: list[threading.Thread] = []
        self.address: tuple[str, int] | None = None

    def serve_forever(self) -> None:
        if self._listener is not None:
            raise RuntimeError("Server is already running")
        listener = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        listener.bind((self.config.host, self.config.port))
        listener.listen(self.config.backlog)
        listener.settimeout(0.5)
        self._listener = listener
        bound_host, bound_port = listener.getsockname()[:2]
        self.address = (str(bound_host), int(bound_port))
        self._start_workers()
        self._ready.set()
        self.logger.info("Listening on http://%s:%s", *self.address)
        try:
            while not self._stop.is_set():
                try:
                    connection, address = listener.accept()
                except socket.timeout:
                    continue
                except OSError:
                    if self._stop.is_set():
                        break
                    raise
                try:
                    self._connections.put_nowait((connection, address))
                except queue.Full:
                    self._reject_overload(connection)
        finally:
            try:
                listener.close()
            finally:
                self._listener = None
                self._signal_workers()

    def wait_until_ready(self, timeout: float = 5.0) -> bool:
        return self._ready.wait(timeout)

    def shutdown(self) -> None:
        self._stop.set()
        listener = self._listener
        if listener is not None:
            try:
                listener.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass
            listener.close()
        self._signal_workers()
```

```
        current = threading.current_thread()
        for worker in self._workers:
            if worker is not current:
                worker.join(timeout=self.config.read_timeout + 1.0)

    def _start_workers(self) -> None:
        for number in range(self.config.workers):
            worker = threading.Thread(
                target=self._worker,
                name=f"tinyhttp-worker-{number + 1}",
                daemon=True,
            )
            worker.start()
            self._workers.append(worker)

    def _signal_workers(self) -> None:
        for _ in self._workers:
            try:
                self._connections.put_nowait(None)
            except queue.Full:
                break

    def _worker(self) -> None:
        while not self._stop.is_set():
            try:
                item = self._connections.get(timeout=0.5)
            except queue.Empty:
                continue
            if item is None:
                self._connections.task_done()
                break
            connection, address = item
            try:
                self._handle_connection(connection, address)
            finally:
                self._connections.task_done()

    def _handle_connection(
        self,
        connection: socket.socket,
        address: tuple[str, int],
    ) -> None:
        parser = RequestParser(self.config.parser_limits())
        connection.settimeout(self.config.read_timeout)
        request_count = 0
        with connection:
            while not self._stop.is_set():
                try:
                    request = self._read_request(connection, parser, address)
                except HTTPError as error:
                    response = Response.problem(
                        error.status,
                        error.detail,
                        code=error.code,
```

```
                        title=error.title,
                        errors=error.errors,
                        type_uri=error.type_uri,
                    )
                    self._send_response(connection, None, response,
    ↪ keep_alive=False)
                    return
                except socket.timeout:
                    if parser.has_buffered_data:
                        response = Response.problem(
                            408,
                            "The request was not completed before the timeout.
    ↪ ",
                            code="request_timeout",
                        )
                        self._send_response(connection, None, response,
    ↪ keep_alive=False)
                    return
                except OSError:
                    return
                if request is None:
                    return
                request_count += 1
                started = time.monotonic()
                response = self.app.handle(request)
                keep_alive = self._should_keep_alive(request, request_count)
                try:
                    sent = self._send_response(
                        connection,
                        request,
                        response,
                        keep_alive=keep_alive,
                    )
                except OSError:
                    return
                elapsed_ms = (time.monotonic() - started) * 1000
                self.logger.info(
                    '%s "%s %s" %s %s %.1fms',
                    address[0],
                    request.method,
                    request.target,
                    response.status,
                    sent,
                    elapsed_ms,
                )
                if not keep_alive:
                    return

    @staticmethod
    def _read_request(
        connection: socket.socket,
        parser: RequestParser,
        address: tuple[str, int],
    ) -> Request | None:
```

```
        while True:
            request = parser.pop_request(address)
            if request is not None:
                return request
            chunk = connection.recv(16 * 1024)
            if not chunk:
                if parser.has_buffered_data:
                    raise HTTPError(
                        400,
                        "The connection closed during an incomplete request.",
                        code="incomplete_request",
                    )
                return None
            parser.feed(chunk)

    def _should_keep_alive(self, request: Request, request_count: int) -> bool:
        connection = (request.headers.get("Connection") or "").casefold()
        return (
            self.config.keep_alive
            and connection != "close"
            and request_count < self.config.max_requests_per_connection
            and not self._stop.is_set()
        )

    @staticmethod
    def _send_response(
        connection: socket.socket,
        request: Request | None,
        response: Response,
        *,
        keep_alive: bool,
    ) -> int:
        try:
            reason = HTTPStatus(response.status).phrase
        except ValueError:
            reason = "Unknown"
        headers = response.headers.copy()
        headers.set("Date", formatdate(usegmt=True))
        headers.set("Server", "tinyhttp/1.0")
        headers.set("Connection", "keep-alive" if keep_alive else "close")
        body = response.body
        no_content = 100 <= response.status < 200 or response.status in {204, 304}
        if no_content:
            body_to_send = b""
            headers.remove("Content-Length")
        else:
            headers.set("Content-Length", str(len(body)))
            body_to_send = b"" if request and request.method == "HEAD" else body
        start = f"HTTP/1.1 {response.status} {reason}\r\n".encode("ascii")
        encoded_headers = b"".join(
```

```
260             f"{name}: {value}\r\n".encode("iso-8859-1") for name, value in
    ↪ headers
261         )
262         connection.sendall(start + encoded_headers + b"\r\n" + body_to_send)
263         return len(body_to_send)
264
265     @staticmethod
266     def _reject_overload(connection: socket.socket) -> None:
267         with connection:
268             response = Response.problem(
269                 503,
270                 "The bounded connection queue is full. Try again later.",
271                 code="server_overloaded",
272             )
273             try:
274                 Server._send_response(connection, None, response, keep_alive=
    ↪ False)
275             except OSError:
276                 pass
```

14.5.1 Readiness and ephemeral ports

When configured with port zero, the operating system selects a free port. The server records its bound address and sets a readiness event after listening and starting workers. Integration tests can start a server thread, wait deterministically, and connect to the chosen port without guessing.

14.5.2 Timeout and protocol errors

An `HTTPError` becomes a problem response and closes the connection so no ambiguous bytes survive. A timeout with buffered data can receive 408; an idle connection simply closes. Socket errors are normal peer-failure outcomes and do not crash the accept loop.

14.5.3 Shutdown

`shutdown` sets a stop event, closes the listener to wake accept, places sentinels for workers, and joins them within a bound. Workers also observe the event in their loops. This is modest graceful behavior: it stops new acceptance and lets current operations finish or time out. It does not provide a configurable long drain period or connection migration.

14.6 Threads and SQLite

Python threads are appropriate for this teaching server because socket and SQLite waits release execution opportunities and blocking control flow remains readable. They do not make CPU-bound Python code scale linearly across cores.

The Task repository opens a short-lived SQLite connection for each operation. Connections are not shared across workers. SQLite serializes writes as needed, and a busy timeout gives brief contention time to resolve. WAL mode improves reader/writer coexistence, but SQLite remains one embedded database with finite write capacity.

14.7 Alternatives

An event loop can manage many sockets with fewer threads by running non-blocking operations and resuming tasks when ready. ASGI frameworks use asynchronous interfaces for such workloads. Process pools or multiple server processes can use more CPU cores and isolate failures. Mature servers combine strategies.

The important lesson is not “threads are best.” It is that every concurrency model has ownership, scheduling, cancellation, backpressure, and shutdown rules. Our bounded thread model makes those rules small enough to inspect.

Security boundary

Timeouts and queues reduce resource exhaustion but do not make this server safe for hostile Internet traffic. Production parsers, TLS stacks, operating-system tuning, rate limiting, monitoring, patch processes, and defense in depth remain necessary.

Chapter summary

The server accepts into a finite queue and uses a fixed worker set. Each connection owns its parser and is constrained by timeout and request count. Overload receives 503, protocol errors close ambiguous streams, and shutdown releases listener and workers. Bounds turn failure from accidental exhaustion into explicit behavior.

Review questions

1. Why is a fixed worker count insufficient without a bounded queue?
2. Why must a parser buffer be connection-local?
3. Which conditions end keep-alive in this implementation?
4. What work remains for a production-grade graceful shutdown?

Exercise 14.1: Capacity timeline

With two workers and a queue of three, draw the state after six clients connect and each worker blocks on a partial header. Identify which clients are active, queued, or rejected. Repeat after one worker times out.

Exercise 14.2: Concurrency test

Write an integration test that starts the server on port zero and sends 50 concurrent GET requests. Collect every status with a deadline. The assertion should allow success and intentional overload, but no hangs or malformed responses.

CHAPTER 15

The complete tinyhttp framework

The pieces now form a small framework: parser, public message types, router, middleware application, and bounded server. This chapter follows one request across every interface.

Learning objectives

- Use the public framework API without depending on implementation internals.
- Trace data from socket bytes to handler and back to response bytes.
- Recognize invariants guaranteed at each boundary.
- Run the package tests and extend it without widening its protocol claims silently.

15.1 Package boundaries

The final `tinyhttp` package contains five focused modules:

Module	Owns	Does not own
models	Headers, Request, Response, HTTPError	socket I/O or routing
parser	HTTP/1.1 request syntax, framing, target parsing	JSON domain rules
router	method/path patterns and parameters	authorization or database lookup
application	middleware, dispatch, safe error boundary	TCP lifecycle
server	listener, workers, timeouts, keep-alive, serialization	task semantics

`tinyhttp.__init__` exports only the teaching API. Application code imports `Application`, `Request`, `Response`, `HTTPError`, `Server`, and `ServerConfig`; it need not import parser details.

15.2 A minimal application

The public shape is intentionally familiar but transparent:

```python
from tinyhttp import Application, Request, Response, Server

app = Application()

@app.route("GET", "/hello/{name}")
def hello(request: Request) -> Response:
    return Response.json(
        {"message": f"Hello, {request.path_params['name']}!"}
    )

Server(app).serve_forever()
```

The decorator registers a handler. The handler accepts one Request and returns one Response. The server depends on the application's `handle` method. There is no global hidden request object and no automatic database injection.

15.3 Trace one request

For **GET** `/hello/Ada?formal=yes`:

1. The listener accepts a connected socket and a worker takes it from the queue.
2. A connection-local parser receives bytes until one complete message exists.
3. The parser validates HTTP/1.1, Host, fields, and framing; it decodes the path and creates query lists.
4. `Application.handle` composes middleware and calls its dispatcher.
5. The router matches `/hello/{name}` and sets `path_params["name"]` to `"Ada"`.
6. The handler creates a UTF-8 JSON Response.
7. Middleware unwinds and can inspect the response.
8. The server adds wire fields, computes length, serializes, and calls `sendall`.
9. If policy permits, the parser and socket remain for the next request.

Each transition narrows the type of uncertainty. Socket code sees bytes; parser code produces an HTTP request; router code produces parameters; the handler produces a response value; the server produces bytes again.

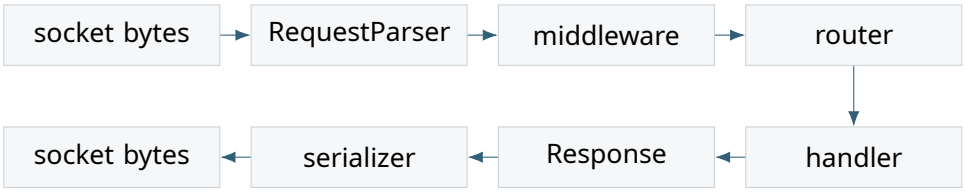


Figure 15.1: The framework data flow. The return path is represented on the lower row.

15.4 Configuration is policy

`ServerConfig` centralizes address, port, workers, queue size, listen backlog, read timeout, keep-alive, request count, and parser limits. Tests replace the port with zero and use fewer workers. An application can disable keep-alive without rewriting server logic.

Configuration values should not be changed independently without thought. A larger body limit increases memory pressure. More workers increase simultaneous database contention. A longer timeout multiplies the duration for which slow clients hold capacity.

15.5 Run the verification suite

From `code/final`:

```
python -m unittest discover -s tests -v
```

The suite tests models and parser, application/router/middleware, real loopback socket exchange, and the Task API. Integration tests bind an ephemeral port and stop the server in teardown. A sandbox that forbids local sockets must grant that specific capability; skipping every network test would leave the central artifact unverified.

15.6 How to extend honestly

Before adding a feature, update the contract:

1. name the semantics and relevant standard rules;
2. define accepted and rejected inputs;
3. identify which module owns the feature;
4. add adversarial tests before widening acceptance;
5. update documentation and limits;
6. preserve behavior for existing clients or version the interface.

Adding chunked bodies, for example, is not just decoding hexadecimal sizes. It affects framing precedence, decoded limits, trailers, connection buffering, tests, and intermediary compatibility. Honest software names the whole change.

Checkpoint

You now have a reusable educational framework. Before beginning the backend, you should be able to open any module and state its inputs, outputs, invariants, resource ownership, and failure modes. The next part uses the framework rather than changing HTTP rules casually.

Chapter summary

`tinyhttp` is small because its boundaries are narrow, not because it hides work. Five modules transform connection bytes into requests, route through middleware and handlers, and serialize responses under explicit limits. The public API is enough to build the Task backend while implementation invariants remain inspectable.

Review questions

1. Which module owns each request-to-response transition?
2. What invariants may a handler assume about its Request?

3. Why is ServerConfig more than cosmetic settings?
4. What must change before claiming support for chunked requests?

Exercise 15.1: Add a route safely

Add `GET /echo/{value}` to a small application, returning JSON with the path value and repeated `tag` query values. Test Unicode percent encoding, blank query values, HEAD fallback, OPTIONS, and a wrong method.

Exercise 15.2: Boundary review

Choose one class from each module and write its contract as preconditions, postconditions, invariants, and possible errors. Compare those contracts with the actual tests and add one missing case.

Part III

From server to backend REST API

CHAPTER 16

Designing the Task REST API

Before writing SQL or handlers, define resources, representations, method semantics, validation, and errors. A written contract prevents implementation accidents from becoming the API.

Learning objectives

- Model a task resource and its collection representation.
- Define CRUD behavior with precise method and status semantics.
- Distinguish full replacement, partial modification, filtering, and pagination.
- Specify stable success and problem documents before implementation.

16.1 Resource model

The service exposes a task collection at `/tasks` and individual tasks at `/tasks/{id}`. A task representation contains:

Field	Type	Rule
id	integer	Server-generated positive identifier
title	string	Required, trimmed, 1–200 characters
description	string	Optional, at most 2000 characters; default empty
status	enum	`todo`, `in_progress`, or `done`
priority	enum	`low`, `medium`, or `high`
due_date	date or null	Exact `YYYY-MM-DD` form
created_at	timestamp	Server-generated UTC RFC 3339 text
updated_at	timestamp	Server-generated UTC RFC 3339 text

Clients may write only title, description, status, priority, and due date. Unknown fields are rejected rather than ignored, catching misspellings and preventing clients from believing a value was stored when it was not.

16.2 Collection operations

POST /tasks accepts a JSON object, applies defaults, creates a resource, and returns 201 Created. The body is the complete created representation and `Location` identifies `/tasks/{id}`. The operation is not idempotent: repeating it creates another task.

GET /tasks returns:

```
{
  "items": [ ... task representations ... ],
  "total": 37,
  "limit": 20,
  "offset": 0
}
```

Supported filters are `status`, `priority`, `due_before`, and text query `q`. `limit` defaults to 20 and is between 1 and 100. `offset` defaults to zero and is non-negative. Every supported parameter may appear once; unknown or repeated parameters receive a validation problem.

Offset pagination is simple and suitable for a teaching database. Under concurrent inserts it can repeat or skip items between pages. Cursor pagination is often preferable for large, changing datasets but requires a stable ordering key and a documented cursor format.

16.3 Individual operations

GET /tasks/{id} returns 200 with a task or 404 with a stable `task-not-found` problem.

PUT /tasks/{id} is full replacement. Title is required. Omitted optional fields return to defaults. Server-owned identity and creation timestamp remain unchanged; update timestamp changes. Repeating the same PUT has the same intended resource effect, although the timestamp changes.

PATCH /tasks/{id} updates only supplied writable fields. An empty object is rejected. PATCH is not assumed idempotent in general; this particular merge-style field assignment is idempotent for the same values.

DELETE /tasks/{id} deletes an existing task and returns 204 with no content. Deleting an absent task returns 404. The intended final resource state still makes DELETE idempotent.

16.4 Validation sequence

Validation proceeds from transport toward domain:

1. HTTP parser validates message framing and limits.
2. A body-required endpoint requires `Content-Length` and nonempty bytes.
3. Request JSON helper requires the media type, UTF-8, and JSON grammar.
4. Task validation requires an object, writable names, types, lengths, enum membership, and exact date format.
5. The repository uses database constraints as a final durable backstop.

Validation errors use 422 and an `errors` object. Malformed JSON remains 400 because no valid JSON value exists to validate. Unsupported media type is 415. These distinctions help a client decide what to fix.

On the wire

```
POST /tasks HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json
Content-Length: 47

{"title":"Learn routing","priority":"high"}
```

The server supplies `status=todo`, an empty description, null due date, timestamps, and an identifier.

16.5 Error contract

Every application error is `application/problem+json`. A task validation problem uses a stable type URI. A client can render the title/detail for a person and map `errors` to form fields. It should use `status` and `type` for program logic.

Not every failure is safe to expose. A database exception becomes a generic 500 problem; the traceback is logged. A missing task is ordinary control flow and does not produce an error log.

16.6 What the API deliberately omits

There are no users, projects, labels, attachments, authentication, authorization, soft deletion, synchronization, ETags, idempotency keys, or schema negotiation. Each could be valuable, but each introduces new resource and security decisions. The capstone stays focused on HTTP, framework flow, validation, SQL, and tests.

Chapter summary

The contract defines tasks, writable fields, collection and individual methods, filter/pagination rules, validation layers, and structured failures. PUT replaces, PATCH partially updates, POST creates, and DELETE removes. Implementation can now be judged against a stable design.

Review questions

1. Why does POST return both a representation and Location?
2. What happens to omitted optional fields in PUT versus PATCH?
3. Why can offset pagination become inconsistent under concurrent inserts?
4. Which layer distinguishes malformed JSON from invalid task fields?

Exercise 16.1: Contract examples

Write one complete request and response example for every endpoint, including headers and status. Add examples for every error class. These become documentation fixtures and integration-test candidates.

Exercise 16.2: Extend without coding

Design an optional `completed_at` field. Define who writes it, how status transitions affect it, PUT/PATCH behavior, database constraints, filtering, and backward compatibility. Notice how one field crosses every layer.

CHAPTER 17

Persistence with SQLite

> The API needs durable state. SQLite gives the project transactions, constraints, indexes, and SQL without introducing a separate database server or an object-relational mapper.

Learning objectives

- Translate the Task contract into a constrained relational schema.
- Use parameterized statements and explicit transaction boundaries.
- Separate database setup and repository operations from HTTP handlers.
- Explain per-operation connections, WAL, busy timeout, and migration versioning.

17.1 Why SQLite fits the capstone

SQLite stores a relational database in a local file and runs in the application process. Python includes a DB-API interface. The intern can inspect real SQL, constraints, indexes, transactions, and concurrency behavior without installing or administering a database server.

This convenience has a boundary. SQLite has one file-oriented write path and is not a drop-in architecture for every distributed, high-write service. Its strength here is visibility and low operational overhead.

17.2 Schema as durable validation

```sql
CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),
    description TEXT NOT NULL DEFAULT '' CHECK (length(description) ⩽ 2000),
    status TEXT NOT NULL DEFAULT 'todo'
        CHECK (status IN ('todo', 'in_progress', 'done')),
    priority TEXT NOT NULL DEFAULT 'medium'
        CHECK (priority IN ('low', 'medium', 'high')),
    due_date TEXT,
    created_at TEXT NOT NULL,
```

```
    updated_at TEXT NOT NULL
);

CREATE INDEX IF NOT EXISTS tasks_status_idx ON tasks(status);
CREATE INDEX IF NOT EXISTS tasks_priority_idx ON tasks(priority);
CREATE INDEX IF NOT EXISTS tasks_due_date_idx ON tasks(due_date);

PRAGMA user_version = 1;
```

The primary key assigns identity. NOT NULL and CHECK constraints protect required values even if a future handler forgets validation. Enum checks mirror the API allow-lists. Length checks provide a final bound. Indexes support filters by status, priority, and due date.

The schema stores dates and UTC timestamps as consistently formatted text. Lexicographic ordering works for the chosen ISO forms. SQLite does not enforce a native date type here, so application validation remains necessary.

`PRAGMA user_version = 1` records schema version. Startup accepts a new version-zero database and initializes it, accepts version one, and refuses an unknown version. A real migration system would apply ordered, reversible or carefully forward-only changes and back up data as policy requires.

17.3 Parameterized SQL

Values never enter SQL through string interpolation. Statements contain `?` placeholders and pass values separately. The driver encodes each value as data rather than executable SQL syntax.

Dynamic query structure still requires care. The list operation builds WHERE fragments only from code-owned strings and puts every client value in parameters. The update operation chooses column names from a fixed allow-list. Parameter binding cannot substitute table or column identifiers, so those must never come directly from a request.

> **Security boundary**
>
> Escaping quotes manually is not a substitute for parameterization. It is easy to miss data types, encodings, wildcard semantics, or a future code path. Keep SQL structure controlled by code and values controlled by parameters.

17.4 Connections and transactions

`Database.connect` creates a connection, configures rows by name, enables foreign-key enforcement for future schemas, and sets a five-second busy timeout. Each repository operation opens a connection in a context manager. Successful write blocks commit; exceptions roll back and close.

Connections are not shared between worker threads. This avoids accidental cross-thread transaction state. The overhead is acceptable for a local teaching service and the ownership is obvious.

WAL mode lets readers coexist better with a writer by appending changes to a write-ahead log before checkpointing. It does not permit unlimited simultaneous writes. The busy timeout allows short contention to resolve instead of failing immediately.

17.5 The database and repository code

```
"""SQLite setup and a deliberately explicit repository layer."""

from __future__ import annotations

import sqlite3
from collections.abc import Mapping
from pathlib import Path
from typing import Any


SCHEMA_VERSION = 1


class Database:
    def __init__(self, path: str | Path) -> None:
        self.path = str(path)

    def connect(self) -> sqlite3.Connection:
        connection = sqlite3.connect(self.path, timeout=5.0)
        connection.row_factory = sqlite3.Row
        connection.execute("PRAGMA foreign_keys = ON")
        connection.execute("PRAGMA busy_timeout = 5000")
        return connection

    def initialize(self) -> None:
        schema = Path(__file__).with_name("schema.sql").read_text(encoding="
    ↪ utf-8")
        with self.connect() as connection:
            version = int(connection.execute("PRAGMA user_version").fetchone()
    ↪ [0])
            if version not in {0, SCHEMA_VERSION}:
                raise RuntimeError(
                    f"Database schema version {version} is not supported; "
                    f"expected {SCHEMA_VERSION}."
                )
            connection.execute("PRAGMA journal_mode = WAL")
            if version == 0:
                connection.executescript(schema)


class TaskRepository:
    def __init__(self, database: Database) -> None:
        self.database = database

    def create(self, values: Mapping[str, Any]) -> dict[str, Any]:
        columns = (
            "title",
            "description",
            "status",
            "priority",
            "due_date",
```

```
            "created_at",
            "updated_at",
        )
        with self.database.connect() as connection:
            cursor = connection.execute(
                """
                INSERT INTO tasks
                    (title, description, status, priority, due_date,
                     created_at, updated_at)
                VALUES (?, ?, ?, ?, ?, ?, ?)
                """,
                tuple(values[column] for column in columns),
            )
            task_id = int(cursor.lastrowid)
            row = connection.execute(
                "SELECT * FROM tasks WHERE id = ?", (task_id,)
            ).fetchone()
        return self._task(row)

    def get(self, task_id: int) -> dict[str, Any] | None:
        with self.database.connect() as connection:
            row = connection.execute(
                "SELECT * FROM tasks WHERE id = ?", (task_id,)
            ).fetchone()
        return self._task(row) if row else None

    def list(
        self,
        *,
        status: str | None = None,
        priority: str | None = None,
        due_before: str | None = None,
        search: str | None = None,
        limit: int = 20,
        offset: int = 0,
    ) -> tuple[list[dict[str, Any]], int]:
        conditions: list[str] = []
        parameters: list[Any] = []
        if status:
            conditions.append("status = ?")
            parameters.append(status)
        if priority:
            conditions.append("priority = ?")
            parameters.append(priority)
        if due_before:
            conditions.append("due_date IS NOT NULL AND due_date <= ?")
            parameters.append(due_before)
        if search:
            conditions.append("(title LIKE ? ESCAPE '\\' OR description LIKE
    ↪ ? ESCAPE '\\')")
            escaped = search.replace("\\", "\\\\").replace("%", "\\%").
    ↪ replace("_", "\\_")
            needle = f"%{escaped}%"
            parameters.extend((needle, needle))
```

```
        where = " WHERE " + " AND ".join(conditions) if conditions else ""
        with self.database.connect() as connection:
            total = int(
                connection.execute(
                    "SELECT count(*) FROM tasks" + where,
                    parameters,
                ).fetchone()[0]
            )
            rows = connection.execute(
                "SELECT * FROM tasks"
                + where
                + " ORDER BY created_at DESC, id DESC LIMIT ? OFFSET ?",
                [*parameters, limit, offset],
            ).fetchall()
        return [self._task(row) for row in rows], total

    def replace(self, task_id: int, values: Mapping[str, Any]) -> dict[str, Any] | None:
        with self.database.connect() as connection:
            cursor = connection.execute(
                """
                UPDATE tasks
                SET title = ?, description = ?, status = ?, priority = ?,
                    due_date = ?, updated_at = ?
                WHERE id = ?
                """,
                (
                    values["title"],
                    values["description"],
                    values["status"],
                    values["priority"],
                    values["due_date"],
                    values["updated_at"],
                    task_id,
                ),
            )
            if cursor.rowcount == 0:
                return None
            row = connection.execute(
                "SELECT * FROM tasks WHERE id = ?", (task_id,)
            ).fetchone()
        return self._task(row)

    def update(self, task_id: int, values: Mapping[str, Any]) -> dict[str, Any] | None:
        allowed = {"title", "description", "status", "priority", "due_date", "updated_at"}
        selected = {name: value for name, value in values.items() if name in allowed}
        assignments = ", ".join(f"{name} = ?" for name in selected)
        parameters = [*selected.values(), task_id]
        with self.database.connect() as connection:
            cursor = connection.execute(
                f"UPDATE tasks SET {assignments} WHERE id = ?",
```

```
                parameters,
            )
            if cursor.rowcount == 0:
                return None
            row = connection.execute(
                "SELECT * FROM tasks WHERE id = ?", (task_id,)
            ).fetchone()
        return self._task(row)

    def delete(self, task_id: int) -> bool:
        with self.database.connect() as connection:
            cursor = connection.execute("DELETE FROM tasks WHERE id = ?", (
    ↪ task_id,))
        return cursor.rowcount > 0

    @staticmethod
    def _task(row: sqlite3.Row) -> dict[str, Any]:
        return {
            "id": int(row["id"]),
            "title": str(row["title"]),
            "description": str(row["description"]),
            "status": str(row["status"]),
            "priority": str(row["priority"]),
            "due_date": row["due_date"],
            "created_at": str(row["created_at"]),
            "updated_at": str(row["updated_at"]),
        }
```

The repository returns plain dictionaries matching the API representation. It does not import HTTP types. A missing row becomes `None` or `False`; the handler decides that this means a 404 response.

Create uses one INSERT then reads the committed representation. Get and delete are direct identifier operations. Replace names every writable column. Partial update builds assignments only from validated writable keys and always adds `updated_at`.

17.6 Filtering, searching, and pagination

The list method accumulates code-owned predicates and parameters. Search treats `%` and `_` as literal characters by escaping their LIKE wildcard meaning. It issues one count query and one ordered page query with the same conditions.

Ordering uses `created_at DESC, id DESC`. The identifier breaks timestamp ties, producing deterministic pages for one database state. The count and page are separate statements, so a concurrent write can occur between them. A stricter snapshot would wrap both in one read transaction. This implementation accepts the small inconsistency and documents it.

17.7 Repository tests use temporary files

An in-memory SQLite database is normally private to one connection. Because the repository intentionally opens short-lived connections, tests use a temporary database file in-

stead of `:memory:`. Each test case initializes a fresh file and deletes its temporary directory afterward. This mirrors connection behavior without sharing state between tests.

Pitfall

Do not put a production-like SQLite file inside a directory that tests delete, and do not point tests at the developer's real `tasks.db`. Test isolation is a data-safety property.

Chapter summary

SQLite supplies durable constraints and transactions behind a repository that does not know HTTP. Parameterized SQL separates code-owned structure from client values. Per-operation connections simplify thread ownership; WAL and busy timeout improve modest concurrency. Schema versioning makes compatibility explicit.

Review questions

1. Which task rules are enforced both in Python and in SQL, and why?
2. Why can parameters replace values but not arbitrary column names?
3. Why do tests use a temporary file instead of a simple in-memory database?
4. What inconsistency can occur between list count and page queries?

Exercise 17.1: Constraint failure

Bypass HTTP validation and call the repository with an invalid status. Observe the database constraint failure and transaction rollback. Explain why the HTTP layer should still validate first and return a controlled 422.

Exercise 17.2: Query plan

Populate tasks with several statuses and due dates. Use SQLite's query-plan inspection to see which filters can use indexes. Compare an exact status filter with a leading-wildcard text search and explain the difference.

CHAPTER 18

Implementing the Task API

The framework and repository meet in a thin application layer. Handlers translate HTTP inputs into validated repository calls and translate results into resource representations.

Learning objectives

- Build CRUD handlers without leaking HTTP into the repository.
- Validate create, replace, patch, path IDs, and list queries consistently.
- Run the API and exercise it with a standard-library client.
- Trace one create and one update through every layer.

18.1 Application construction

`build_app(database_path)` initializes the schema, creates a repository, creates an Application, registers handlers, and returns it. The function makes tests easy: every test supplies a temporary path and receives a fully configured application without starting a socket.

The complete application layer follows. Read it once from top to bottom, then return to the route-by-route discussion.

```
"""HTTP resource handlers and validation for the Task API."""

from __future__ import annotations

from datetime import date, datetime, timezone
from pathlib import Path
from typing import Any, Mapping

from tinyhttp import Application, HTTPError, Request, Response

from .database import Database, TaskRepository


STATUSES = {"todo", "in_progress", "done"}
PRIORITIES = {"low", "medium", "high"}
```

```
WRITABLE_FIELDS = {"title", "description", "status", "priority", "due_date"}


def utc_now() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="seconds").replace("
    ↪ +00:00", "Z")


def build_app(database_path: str | Path) -> Application:
    database = Database(database_path)
    database.initialize()
    repository = TaskRepository(database)
    app = Application()

    @app.route("GET", "/health")
    def health(request: Request) -> Response:
        return Response.json({"status": "ok"})

    @app.route("POST", "/tasks")
    def create_task(request: Request) -> Response:
        require_body(request)
        values = validate_task_document(request.json(), partial=False)
        now = utc_now()
        values.update(created_at=now, updated_at=now)
        task = repository.create(values)
        response = Response.json(task, status=201)
        response.headers.set("Location", f"/tasks/{task['id']}")
        return response

    @app.route("GET", "/tasks")
    def list_tasks(request: Request) -> Response:
        options = validate_list_query(request.query)
        items, total = repository.list(**options)
        return Response.json(
            {
                "items": items,
                "total": total,
                "limit": options["limit"],
                "offset": options["offset"],
            }
        )

    @app.route("GET", "/tasks/{task_id}")
    def get_task(request: Request) -> Response:
        task_id = path_id(request)
        task = repository.get(task_id)
        if task is None:
            raise not_found(task_id)
        return Response.json(task)

    @app.route("PUT", "/tasks/{task_id}")
    def replace_task(request: Request) -> Response:
        task_id = path_id(request)
        require_body(request)
```

```
        values = validate_task_document(request.json(), partial=False)
        values["updated_at"] = utc_now()
        task = repository.replace(task_id, values)
        if task is None:
            raise not_found(task_id)
        return Response.json(task)

    @app.route("PATCH", "/tasks/{task_id}")
    def update_task(request: Request) -> Response:
        task_id = path_id(request)
        require_body(request)
        values = validate_task_document(request.json(), partial=True)
        values["updated_at"] = utc_now()
        task = repository.update(task_id, values)
        if task is None:
            raise not_found(task_id)
        return Response.json(task)

    @app.route("DELETE", "/tasks/{task_id}")
    def delete_task(request: Request) -> Response:
        task_id = path_id(request)
        if not repository.delete(task_id):
            raise not_found(task_id)
        return Response(status=204)

    return app


def require_body(request: Request) -> None:
    if request.headers.get("Content-Length") is None:
        raise HTTPError(
            411,
            "This endpoint requires a Content-Length header.",
            code="length_required",
        )
    if not request.body:
        raise HTTPError(400, "The request body cannot be empty.", code="
    ↪ empty_body")


def path_id(request: Request) -> int:
    raw = request.path_params["task_id"]
    try:
        task_id = int(raw, 10)
    except ValueError as error:
        raise HTTPError(
            404,
            "Task identifiers are positive integers.",
            code="task_not_found",
        ) from error
    if task_id < 1:
        raise HTTPError(404, "Task not found.", code="task_not_found")
    return task_id

```

```
122
123 def not_found(task_id: int) -> HTTPError:
124     return HTTPError(
125         404,
126         f"Task {task_id} does not exist.",
127         code="task_not_found",
128         type_uri="urn:task-api:problem:task-not-found",
129     )
130
131
132 def validate_task_document(value: Any, *, partial: bool) -> dict[str, Any]:
133     if not isinstance(value, dict):
134         raise validation_error({"body": "Expected a JSON object."})
135     unknown = set(value) - WRITABLE_FIELDS
136     errors: dict[str, str] = {}
137     if unknown:
138         errors["body"] = "Unknown fields: " + ", ".join(sorted(unknown))
139     if partial and not value:
140         errors["body"] = "PATCH requires at least one writable field."
141     if not partial and "title" not in value:
142         errors["title"] = "This field is required."
143
144     result: dict[str, Any] = {}
145     if "title" in value:
146         title = value["title"]
147         if not isinstance(title, str) or not title.strip():
148             errors["title"] = "Must be a non-empty string."
149         elif len(title.strip()) > 200:
150             errors["title"] = "Must contain at most 200 characters."
151         else:
152             result["title"] = title.strip()
153     elif not partial:
154         result["title"] = ""
155
156     if "description" in value:
157         description = value["description"]
158         if not isinstance(description, str):
159             errors["description"] = "Must be a string."
160         elif len(description) > 2000:
161             errors["description"] = "Must contain at most 2000 characters."
162         else:
163             result["description"] = description
164     elif not partial:
165         result["description"] = ""
166
167     validate_choice(value, result, errors, "status", STATUSES, "todo",
    ↪ partial)
168     validate_choice(value, result, errors, "priority", PRIORITIES, "medium",
    ↪ partial)
169
170     if "due_date" in value:
171         due_date = value["due_date"]
172         if due_date is not None:
173             if not isinstance(due_date, str):
```

```
                errors["due_date"] = "Must be an ISO date or null."
            else:
                try:
                    parsed_date = date.fromisoformat(due_date)
                except ValueError:
                    errors["due_date"] = "Use the YYYY-MM-DD format."
                else:
                    if parsed_date.isoformat() != due_date:
                        errors["due_date"] = "Use the YYYY-MM-DD format."
                    else:
                        result["due_date"] = due_date
        else:
            result["due_date"] = None
    elif not partial:
        result["due_date"] = None

    if errors:
        raise validation_error(errors)
    return result


def validate_choice(
    source: Mapping[str, Any],
    target: dict[str, Any],
    errors: dict[str, str],
    name: str,
    choices: set[str],
    default: str,
    partial: bool,
) -> None:
    if name in source:
        value = source[name]
        if not isinstance(value, str) or value not in choices:
            errors[name] = "Must be one of: " + ", ".join(sorted(choices))
        else:
            target[name] = value
    elif not partial:
        target[name] = default


def validate_list_query(query: dict[str, list[str]]) -> dict[str, Any]:
    supported = {"status", "priority", "due_before", "q", "limit", "offset"}
    unknown = set(query) - supported
    errors: dict[str, str] = {}
    if unknown:
        errors["query"] = "Unknown parameters: " + ", ".join(sorted(unknown))
    values: dict[str, str | None] = {}
    for name in supported:
        candidates = query.get(name, [])
        if len(candidates) > 1:
            errors[name] = "May be provided only once."
        values[name] = candidates[0] if candidates else None
    if values["status"] and values["status"] not in STATUSES:
        errors["status"] = "Must be one of: " + ", ".join(sorted(STATUSES))
```

```
    if values["priority"] and values["priority"] not in PRIORITIES:
        errors["priority"] = "Must be one of: " + ", ".join(sorted(PRIORITIES)
    ↪ )
    if values["due_before"]:
        try:
            parsed_due_before = date.fromisoformat(str(values["due_before"]))
        except ValueError:
            errors["due_before"] = "Use the YYYY-MM-DD format."
        else:
            if parsed_due_before.isoformat() != values["due_before"]:
                errors["due_before"] = "Use the YYYY-MM-DD format."
    limit = parse_query_integer(values["limit"], "limit", 20, minimum=1,
    ↪ maximum=100, errors=errors)
    offset = parse_query_integer(values["offset"], "offset", 0, minimum=0,
    ↪ maximum=1_000_000, errors=errors)
    search = values["q"]
    if search is not None and len(search) > 200:
        errors["q"] = "Must contain at most 200 characters."
    if errors:
        raise validation_error(errors)
    return {
        "status": values["status"],
        "priority": values["priority"],
        "due_before": values["due_before"],
        "search": search,
        "limit": limit,
        "offset": offset,
    }


def parse_query_integer(
    value: str | None,
    name: str,
    default: int,
    *,
    minimum: int,
    maximum: int,
    errors: dict[str, str],
) -> int:
    if value is None:
        return default
    try:
        result = int(value, 10)
    except ValueError:
        errors[name] = "Must be an integer."
        return default
    if not minimum ⩽ result ⩽ maximum:
        errors[name] = f"Must be between {minimum} and {maximum}."
    return result


def validation_error(errors: Mapping[str, str]) -> HTTPError:
    return HTTPError(
        422,
```

```
        "One or more values failed validation.",
        code="validation_error",
        title="Validation failed",
        errors=errors,
        type_uri="urn:task-api:problem:validation-error",
    )
```

18.2 Create flow

The POST handler requires a declared nonempty body, decodes JSON, validates a full task document, supplies timestamps, and calls `repository.create`. It returns the created dictionary as JSON with 201 and Location.

The server-generated fields never come from the request. This prevents a client from choosing another task's identity or falsifying creation history. Unknown input names fail validation, so a client typo such as `prioriy` is visible.

18.3 Read and list flow

The individual GET converts the captured ID. Invalid or nonpositive IDs receive the same task-not-found category as absent records, keeping the public resource contract simple.

The collection GET validates every query name, repetition count, enum, date, integer range, and search length before repository access. The list result combines items with total and effective pagination values. A caller can construct the next offset without inferring defaults.

18.4 PUT and PATCH

Both update handlers validate content before touching storage and set a new UTC update timestamp. PUT calls full-document validation, which requires title and supplies defaults. PATCH calls partial validation, which rejects empty input and includes only provided fields.

The repository return value distinguishes missing ID. A validation error is 422; a valid modification targeting an absent task is 404. These outcomes must not be collapsed into a generic 400.

18.5 Date and timestamp choices

Due date is a calendar date with no timezone. Validation parses it and requires that serializing the parsed date produces exactly the original text, excluding alternate compact forms accepted by some library versions.

Audit timestamps represent instants and use UTC with a trailing Z, to second precision. A production system might use database-generated timestamps, higher precision, or a test-injectable clock. Here the helper keeps one format throughout the application.

18.6 Run the API

From `code/final`:

```
python -m task_api --database tasks.db
```

The command binds to `127.0.0.1:8080`, initializes the database, starts 16 workers, and enables bounded keep-alive. Optional arguments select host, port, database path, worker count, and keep-alive behavior.

In a second terminal:

```
python client_example.py
```

The client uses `http.client`, creates a task, reads and prints the response, then reuses the connection to list matching tasks. Because it is standard-library code, the core demonstration does not depend on curl or a third-party SDK.

```
"""Create and list tasks with the standard-library HTTP client."""

from __future__ import annotations

import http.client
import json


connection = http.client.HTTPConnection("127.0.0.1", 8080, timeout=5)
payload = json.dumps(
    {
        "title": "Read the HTTP message chapter",
        "priority": "high",
        "status": "in_progress",
    }
)
connection.request(
    "POST",
    "/tasks",
    body=payload.encode("utf-8"),
    headers={"Content-Type": "application/json"},
)
created = connection.getresponse()
print(created.status, created.read().decode("utf-8"))

connection.request("GET", "/tasks?status=in_progress")
listed = connection.getresponse()
print(listed.status, listed.read().decode("utf-8"))
connection.close()
```

18.7 Trace a PATCH

For **PATCH /tasks/1** with `{"status":"done"}`:

1. HTTP parser frames the declared body and parses the target.
2. Router captures `task_id = "1"`.

3. Handler checks body, media type, UTF-8, and JSON.
4. Partial validator accepts only the status field and adds no defaults.
5. Handler adds `updated_at`.
6. Repository generates assignments from its allow-list and binds values.
7. SQLite updates inside a transaction and returns the complete row.
8. Response factory encodes the representation; server frames and sends it.

If any step fails, ownership is clear. A malformed request never reaches routing; an invalid enum never reaches SQL; a missing row becomes a resource 404; an unexpected database error crosses the generic 500 boundary.

Try it

Create three tasks with different status, priority, and due dates. Query combined filters and pagination. Then send an unknown query parameter, repeated limit, invalid date, list instead of object, unsupported media type, and nonexistent ID. Record status and problem type for each.

Chapter summary

The Task application is thin translation code between HTTP and a repository. It validates each input category, owns resource semantics and status choices, protects server-generated fields, and returns stable representations/problems. The complete backend runs without third-party Python packages.

Review questions

1. Why does `build_app` accept a database path instead of using a fixed global database?
2. Which fields can a client write, and which remain server-owned?
3. Why are alternate date spellings rejected even if Python can parse them?
4. At which step does a missing row become an HTTP 404?

Exercise 18.1: End-to-end extension

Add a `GET /tasks/summary` static route returning counts by status. Write the contract first, add a repository aggregate with parameter-free SQL, ensure the static route beats `/tasks/{task_id}`, and test response/problem cases.

Exercise 18.2: Optimistic concurrency design

Without implementing it, design ETag and `If-Match` support for updates. Define validator generation, success and precondition-failure statuses, database atomicity, and behavior when the field is absent.

CHAPTER 19

Testing from functions to hostile streams

The happy path proves almost nothing about a protocol implementation. Confidence comes from testing boundaries: fragmentation, ambiguity, limits, concurrency, rollback, and shutdown.

Learning objectives

- Organize model, parser, application, repository, API, and network tests.
- Test stream fragmentation independently from packet timing.
- Use temporary databases and ephemeral ports for isolated integration tests.
- Harden behavior through explicit malformed-input and concurrency cases.

19.1 A testing pyramid for this project

Fast unit tests cover deterministic functions and objects: Headers, JSON factories, parser feeds, router precedence, validation, and repository operations. Application tests call `app.handle` directly with Request objects. Network tests start a real listener on loopback and use raw sockets or `http.client`.

Most cases belong below the network boundary because unit tests can control every fragment without timing. A smaller set of socket tests confirms that accept, timeouts, keep-alive, serialization, and shutdown connect correctly.

19.2 Parser tests are protocol examples

The parser suite demonstrates both ordinary and hostile messages:

```
from __future__ import annotations

import json
import unittest

from tinyhttp import Headers, HTTPError, Request, Response
from tinyhttp.parser import ParserLimits, RequestParser
```

```
class HeadersTests(unittest.TestCase):
    def test_case_insensitive_multi_value_access(self) -> None:
        headers = Headers([("X-Trace", "one"), ("x-trace", "two")])
        self.assertEqual(headers.get("X-TRACE"), "two")
        self.assertEqual(headers.get_all("x-trace"), ["one", "two"])
        headers.set("X-Trace", "three")
        self.assertEqual(headers.get_all("x-trace"), ["three"])

    def test_json_response_is_utf8(self) -> None:
        response = Response.json({"message": "café"})
        self.assertEqual(json.loads(response.body), {"message": "café"})
        self.assertIn("application/json", response.headers.get("content-type")
    ↪  or "")

    def test_rejects_response_splitting_characters(self) -> None:
        with self.assertRaises(ValueError):
            Headers({"X-Unsafe": "value\r\nX-Injected: yes"})


class ParserTests(unittest.TestCase):
    def test_fragmented_request_and_leftover_request(self) -> None:
        first = (
            b"POST /tasks?tag=a&tag=b HTTP/1.1\r\n"
            b"Host: localhost\r\nContent-Length: 2\r\n\r\n{}"
        )
        second = b"GET /health HTTP/1.1\r\nHost: localhost\r\n\r\n"
        parser = RequestParser()
        data = first + second
        for byte in data:
            parser.feed(bytes([byte]))
            request = parser.pop_request(("127.0.0.1", 1234))
            if byte < 0:  # pragma: no cover - keeps the loop shape explicit
                self.fail("unreachable")
            if request is not None:
                break
        self.assertIsNotNone(request)
        assert request is not None
        self.assertEqual(request.method, "POST")
        self.assertEqual(request.path, "/tasks")
        self.assertEqual(request.query, {"tag": ["a", "b"]})
        self.assertEqual(request.body, b"{}")
        remaining_start = len(first)
        parser.feed(data[remaining_start:])
        following = parser.pop_request()
        self.assertIsNotNone(following)
        assert following is not None
        self.assertEqual(following.path, "/health")

    def test_two_requests_from_one_feed(self) -> None:
        parser = RequestParser()
        parser.feed(
            b"GET /a HTTP/1.1\r\nHost: test\r\n\r\n"
```

```
            b"GET /b HTTP/1.1\r\nHost: test\r\n\r\n"
        )
        self.assertEqual(parser.pop_request().path, "/a")  # type: ignore[
    ↪ union-attr]
        self.assertEqual(parser.pop_request().path, "/b")  # type: ignore[
    ↪ union-attr]

    def test_rejects_duplicate_content_length(self) -> None:
        parser = RequestParser()
        parser.feed(
            b"POST / HTTP/1.1\r\nHost: test\r\n"
            b"Content-Length: 1\r\nContent-Length: 1\r\n\r\nx"
        )
        with self.assertRaisesRegex(HTTPError, "Duplicate") as context:
            parser.pop_request()
        self.assertEqual(context.exception.status, 400)

    def test_rejects_transfer_encoding(self) -> None:
        parser = RequestParser()
        parser.feed(
            b"POST / HTTP/1.1\r\nHost: test\r\n"
            b"Transfer-Encoding: chunked\r\n\r\n"
        )
        with self.assertRaises(HTTPError) as context:
            parser.pop_request()
        self.assertEqual(context.exception.status, 501)

    def test_requires_exactly_one_host(self) -> None:
        parser = RequestParser()
        parser.feed(b"GET / HTTP/1.1\r\n\r\n")
        with self.assertRaises(HTTPError) as context:
            parser.pop_request()
        self.assertEqual(context.exception.code, "invalid_host")

    def test_enforces_body_limit_before_waiting_for_body(self) -> None:
        parser = RequestParser(ParserLimits(body_bytes=3))
        parser.feed(
            b"POST / HTTP/1.1\r\nHost: test\r\nContent-Length: 4\r\n\r\n"
        )
        with self.assertRaises(HTTPError) as context:
            parser.pop_request()
        self.assertEqual(context.exception.status, 413)

    def test_preserves_method_case_and_rejects_bad_percent_escape(self) ->
    ↪ None:
        parser = RequestParser()
        parser.feed(b"get /ok HTTP/1.1\r\nHost: test\r\n\r\n")
        request = parser.pop_request()
        self.assertIsNotNone(request)
        assert request is not None
        self.assertEqual(request.method, "get")

        parser.feed(b"GET /bad%ZZ HTTP/1.1\r\nHost: test\r\n\r\n")
        with self.assertRaises(HTTPError) as context:
```

```
            parser.pop_request()
        self.assertEqual(context.exception.code, "invalid_percent_encoding")

    def test_request_json_reports_content_type_and_syntax(self) -> None:
        request = Request(
            method="POST",
            target="/",
            version="HTTP/1.1",
            headers=Headers(),
            body=b"{}",
            path="/",
            query={},
        )
        with self.assertRaises(HTTPError) as context:
            request.json()
        self.assertEqual(context.exception.status, 415)
        request.headers.set("Content-Type", "application/json")
        request.body = b"{"
        with self.assertRaises(HTTPError) as context:
            request.json()
        self.assertEqual(context.exception.status, 400)


if __name__ == "__main__":
    unittest.main()
```

Feeding one byte at a time is more deterministic than trying to force TCP to produce one-byte reads. Putting two messages in one feed verifies leftover preservation. Declaring an oversized body without sending it verifies early rejection. Duplicate lengths and transfer coding test ambiguity policy.

Every accepted feature should have malformed neighbors. If one Content-Length is accepted, test duplicate, negative, nondecimal, too large, incomplete, and combined transfer encoding. If percent encoding is accepted, test invalid hex and invalid UTF-8.

19.3 Application and API tests

Application tests prove static-route precedence, parameter capture, HEAD fallback, OPTIONS, 404/405, Allow, and middleware nesting without a socket. Task tests use a fresh temporary SQLite file and call the application directly.

```
from __future__ import annotations

import json
import tempfile
import unittest
from pathlib import Path
from urllib.parse import parse_qs, urlsplit

from task_api import build_app
from tinyhttp import Headers, Request

```

```
12
13  class TaskAPITests(unittest.TestCase):
14      def setUp(self) -> None:
15          self.temporary = tempfile.TemporaryDirectory()
16          self.database = Path(self.temporary.name) / "tasks.db"
17          self.app = build_app(self.database)
18
19      def tearDown(self) -> None:
20          self.temporary.cleanup()
21
22      def call(self, method: str, target: str, document=None):
23          parsed = urlsplit(target)
24          body = b""
25          headers = Headers({"Host": "test"})
26          if document is not None:
27              body = json.dumps(document).encode("utf-8")
28              headers.set("Content-Type", "application/json")
29              headers.set("Content-Length", str(len(body)))
30          request = Request(
31              method=method,
32              target=target,
33              version="HTTP/1.1",
34              headers=headers,
35              body=body,
36              path=parsed.path,
37              query=parse_qs(parsed.query, keep_blank_values=True),
38          )
39          return self.app.handle(request)
40
41      def test_full_crud_lifecycle(self) -> None:
42          created = self.call(
43              "POST",
44              "/tasks",
45              {
46                  "title": "Understand TCP streams",
47                  "description": "Practice fragmented reads",
48                  "priority": "high",
49                  "due_date": "2026-12-01",
50              },
51          )
52          self.assertEqual(created.status, 201)
53          self.assertEqual(created.headers.get("Location"), "/tasks/1")
54          task = json.loads(created.body)
55          self.assertEqual(task["status"], "todo")
56
57          fetched = self.call("GET", "/tasks/1")
58          self.assertEqual(fetched.status, 200)
59          self.assertEqual(json.loads(fetched.body)["title"], "Understand TCP
    ↪ streams")
60
61          patched = self.call("PATCH", "/tasks/1", {"status": "done"})
62          self.assertEqual(json.loads(patched.body)["status"], "done")
63
64          replaced = self.call(
```

```
            "PUT",
            "/tasks/1",
            {"title": "Review HTTP", "priority": "low"},
        )
        replaced_task = json.loads(replaced.body)
        self.assertEqual(replaced_task["description"], "")
        self.assertEqual(replaced_task["priority"], "low")

        deleted = self.call("DELETE", "/tasks/1")
        self.assertEqual(deleted.status, 204)
        self.assertEqual(self.call("GET", "/tasks/1").status, 404)

    def test_filter_pagination_and_search(self) -> None:
        self.call("POST", "/tasks", {"title": "Alpha HTTP", "status": "todo"})
        self.call("POST", "/tasks", {"title": "Beta SQL", "status": "done"})
        response = self.call("GET", "/tasks?status=todo&q=HTTP&limit=1&offset=0")
        document = json.loads(response.body)
        self.assertEqual(document["total"], 1)
        self.assertEqual(document["items"][0]["title"], "Alpha HTTP")

    def test_validation_problem_is_structured(self) -> None:
        response = self.call(
            "POST",
            "/tasks",
            {"title": "", "status": "almost", "unexpected": True},
        )
        self.assertEqual(response.status, 422)
        self.assertIn("application/problem+json", response.headers.get("Content-Type") or "")
        problem = json.loads(response.body)
        self.assertEqual(problem["type"], "urn:task-api:problem:validation-error")
        self.assertIn("title", problem["errors"])

    def test_query_validation_and_sql_shaped_search(self) -> None:
        invalid = self.call("GET", "/tasks?limit=1000&unknown=yes")
        self.assertEqual(invalid.status, 422)
        safe = self.call("GET", "/tasks?q=%25%27%20OR%201%3D1--")
        self.assertEqual(safe.status, 200)
        self.assertEqual(json.loads(safe.body)["total"], 0)

    def test_requires_content_length_on_body_endpoint(self) -> None:
        response = self.call("POST", "/tasks")
        self.assertEqual(response.status, 411)


if __name__ == "__main__":
    unittest.main()
```

The CRUD lifecycle test is intentionally stateful within one test: create, read, patch, replace, delete, and confirm absence. Separate tests cover filters, pagination, problem

structure, query validation, SQL-shaped search text, and length requirements. Each test case receives its own database, so order does not matter.

19.4 Real socket integration

```
from __future__ import annotations

import http.client
import json
import socket
import threading
import unittest
from concurrent.futures import ThreadPoolExecutor

from tinyhttp import Application, Request, Response, Server, ServerConfig


class ServerTests(unittest.TestCase):
    def setUp(self) -> None:
        app = Application()

        @app.route("GET", "/hello")
        def hello(request: Request) -> Response:
            return Response.json({"hello": "world"})

        self.server = Server(
            app,
            ServerConfig(port=0, workers=4, queue_size=8, read_timeout=1.0),
        )
        self.thread = threading.Thread(target=self.server.serve_forever, daemon=True)
        self.thread.start()
        self.assertTrue(self.server.wait_until_ready())
        assert self.server.address is not None
        self.host, self.port = self.server.address

    def tearDown(self) -> None:
        self.server.shutdown()
        self.thread.join(timeout=2)

    def test_standard_library_client_and_keep_alive(self) -> None:
        connection = http.client.HTTPConnection(self.host, self.port, timeout=2)
        connection.request("GET", "/hello")
        first = connection.getresponse()
        self.assertEqual(first.status, 200)
        self.assertEqual(json.loads(first.read()), {"hello": "world"})
        connection.request("HEAD", "/hello")
        second = connection.getresponse()
        self.assertEqual(second.status, 200)
        self.assertEqual(second.read(), b"")
        self.assertGreater(int(second.getheader("Content-Length")), 0)
```

```
        connection.close()

    def test_raw_malformed_request_gets_problem_response(self) -> None:
        with socket.create_connection((self.host, self.port), timeout=2) as
    ↪ client:
            client.sendall(b"GET /hello HTTP/1.1\r\n\r\n")
            received = receive_until_close(client)
        self.assertIn(b"400 Bad Request", received)
        self.assertIn(b"application/problem+json", received)

    def test_fragmented_network_request(self) -> None:
        with socket.create_connection((self.host, self.port), timeout=2) as
    ↪ client:
            for part in (
                b"GET /hel",
                b"lo HTTP/1.1\r\nHo",
                b"st: localhost\r\nConnection: close\r\n\r\n",
            ):
                client.sendall(part)
            received = receive_until_close(client)
        self.assertIn(b"200 OK", received)
        self.assertIn(b'{"hello":"world"}', received)

    def test_fifty_concurrent_clients_complete(self) -> None:
        def fetch(_: int) -> int:
            connection = http.client.HTTPConnection(self.host, self.port,
    ↪ timeout=3)
            connection.request("GET", "/hello", headers={"Connection": "close
    ↪ "})
            response = connection.getresponse()
            response.read()
            connection.close()
            return response.status

        with ThreadPoolExecutor(max_workers=50) as pool:
            statuses = list(pool.map(fetch, range(50)))
        self.assertEqual(statuses, [200] * 50)


def receive_until_close(client: socket.socket) -> bytes:
    parts: list[bytes] = []
    while True:
        chunk = client.recv(4096)
        if not chunk:
            return b"".join(parts)
        parts.append(chunk)


if __name__ == "__main__":
    unittest.main()
```

The test configuration requests port zero. After the readiness event, the test reads the actual bound address. Teardown calls shutdown and joins the accept thread even when an assertion fails.

One test uses `http.client` twice on a persistent connection and verifies HEAD body suppression. Another uses raw fragmented writes. Another sends a syntactically complete request without Host and inspects the 400 problem. The concurrency test launches 50 clients with explicit deadlines and requires every one to complete.

> **Pitfall**
>
> Never let a network test wait forever. Put timeouts on clients, wait for server readiness, and stop/join the server in teardown. A test that can hang the suite is itself a reliability defect.

19.5 Failure injection checklist

Before declaring the capstone complete, test:

- connection close before header and before declared body completion;
- bare LF, folded fields, invalid names, control characters, and missing Host;
- unsupported version/method/target form/transfer coding/media type;
- limits at one byte below, exactly at, and one byte above each boundary;
- two buffered requests and a request split at every byte;
- HEAD, 204, OPTIONS, Allow, and Connection close;
- invalid JSON encoding/grammar/top-level type/fields/date/enums;
- not found, PUT defaults, PATCH emptiness, delete twice;
- database constraint and busy failures without partial writes;
- queue saturation, read timeout, peer reset, and shutdown during activity.

Some OS timing failures can be difficult to make deterministic. Prefer injecting a fake or using socket pairs when the unit boundary permits. For the few end-to- end cases, use generous but finite deadlines and assert outcomes rather than sleeping for guessed durations.

19.6 Hardening review

The implementation has several deliberate defenses: loopback default, strict CRLF, Host requirement, framing ambiguity rejection, finite bytes/fields/time, finite workers/queue/connection requests, response-field control validation, media and UTF-8 checks, allow-list domain validation, parameterized SQL, generic 500 responses, and no toy authentication.

Remaining risks include an incomplete general HTTP implementation, limited timeout granularity, no TLS, no rate limiting, no authentication/authorization, no process isolation, minimal signal handling, simple logs, one-process capacity, and no security maintenance process. Naming these is part of hardening.

19.7 One-command verification

From the repository root:

```
python build.py test
```

The helper runs snapshot tests and final tests. `python build.py all` also compiles the book, fails on undefined references and overfull boxes, and copies the approved PDF into `dist`.

Checkpoint

The capstone passes when a clean checkout can run all tests and build the PDF; 50 concurrent local clients finish within deadlines; invalid framing never reaches a handler; temporary databases leave no persistent test state; and the reader can explain every intentional unsupported feature.

Chapter summary

Tests mirror architectural boundaries. Parser tests control exact fragments; application tests bypass sockets; API tests isolate databases; network tests use ephemeral ports and deadlines. Hardening is the combination of strict parsing, finite resources, safe errors, storage discipline, and honest limitations.

Review questions

1. Why is byte-at-a-time parser feeding better than hoping TCP fragments that way?
2. Which properties can only a real socket integration test establish?
3. Why must tests use temporary database paths and ephemeral ports?
4. Name five remaining production risks after the suite passes.

Exercise 19.1: Boundary expansion

Choose one configured limit and add three tests: just below, exactly at, and just above it. State whether rejection can occur before the complete request arrives.

Exercise 19.2: Mini threat model

List assets, entry points, trust boundaries, attacker capabilities, existing controls, and missing controls for the Task API. Prioritize five risks by impact and likelihood without pretending the educational server will be deployed.

CHAPTER 20

From learning server to production ecosystem

Building the stack makes mature web infrastructure less magical. It should also make us humble: production systems solve a broader, continuously changing problem.

Learning objectives

- Explain the roles of WSGI, ASGI, production servers, frameworks, and reverse proxies.
- Identify the authentication, deployment, observability, and operations work omitted here.
- Transfer low-level lessons to everyday backend development.
- Complete a final assessment and choose safe next projects.

20.1 What frameworks no longer hide

When a mature framework gives a request object, you now know the work behind it: stream reads, framing, syntax, field normalization, limits, target parsing, routing, content decoding, exception boundaries, and response serialization. When a production server exposes worker and timeout settings, you can connect them to capacity and slow-client behavior rather than treating them as arbitrary knobs.

This knowledge changes debugging. A 400 before application logs suggests an earlier parser or proxy. A 405 differs from a missing route. A hanging client can indicate missing length or an incomplete stream. A database error belongs after valid HTTP and domain input.

20.2 WSGI and ASGI

WSGI defines a synchronous interface between Python web servers and applications. The server handles network protocol work and calls an application with request environment data; the application returns status, fields, and body iterable. It enabled portable synchronous applications across many servers.

ASGI generalizes the boundary for asynchronous and long-lived interactions. An ASGI application receives a connection scope and exchanges event messages through `receive` and `send` callables. It supports HTTP, WebSocket, lifespan events, and asynchronous application work.

Neither interface is "HTTP itself." They are Python application/server contracts above protocol parsing. Our `Application.handle(Request) → Response` is a smaller educational boundary with similar separation but no compatibility claim.

20.3 A typical production path

A deployment might place these responsibilities in layers:

1. edge network and firewall absorb or filter unwanted traffic;
2. load balancer or reverse proxy terminates TLS, enforces limits, and routes;
3. production WSGI/ASGI server manages processes, workers, timeouts, and protocol details;
4. framework supplies routing, validation integration, middleware, and application tools;
5. application implements resource and authorization rules;
6. database/cache/message systems provide durable or shared state;
7. logs, metrics, traces, alerts, deployment, backups, and incident processes operate it.

Each boundary has timeout and size policy. If an outer proxy allows a 100 MiB body while an inner service allows 1 MiB, rejection occurs later after wasting bandwidth. If an inner timeout exceeds every outer timeout, work can continue after the client has already received a gateway error. Production configuration is an end-to-end design.

20.4 Authentication and authorization

Authentication establishes a principal; authorization decides what that principal may do. Adding users safely requires choices about password hashing, multi-factor authentication, account recovery, session or token lifetime, revocation, CSRF, credential storage, TLS, rate limiting, audit, privacy, and authorization at resource boundaries.

Use established framework and identity-system capabilities. Do not hash passwords with a fast general-purpose hash, invent token signatures, store secrets in source, or trust identity fields from untrusted proxies. Even a correct cryptographic primitive can be assembled into an insecure authentication system.

20.5 Observability and operations

Access logs answer individual request questions. Metrics reveal rates, latency distributions, saturation, queue depth, error classes, and dependency health. Traces connect work across services. Alerts should correspond to user impact and actionable operator decisions.

Operations also includes reproducible builds, dependency updates, vulnerability response, configuration management, least privilege, secret rotation, database migrations, backups, restore tests, capacity planning, rolling deploys, rollback, and incident learn-

ing. A server is not production-ready merely because it answers requests quickly in a benchmark.

20.6 Safe next projects

Good continuations preserve one learning question at a time:

- implement chunked request decoding with a written grammar and adversarial tests;
- add TLS only through Python's established `ssl` module and local certificates, studying verification rather than implementing cryptography;
- adapt the Task application to WSGI, comparing boundaries with tinyhttp;
- adapt it to ASGI and measure slow concurrent clients;
- add conditional updates using ETag and `If-Match`;
- replace offset pagination with a documented stable cursor;
- run the API behind a local reverse proxy and trace forwarded metadata trust.

Do not expose `tinyhttp` publicly as the next project. The goal is transfer of understanding to maintained tools, not a new production server ecosystem.

20.7 Final assessment

The intern should complete four demonstrations without copying prose:

1. On a whiteboard, trace an HTTPS Task POST from URL to DNS, TCP/TLS, HTTP, parser, router, validation, transaction, and response.
2. Starting from a blank file, create a one-response socket server and then describe every missing property required for the final subset.
3. Diagnose five supplied failures using wire bytes, logs, status, and tests, locating the owning layer before changing code.
4. Extend the Task API with one read-only aggregate endpoint, including contract, repository method, handler, unit tests, network test, and docs.

Assessment emphasizes explanation and evidence. A working patch without a clear boundary model is incomplete; an elegant explanation without tested bytes is also incomplete.

20.8 The central lesson

An HTTP backend is a sequence of contracts. Bytes become a framed message; syntax becomes typed internal state; paths and methods become a resource operation; untrusted values become validated data; a transaction becomes durable state; a response value becomes bytes again. Reliability comes from making each contract explicit, bounded, testable, and honest about what it does not support.

Chapter summary

WSGI, ASGI, mature servers, frameworks, proxies, identity systems, and operational platforms divide responsibilities that our small project exposes directly. Low-level understanding improves everyday design and debugging, while production readiness requires

maintained implementations and organizational processes far beyond a successful local API.

Review questions

1. At what layer do WSGI and ASGI sit relative to HTTP parsing?
2. Why must timeout policy be coordinated across proxies and applications?
3. What work turns authentication from a token check into a system?
4. Which tinyhttp lessons transfer directly to a mature framework?

Exercise 20.1: Production comparison

Choose one maintained Python framework and production server. Map their request, response, routing, middleware, worker, limit, timeout, and shutdown concepts to tinyhttp. Cite official documentation and identify capabilities with no tinyhttp equivalent.

Exercise 20.2: Capstone retrospective

Write a two-page engineering note: three assumptions you held before the book, three experiments that changed your model, the most security-sensitive parser decision, and the production boundary you would defend most strongly in review.

Part IV

Appendices

APPENDIX A

Setup and troubleshooting

A.1 Required tools

The runnable code requires Python 3.12 or newer. The PDF build additionally requires XeLaTeX, `latexmk`, and the Noto Serif, Noto Sans, and JetBrains Mono fonts. No third-party Python package is required.

Confirm Python:

```
python --version
```

On systems where `python` refers to an older interpreter, try `python3`. On Windows, the Python Launcher commonly uses `py -3.12` or newer. Use the same launcher consistently in later commands.

A.2 Optional virtual environment

There are no packages to install, but a virtual environment can keep tooling habits consistent.

Task	Linux/macOS shell	Windows PowerShell
Create	`python -m venv .venv`	`py -m venv .venv`
Activate	`source .venv/bin/activate`	`.venv\Scripts\Activate.ps1`
Deactivate	`deactivate`	`deactivate`

If PowerShell execution policy blocks activation, the environment can still be used by invoking `.venv\Scripts\python.exe` directly. Do not weaken a machine-wide policy merely for this project.

A.3 Repository map

- `book`: LaTeX source and generated build intermediates;
- `code/stages`: nine frozen learning milestones and snapshot tests;
- `code/final/tinyhttp`: reusable educational framework;
- `code/final/task_api`: SQLite application and CLI;
- `code/final/tests`: parser, application, API, and socket tests;

- `dist`: approved compiled PDF;
- `build.py`: cross-platform test/build helper.

A.4 Run tests

From the repository root:

```
python build.py test
```

To isolate suites:

```
cd code/stages
python -m unittest discover -s tests -v

cd ../final
python -m unittest discover -s tests -v
```

Local socket tests may be restricted in a sandboxed development environment. Grant only loopback-test capability; do not disable unrelated security controls.

A.5 Build the book

```
python build.py book
```

The helper runs `latexmk -xelatex`, inspects the log for undefined references and overfull boxes, and copies the final artifact to `dist/build-http-from-scratch.pdf`. Use `python build.py all` for tests plus PDF. Use `python build.py clean` to remove generated LaTeX files and the distribution PDF.

The manuscript uses `listings`, not `minted`; shell escape and Pygments are unnecessary. Code listings are read directly from the runnable source, so the relative directory structure must remain intact.

A.6 Run the API

```
cd code/final
python -m task_api --database tasks.db
```

Useful options:

```
python -m task_api --host 127.0.0.1 --port 9000 \
  --database practice.db --workers 4 --no-keep-alive
```

The backslash line continuation is for a Unix-like shell. In PowerShell, put the command on one line or use its backtick continuation carefully. The options are otherwise identical.

Stop with Ctrl+C. The database file and possible `-wal`/`-shm` companion files belong to the running SQLite database. Stop the process before moving or deleting a practice database.

A.7 Common failures

A.7.1 Address already in use

Another process is listening on the selected port, or a previous process is still running. Choose another port or stop the exact process you started. Do not kill unrelated processes by a broad name match.

A.7.2 Connection refused

Confirm the server printed its listening URL, the client uses the same address family and port, and the process has not exited with a traceback. A server bound to loopback cannot be reached through another machine's network address.

A.7.3 Client waits forever

Inspect response framing. A wrong Content-Length can make a client wait for bytes that will never arrive. In learning snapshots without timeouts, a server may also wait for an incomplete request delimiter.

A.7.4 400 invalid Host

Raw HTTP/1.1 requests must contain exactly one nonempty Host field. Standard clients add it automatically. When using a raw socket, include `Host: localhost` followed by CRLF.

A.7.5 411 on POST, PUT, or PATCH

Body-required Task endpoints require Content-Length. Standard HTTP clients usually calculate it when given bytes. A raw client must calculate the UTF-8 byte length, not character count.

A.7.6 415 Unsupported Media Type

Set `Content-Type: application/json`. An `Accept` field describes the desired response and does not describe the request body.

A.7.7 SQLite database is locked

Stop stray API processes using the file. Keep transactions short. The repository configures a busy timeout, but a long-held external transaction can exceed it. Do not edit the database with a tool that leaves a write transaction open.

A.7.8 Font or LaTeX error

Confirm XeLaTeX can locate all three configured fonts and that the TeX distribution contains memoir, fontspec, TikZ, tcolorbox, listings, hyperref, bookmark, and cleveref. The local build intentionally avoids automatic network installation.

A.8 A debugging worksheet

For any failure, record:

1. exact command and current directory;
2. Python and tool versions;
3. selected hostname/address/port;
4. raw request start line, fields, and declared body length;
5. first observed error, status, or traceback;
6. earliest layer that could produce that evidence;
7. smallest reproducible case and one falsifiable next hypothesis.

This worksheet prevents random edits and preserves evidence for mentor review.

APPENDIX B

HTTP and framework quick reference

B.1 Request and response shapes

```
METHOD /path?query=value HTTP/1.1
Host: authority
Field-Name: field value
Content-Length: number-of-body-bytes

optional body bytes
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: number-of-body-bytes
Connection: keep-alive

optional body bytes
```

Every displayed line ending above is CRLF. Exactly one empty line terminates the field section. Content-Length counts octets in the framed body.

B.2 Methods

Method	Safe	Idempotent	Meaning in this project
GET	yes	yes	Retrieve collection, task, or health representation
HEAD	yes	yes	GET fields without response content
POST	no	no	Create a task under the collection
PUT	no	yes	Replace all client-writable fields of one task
PATCH	no	project-specific	Assign supplied fields of one task
DELETE	no	yes	Remove one task

Method	Safe	Idempotent	Meaning in this project
OPTIONS	yes	yes	Return supported methods through Allow

B.3 Statuses used

Code	Name	Use
200	OK	Successful retrieval or update with representation
201	Created	Task created; includes Location
204	No Content	Successful deletion or OPTIONS; no body
400	Bad Request	Malformed request, target, encoding, or JSON
404	Not Found	No route or task resource
405	Method Not Allowed	Path exists for other methods; includes Allow
408	Request Timeout	Partial request exceeded read timeout
411	Length Required	Body-required endpoint lacks Content-Length
413	Content Too Large	Declared body exceeds configured maximum
414	URI Too Long	Request line/target exceeds configured maximum
415	Unsupported Media Type	Endpoint requires application/json
422	Unprocessable Content	Task or query fields fail domain validation
431	Request Header Fields Too Large	Header bytes or field count exceeds limit
500	Internal Server Error	Unexpected application failure; detail hidden
501	Not Implemented	Method or transfer encoding is outside subset
503	Service Unavailable	Bounded connection queue is full
505	HTTP Version Not Supported	Request is not HTTP/1.1

B.4 Selected fields

Field	Role
Host	Required target authority in HTTP/1.1 requests
Content-Type	Media type of message content
Content-Length	Decimal number of framed body bytes
Transfer-Encoding	Transfer coding; explicitly rejected by this server
Connection	Requests or announces close/keep-alive policy
Allow	Methods supported for a resource
Location	URI of created resource
Date	Server response origination date
Accept	Client preferences for response media types; explained, not negotiated here
Authorization	Credentials under a scheme; not implemented

Field	Role
Cache-Control	Cache directives; explained, not implemented
ETag	Representation validator; proposed extension only

B.5 Default server limits

Policy	Default
Bind address	`127.0.0.1`
Port	8080 (zero allowed in tests)
Workers	16
Application queue	64 connections
Listen backlog	128
Read/idle timeout	5 seconds
Requests per persistent connection	100
Request line	8 KiB
Header section	32 KiB
Header fields	100
Body	1 MiB

B.6 Task endpoints

Endpoint	Success	Result
`GET /health`	200	`{"status":"ok"}`
`POST /tasks`	201	Created task and Location
`GET /tasks`	200	Filtered paginated collection
`GET /tasks/{id}`	200	One task
`PUT /tasks/{id}`	200	Fully replaced writable state
`PATCH /tasks/{id}`	200	Partially updated state
`DELETE /tasks/{id}`	204	No content

B.7 Problem document

```
{
  "type": "urn:task-api:problem:validation-error",
  "title": "Validation failed",
  "status": 422,
  "detail": "One or more values failed validation.",
  "instance": "/tasks",
  "errors": {"title": "Must be a non-empty string."}
}
```

B.8 Framework public API

- `Headers(values)`, `add`, `set`, `remove`, `get`, `get_all`, `copy`, `items`;

- Request message fields: `method`, `target`, `version`, `headers`, and `body`;
- Request routing helpers: `path`, `query`, `client`, `path_params`, and `json()`;
- `Response(status, body, headers)`, `Response.json`, and `Response.problem`;
- `Application.route`, `add_middleware`, and `handle`;
- `ServerConfig` and `Server.serve_forever`, `wait_until_ready`, `shutdown`;
- `HTTPError(status, detail, code, title, errors, type_uri)`.

B.9 Parser decision sequence

1. Is a complete CRLF–CRLF header section buffered?
2. Are line and total header sizes within limits?
3. Are all line endings strict and request-line components valid?
4. Is the version exactly HTTP/1.1?
5. Are field names/values legal and exactly one Host present?
6. Is framing unambiguous and supported?
7. Is declared body size within limit and fully buffered?
8. Can origin-form target, percent encoding, path, and query be decoded?
9. Consume exactly one message and preserve the remainder.

B.10 Code review checklist

- Every network wait has a bound or documented owner.
- Every allocated resource closes on success and failure.
- Every input collection has size/count limits.
- HTTP and JSON byte/text conversions name their encoding.
- Framing ambiguity closes the connection.
- Handlers return Response rather than touching sockets.
- SQL structure is code-owned and values are bound parameters.
- 500 responses reveal no internal exception detail.
- Tests include fragments, combined messages, boundaries, and cleanup.
- Documentation states unsupported features explicitly.

APPENDIX C

Exercise solutions and mentor notes

These are reference solutions, not the only valid wording. The important quality is an explicit boundary, justified protocol rule, and observable evidence. A mentor should ask the intern to explain differences before showing this appendix.

C.1 Foundations

Exercise 1.1 — Draw the journey

For `http://127.0.0.1:8080/health`, no DNS lookup is required because the host is already an address. IP routing selects loopback; packets do not leave the machine. There is a TCP handshake, no TLS handshake, then HTTP request parsing, routing to health, response serialization, and TCP close or reuse. Through an HTTPS reverse proxy, add DNS, a route across one or more networks, TLS termination at the proxy, one HTTP connection from client to proxy, and usually another proxy-to-application connection. Mark the proxy as a trust boundary.

Exercise 1.2 — Classify failures

“Name not known” belongs to DNS. “Connection refused” is TCP evidence that the destination actively rejected the selected endpoint, commonly because no listener exists. Certificate mismatch is TLS identity verification. A valid 400 is HTTP/application parsing evidence. “Database busy” occurs after enough HTTP and routing succeeded to reach storage. Confirm with resolver output, the exact endpoint, TLS verification detail, raw response, and structured server logs.

Exercise 2.1 — Design a frame

One valid design is a four-byte unsigned big-endian length followed by that many UTF-8 bytes, repeated until the connection closes. Length zero represents an empty note; close between frames ends the sequence; close before the declared length is an incomplete-note error. Reject lengths above a configured maximum before allocating. A delimiter design is possible, but then delimiter escaping or prohibition must be specified.

Exercise 2.2 — Count the bytes

`hello` is five characters and five UTF-8 bytes. `café` is four characters and five UTF-8 bytes when the last character is the single code point U+00E9. A visually identical decomposed form can have five characters/code points and six bytes. Network length is the length of the final encoded bytes, not the visual character count.

Exercise 3.1 — Endpoint inventory

Example: listener `127.0.0.1:8080`; first client `127.0.0.1:53001`; TCP tuple client-to-server is `127.0.0.1:53001 → 127.0.0.1:8080`. A second client might use port 53002. Protocol and server endpoint remain; the client source port changes, allowing the operating system to distinguish connections.

Exercise 3.2 — Resolution narrative

The stub asks a recursive resolver. On a miss, that resolver asks root for a delegation, a TLD server for the authoritative delegation, and the authoritative server for the record; it caches referrals and answer under TTLs. On a complete cache hit, the resolver answers immediately and all upstream DNS round trips disappear. The client may also have a local cache, removing even the resolver exchange.

Exercise 4.1 — Stream outcomes

Legal reads include 800 once; 600 then 200; 100 repeated eight times; or 50, 300, 7, 443. TCP can split anywhere. The CRLF–CRLF delimiter ends fields, then Content-Length (200 in this example) determines body completion. The header's physical segment boundaries are irrelevant.

Exercise 4.2 — Slow client budget

Sixteen clients can each occupy one worker while never completing a header; the seventeenth remains queued or unaccepted. A reasonable learning policy is a five-second read timeout, 64-slot queue, and best-effort 503 on overflow. Shorter timeouts free capacity faster but reject slower legitimate clients; larger queues absorb bursts but consume descriptors and increase waiting latency.

Exercise 5.1 — Method audit

POST creates; GET reads; PUT replaces; PATCH partially modifies; DELETE removes. GET, PUT, and DELETE are idempotent by standard semantics and are candidates for retry after uncertain transport failure, though preconditions and concurrent changes matter. POST creation is not safely retried without an idempotency key. PATCH safety depends on the patch semantics: assignment of a fixed value is idempotent, while "increment by one" is not.

Exercise 5.2 — Choose the result

Creation: 201 with Location. Successful no-body deletion: 204. Malformed JSON: 400. Unknown task: 404. Known path/wrong method: 405 plus Allow. Unsupported media: 415. Invalid fields: 422 with field errors. Unexpected database failure: 500 with generic client detail and internal logging.

Exercise 6.1 — Frame two requests

A GET ends at the second CRLF of its empty line; the next method byte follows immediately. A POST boundary is `header_end + Content-Length`. There is no extra blank line after the body. A parser must retain the next method bytes in its connection buffer.

Exercise 6.2 — Ambiguity review

If a proxy honors Transfer-Encoding while an origin honors Content-Length, the proxy can treat chunk syntax as the first body while the origin consumes four raw bytes. Their next-request positions differ. Attacker-controlled trailing bytes may become a second request for one recipient. The correct project policy is reject the combination and close.

Exercise 7.1 — Theory gate

A complete diagram includes URL parsing, DNS, routed IP, TCP, optional TLS at a proxy, HTTP bytes, parser limits/framing, request object, router parameters, JSON/domain validation, parameterized SQL transaction, response object, HTTP serialization, and the reverse byte path. Mark client/proxy and HTTP/application boundaries as untrusted; mark bytes-to-text conversions at request line/fields, percent-decoded target, JSON UTF-8, and response JSON.

Exercise 7.2 — Feature classification

Implemented: Content-Length, GET, ordinary repeated fields, JSON, and bounded keep-alive. Rejected: HTTP/2 at the HTTP/1 parser, chunked/any Transfer-Encoding, duplicate Content-Length, and unsupported methods/versions. Explanation-only: TLS and authentication. Rejection is a response/closed connection or a separate deployment boundary, never silent guessing.

C.2 Building the server

Exercise 8.1 — Predict before running

The stage answers every received byte sequence with the same 200 because it does not parse. A changed path and POST therefore still succeed. Sending a line is enough to unblock recv. Connecting and sending nothing blocks forever. A second client may complete its handshake and wait in the listen backlog, but the process serves only the first and then exits.

Exercise 8.2 — Resource lifetime

The listener exists from socket creation through the outer `with`. The OS queues connection state after listen. Accept returns a new descriptor owned by the inner `with`; closing it sends/causes TCP teardown independently of the listener. Leaving either block closes its descriptor even during an exception.

Exercise 9.1 — Fragment table

For a request of length n, there are $n - 1$ interior two-fragment divisions. Before the second fragment, the parser returns incomplete because no full final delimiter exists (unless the division is after the entire header). After both, the accumulated bytes are

identical, so the parsed result must be identical. This property is ideal for a loop-driven unit test.

Exercise 9.2 — Limit placement

Request-line limit can be enforced when CRLF arrives or the buffer exceeds the limit before CRLF. Header-byte limit applies while searching for CRLF–CRLF. Field count applies after lines are available. Body limit applies immediately after a valid declared length, before body bytes arrive. These early checks avoid waiting or allocating for already-invalid messages.

Exercise 10.1 — Adversarial table

Valid examples should include GET, zero-length POST, repeated non-framing fields, and percent-encoded Unicode. Incomplete examples include partial request line, partial field delimiter, declared body short by one, and second request prefix. Invalid examples include missing Host, duplicate length, transfer coding, and invalid percent escape. Expected outcomes are Request, None, or HTTPError with 400/413/431/501 as appropriate.

Exercise 10.2 — Invariant proof

Before pop, the buffer is an unconsumed prefix of one or more stream messages. After success, it contains exactly the unconsumed suffix after one complete message. Header end identifies the body start; validated Content-Length identifies the first byte after the body. Deleting their sum therefore removes neither too few nor too many bytes.

Exercise 11.1 — Serializer matrix

GET/200: Content-Length equals body and body is sent. HEAD/200: same length but zero transmitted content. GET/204 and GET/304: no content and this serializer removes Content-Length. GET/500: problem JSON length and body are sent. All have a status line and connection policy.

Exercise 11.2 — Injection attempt

`Headers("X-Test": "ok`
`r`
`nInjected: yes")` raises ValueError. Percent encoding is representation-specific and recipients do not generally decode it in field lines; applying it blindly changes legitimate values and still leaves other control-character contexts. Validate the grammar at the field boundary.

Exercise 12.1 — Route conflict table

Static `/tasks/statistics` must beat `/tasks/{task_id}`. GET on both resolves; POST on the statistics path returns 405 if only GET exists; an unknown two-segment path can match the parameter route; an unknown three-segment path is 404. For every matched path, allowed includes OPTIONS and includes HEAD when GET exists.

Exercise 12.2 — Nested resource

Use `/projects/{project_id}/tasks` and `/projects/{project_id}/tasks/{task_id}`. The router captures both strings. Application code converts IDs, checks posi-

tivity, confirms project and task existence, and ensures the task belongs to the project. Authorization also belongs after routing.

Exercise 13.1 — Error taxonomy

HTTP line/framing errors belong to parser and produce 400/4xx/501 with close. Malformed JSON belongs to Request JSON and is 400. Invalid fields belong to Task validation and are 422. Missing task is resource logic and 404. SQL constraint unexpectedly reached or programmer exception is 500 and logged. Only expected public errors carry detailed client-safe messages.

Exercise 13.2 — Request ID middleware

Accept an incoming ID only from a trusted policy, for example 1–64 ASCII letters, digits, hyphen, and underscore; otherwise generate a UUID. Put it in request context and an `X-Request-ID` response field. Use structured logging so it is a value, not format text. Do not accept CR/LF, unlimited length, or arbitrary Unicode control characters.

Exercise 14.1 — Capacity timeline

Clients 1–2 occupy workers; 3–5 fill the queue; 6 receives 503/close. When one worker times out, it closes its slow client and immediately takes client 3, leaving two queued slots used. Exact TCP accept backlog state can exist before the application queue and should be drawn separately.

Exercise 14.2 — Concurrency test

Start on port zero, wait for readiness, then use a 50-worker executor. Each client sets a finite timeout, sends GET with Connection close, consumes the response, and returns status. Collect with a global deadline. With the default 16/64 capacity, all 50 should be 200 locally; a deliberately smaller queue may allow 200 or 503, but never hangs or malformed responses.

Exercise 15.1 — Add a route safely

Register GET `/echo/{value}`. Read the captured value and `request.query.get("tag", [])`; return both through Response.json. Tests should include UTF-8 percent encoding, `?tag=` as an empty string, repeated tags preserving order, HEAD with no transmitted body in integration, OPTIONS with Allow, and POST returning 405.

Exercise 15.2 — Boundary review

Example parser contract: feed accepts bytes and may raise a limit error; pop returns one Request, None, or HTTPError; its invariant is an unconsumed stream prefix; it never discards following bytes. Compare with tests for fragmentation, combined requests, framing conflicts, and limits. A useful missing case is close during a declared body, tested at the server reader boundary.

C.3 Backend and production bridge

Exercise 16.1 — Contract examples

Examples must include Host, JSON media type, byte-correct length, status, response media type, and body. Creation returns 201/Location; list returns the envelope; individual

reads/updates return task JSON; delete returns 204. Error fixtures cover 400, 404, 405/Allow, 411, 413, 415, 422 field errors, and generic 500.

Exercise 16.2 — completed_at design

Make it server-owned and nullable. Transition into done sets current UTC if null; transition out clears it; repeated done assignment preserves or deliberately refreshes it according to the written contract. Add a nullable schema column and state/status consistency check if expressible. Include it in representations, exclude client writes, migrate version, and preserve old clients that ignore it.

Exercise 17.1 — Constraint failure

Calling create with status `almost` raises an integrity error and the context manager rolls back, leaving row count unchanged. HTTP validation remains necessary because it produces a stable 422 with field detail; a raw database exception would otherwise cross the generic 500 boundary and waste database work.

Exercise 17.2 — Query plan

An equality predicate on indexed status can use `tasks_status_idx`. A pattern `%word%` cannot seek from an ordered B-tree prefix, so SQLite normally scans rows. Full-text search or a different index strategy is needed at scale; parameterization remains required either way.

Exercise 18.1 — End-to-end extension

Register GET `/tasks/summary` before or independently of parameter order; static precedence ensures it wins. Repository SQL groups by status and fills missing enum values with zero in application output. Test all-zero, mixed counts, HEAD, OPTIONS, and wrong method. The handler returns one JSON object and performs no identifier conversion.

Exercise 18.2 — Optimistic concurrency design

Generate a strong ETag from a version column or canonical representation. GET returns it. PUT/PATCH requires or optionally accepts If-Match; a mismatch returns 412 Precondition Failed. The database update includes `WHERE id=? AND version=?`, increments version atomically, and distinguishes missing resource from stale version. Define policy for absent If-Match before implementation.

Exercise 19.1 — Boundary expansion

For a 1 MiB body limit, declare lengths $limit - 1$, $limit$, and $limit + 1$. The first two remain acceptable if fully received; the last is rejected as soon as fields are parsed, without receiving body bytes. Repeat at a small test limit to avoid allocating large fixtures.

Exercise 19.2 — Mini threat model

Assets include task data, host capacity, database integrity, and logs. Entry points are TCP bytes, HTTP fields/body/query/path, CLI/database path, and local file access. Controls include loopback, strict parser, limits, parameterized SQL, and safe 500. Major missing controls include identity/authorization, TLS, rate limits, process isolation, backup, and operational monitoring. Public exposure would make unauthorized data access and resource exhaustion top risks.

Exercise 20.1 — Production comparison

A sound comparison maps server protocol parsing to a production server; Request and Response to framework types; route decorator and middleware to their mature equivalents; worker/timeouts/shutdown to server settings; SQLite repository to the framework's database layer. Capabilities absent here commonly include HTTP/2, TLS, streaming, async, multipart forms, cookies, sessions, authentication, dependency injection, schema validation, and deployment tooling.

Exercise 20.2 — Capstone retrospective

A strong note names specific prior beliefs, cites reproducible experiments, and connects the most security-sensitive decision to framing agreement. It should defend at least the boundary between production protocol server and application: use maintained parsers/TLS/workers while keeping domain logic independently tested. Personal examples vary; unsupported confidence without evidence does not.

APPENDIX D

Glossary

Accept	Create a connected socket from a listening socket's pending connection.
Address	A network-layer locator associated with an interface and routing context.
Application server	A process that hosts application code behind a defined interface.
ASGI	Asynchronous Server Gateway Interface for Python applications and servers.
Backlog	An operating-system limit/hint for pending connections to a listener.
Backpressure	Mechanisms that prevent producers from overwhelming finite consumers.
Bind	Associate a socket with a local address and port.
Body	HTTP/1.1 message bytes after the field-section delimiter, as framed by rules.
Byte	Eight bits; the unit exposed by Python socket reads and writes.
Cache	Component that stores responses for protocol-governed reuse.
Client	The role that initiates a request in an interaction.
Connection	Stateful transport relationship between endpoints.
Content	HTTP message data after transfer coding is removed.
Content negotiation	Selecting a representation using request preferences and availability.
CRLF	Carriage return followed by line feed, bytes 13 and 10.
CRUD	Create, read, update, delete; a data-operation mnemonic, not an HTTP specification.
Datagram	A message-oriented transport unit, as exposed by UDP.
DNS	Distributed hierarchical Domain Name System.

Endpoint	An address and transport port, plus protocol context.
Encoding	Mapping between abstract information such as text and bytes.
Field	Named HTTP metadata with field-specific syntax and semantics.
Flow control	Sender control intended to avoid overwhelming the receiver.
Forward proxy	Intermediary selected on behalf of a client.
Frame	A bounded protocol unit; often specifically a link-layer unit.
Framing	Rules that locate message boundaries in bytes.
Gateway	Intermediary that fronts or translates another system.
Handler	Application callable that turns a Request into a Response.
Header section	Start-line-following HTTP field lines ending at an empty line.
Host	HTTP/1.1 field identifying target authority.
HTTP	Stateless application-level request/response protocol family.
HTTPS	HTTP whose communication is secured for the HTTPS origin rules, commonly with TLS.
Idempotent	Having the same intended effect when repeated as when performed once.
Intermediary	Proxy, gateway, cache, or tunnel participant between endpoints.
IP	Internet Protocol, responsible for addressed packet forwarding.
JSON	Text data-interchange syntax used for Task representations.
Keep-alive	Reuse of a connection for more than one HTTP exchange.
Latency	Elapsed delay across a defined operation or path.
Listener	Socket configured to accept incoming transport connections.
Loopback	Host-internal network path, represented by 127.0.0.1 or ::1.
Media type	Identifier describing the format/nature of content.
Method	Case-sensitive HTTP token expressing request semantics.
Middleware	Callable that wraps downstream request handling.
NAT	Translation of network addresses and often ports across routing contexts.
Origin	Scheme, host, and port tuple governing an HTTP resource namespace.
Packet	Network-layer unit forwarded by IP.
Parameterization	Supplying SQL values separately from statement structure.

Parser	Component that validates and transforms wire syntax into structured state.
Port	Sixteen-bit transport identifier used in TCP/UDP endpoints.
Problem details	Structured HTTP error representation standardized by RFC 9457.
Protocol	Shared rules for communication between independent participants.
Representation	Transferable current or intended state of a resource plus metadata.
Resource	Conceptual target identified by a URI.
REST	Architectural style with constraints including uniform interface and statelessness.
Reverse proxy	Intermediary accepting traffic on behalf of origin applications.
Route	Mapping from method/path shape to a handler.
Router	Component that resolves requests against registered routes.
Safe method	Method whose defined semantics are read-only.
Segment	Common name for a TCP protocol data unit; also a path component by context.
Server	Role that accepts requests and produces responses.
Socket	Operating-system interface/object for network communication.
Stateless	Requiring each request to carry the context needed for its interpretation.
Status code	Three-digit HTTP response result code.
Stream	Ordered sequence of bytes without application message boundaries.
TCP	Reliable ordered byte-stream transport protocol.
TLS	Protocol providing protected channels and authenticated identity under verification.
Transaction	Group of database operations committed atomically or rolled back.
URI	Uniform Resource Identifier.
URL	URI that identifies a resource through a scheme with access semantics.
UTF-8	Variable-length Unicode encoding compatible with ASCII for its first 128 values.
Validation	Checking data against the grammar and rules owned by a layer.

WAL	SQLite write-ahead logging mode.
Worker	Execution unit handling queued connections or requests.
WSGI	Synchronous Python Web Server Gateway Interface.

References and further reading

Bibliography

[1] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Semantics*. RFC 9110, June 2022. https://www.rfc-editor.org/rfc/rfc9110.html

[2] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Caching*. RFC 9111, June 2022. https://www.rfc-editor.org/rfc/rfc9111.html

[3] R. Fielding, M. Nottingham, and J. Reschke. *HTTP/1.1*. RFC 9112, June 2022. https://www.rfc-editor.org/rfc/rfc9112.html

[4] M. Thomson and C. Benfield. *HTTP/2*. RFC 9113, June 2022. https://www.rfc-editor.org/rfc/rfc9113.html

[5] M. Bishop. *HTTP/3*. RFC 9114, June 2022. https://www.rfc-editor.org/rfc/rfc9114.html

[6] W. Eddy, editor. *Transmission Control Protocol (TCP)*. RFC 9293, August 2022. https://www.rfc-editor.org/rfc/rfc9293.html

[7] T. Berners-Lee, R. Fielding, and L. Masinter. *Uniform Resource Identifier (URI): Generic Syntax*. RFC 3986, January 2005. https://www.rfc-editor.org/rfc/rfc3986.html

[8] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, August 2018. https://www.rfc-editor.org/rfc/rfc8446.html

[9] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259, December 2017. https://www.rfc-editor.org/rfc/rfc8259.html

[10] E. Wilde, S. Dalal, and M. Nottingham. *Problem Details for HTTP APIs*. RFC 9457, July 2023. https://www.rfc-editor.org/rfc/rfc9457.html

[11] R. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine, 2000. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[12] Python Software Foundation. *socket — Low-level networking interface*. https://docs.python.org/3/library/socket.html

[13] Python Software Foundation. *Socket Programming HOWTO*. https://docs.python.org/3/howto/sockets.html

[14] Python Software Foundation. *sqlite3 — DB-API 2.0 interface for SQLite databases*. https://docs.python.org/3/library/sqlite3.html

[15] SQLite Project. *SQLite Documentation*. https://www.sqlite.org/docs.html

[16] OWASP Foundation. *REST Security Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

[17] Python Software Foundation. *PEP 3333 — Python Web Server Gateway Interface v1.0.1*. https://peps.python.org/pep-3333/

[18] ASGI Documentation. *ASGI Specification*. https://asgi.readthedocs.io/en/latest/specs/main.html